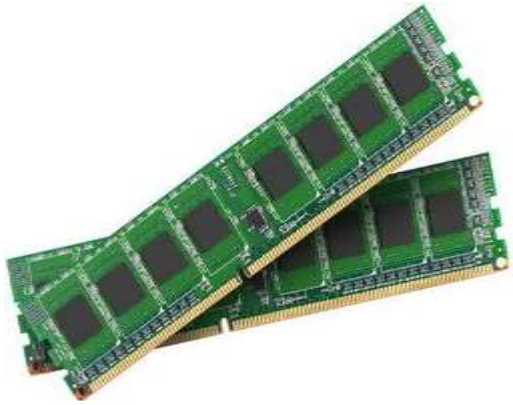
Université Ibn Zohr
Faculté des Sciences d'Agadir
Département de Physique

Filière: Électronique et Systèmes
Semestre: S5

Module: Électronique numérique 2







Professeur responsable : Noureddine MAOUHOUB

Année Universitaire: 2025-2026

Plan du cours

- ***Chapitre I : Technologies des circuits numériques intégrés***
- ***Chapitre II : Mémoires à semiconducteur***
- ***Chapitre III : Circuits logiques programmables***
- ***Chapitre IV : Introduction au langage VHDL***
- ***Chapitre V : Machines à états finis***

Chapitre I :
Technologies des circuits numériques intégrés

1- Introduction

- Un circuit intégré logique est classé selon sa technologie de fabrication.
- On distingue plusieurs familles de circuits intégrés:
  - **Famille bipolaire** fabriquée à base de transistors bipolaires, comme la famille la plus utilisée TTL (Transistor Transistor Logic).
  - **Famille CMOS** (Complementary Metal Oxyde Semiconductor) fabriquée à base de transistor à effet de champ MOSFET de type N et P.
  - **Famille BiCMOS** fabriquée à base de transistors bipolaires et MOSFET.

2- Caractéristiques électriques des circuits intégrés

❑ Notations

❑ Tensions

- V_{OH} : Tension de sortie à l'état haut.
- V_{OL} : Tension de sortie à l'état bas.
- V_{IH} : Tension d'entrée à l'état haut.
- V_{IL} : Tension d'entrée à l'état bas.
- V_{CC} ou V_{DD} : Tension d'alimentation positive.

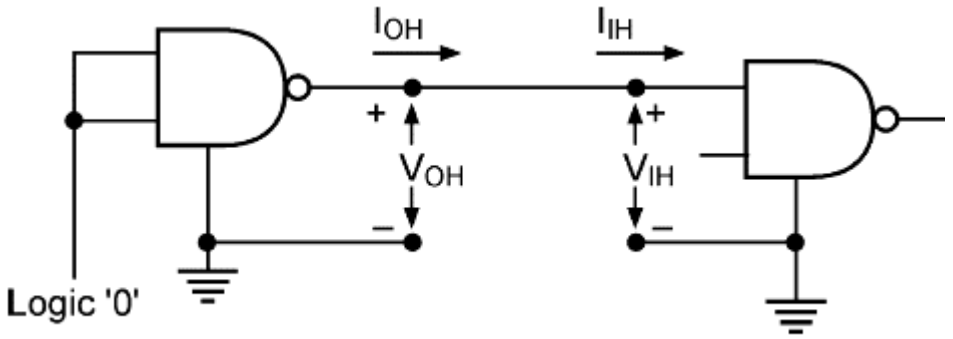
❑ Courants

- I_{OH} : Courant de sortie à l'état haut.
- I_{OL} : Courant de sortie à l'état bas.
- I_{IH} : Courant d'entrée à l'état haut.
- I_{IL} : Courant d'entrée à l'état bas.
- I_{CC} : Courant d'alimentation.

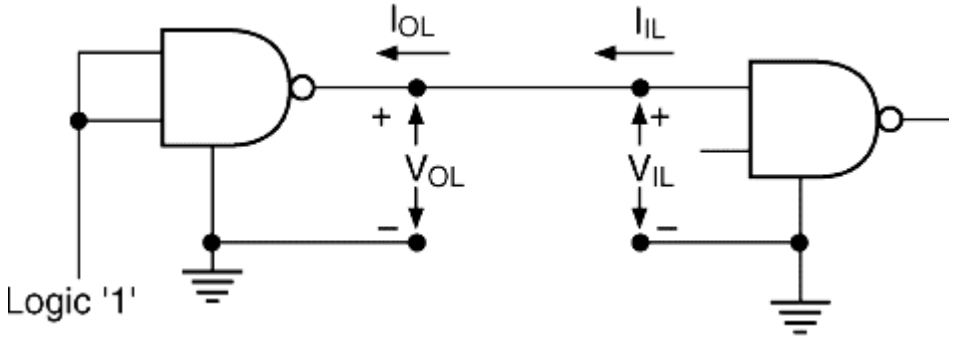
2- Caractéristiques électriques des circuits intégrés

- **Notation tensions et courants**

Etat HAUT:

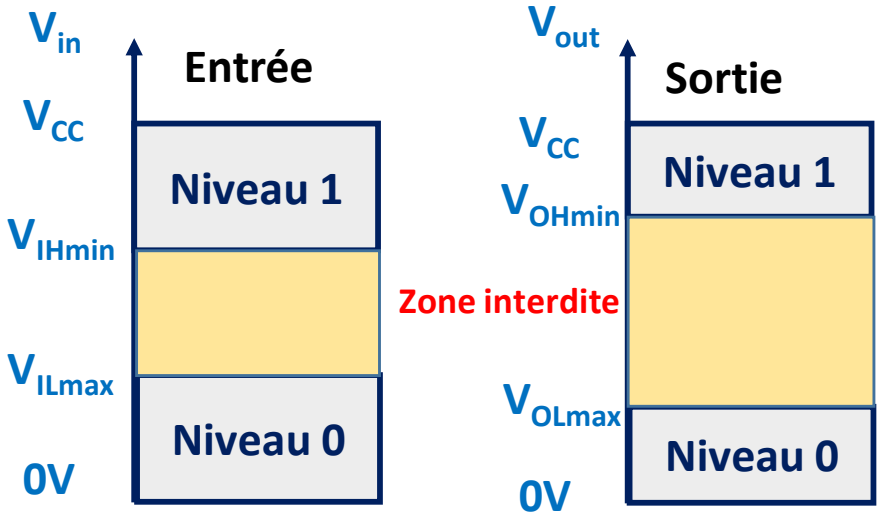


Etat BAS:



2- Caractéristiques électriques des circuits intégrés

❑ Gabarit de Tensions



En entrée:

- Le niveau logique "0" est assuré par: $0V < V_{in} < V_{ILmax}$
- Le niveau logique "1" est assure par : $V_{IHmin} < V_{in} < V_{CC}$

En sortie:

- Le niveau logique "0" est assuré par: $0V < V_{out} < V_{OLmax}$
- Le niveau logique "1" est assure par : $V_{OHmin} < V_{out} < V_{CC}$

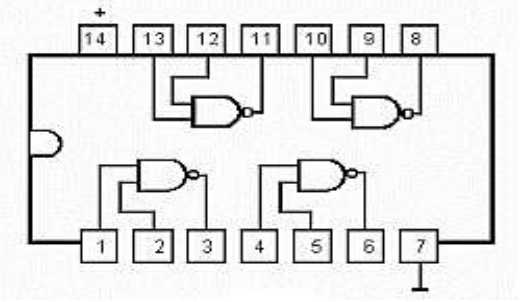
- Dans les zones interdites(indéfinies) les entrées et les sorties logiques sont en dehors du gabarit et ne fonctionnent pas correctement

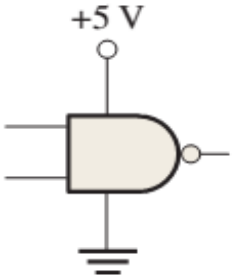
2- Caractéristiques électriques des circuits intégrés

❑ Tension d'alimentation

- **Famille logique TTL** : L'alimentation est fixée à **5 Volts** avec une tolérance de **± 5%**
- **Famille logique CMOS :** la tension d'alimentation est de **3 à 18 Volts**.(Exemple: CMOS 3,3 V, CMOS 5,5V)







❑ Niveaux logiques

En entrée:

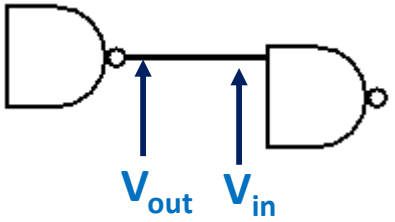
- En TTL : V_{IHmin} =2 V et V_{ILmax} = 0,8 V
- En CMOS : V_{IHmin} =70%*VDD et V_{ILmax} =30%*VDD

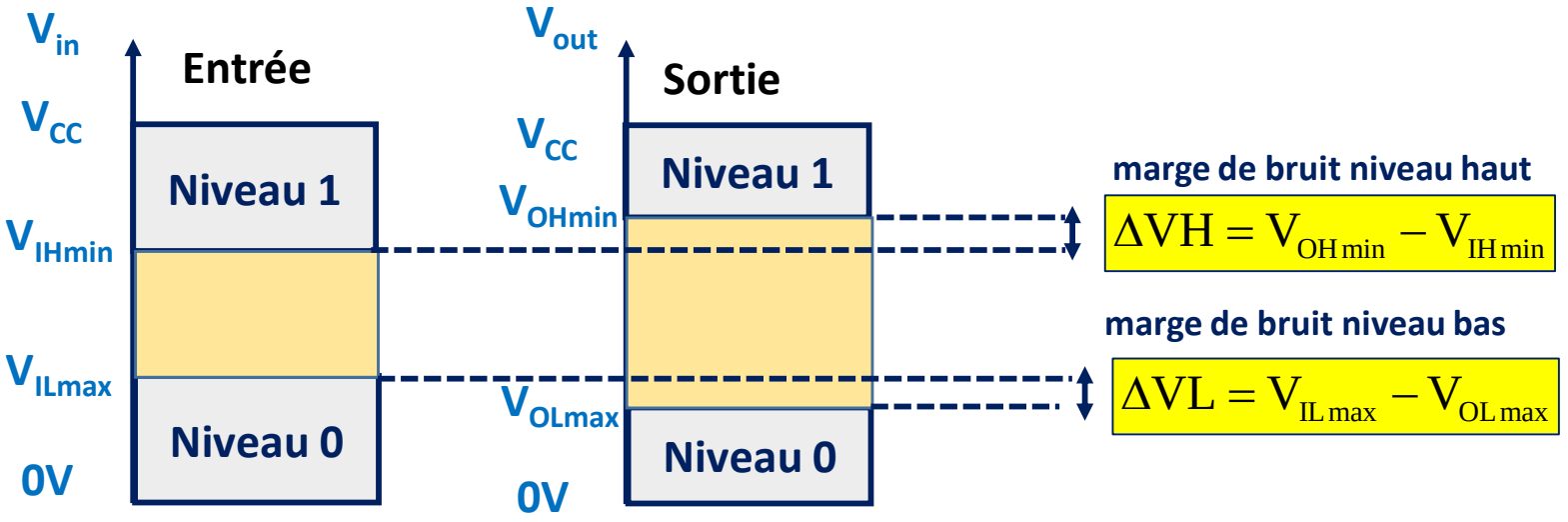
En sortie:

- En TTL : V_{OHmin} =2,4 V et V_{OLmax} = 0,4 V
- En CMOS : V_{OHmin} =95%*VDD et V_{OLmax} =5%*VDD

3- Immunité au bruit des circuits intégrés

- La tension d'entrée d'une porte logique est la tension de sortie de la porte logique précédente.
- **La marge de bruit** d'un circuit intégré logique est la plage de tension parasite que le circuit peut supporter sur ses entrées sans provoquer d'erreur logique.





3- Immunité au bruit des circuits intégrés

❑ Exemple

- Calculer les marges de bruit d'un circuit TTL dont les caractéristiques électriques sont données par:

$$V_{IH\min} = 2\ V$$
$$V_{IL\max} = 0.8\ V$$
$$V_{OH min} = 2.4\ V$$
$$V_{OL max} = 0.4\ V$$

$$\Delta VH = V_{OH\,min} - V_{IH\,min} = 0.4V$$

$$\Delta VL = V_{IL\,max} - V_{OL\,max} = 0.4V$$

- Calculer les marges de bruit d'un circuit CMOS 5V dont les caractéristiques électriques sont données par:

$$V_{IH\min} = 3{,}5\ V$$
$$V_{IL\max} = 1{,}5\ V$$
$$V_{OH min} = 4.4\ V$$
$$V_{OL max} = 0.33\ V$$

$$\Delta VH = V_{OH\,min} - V_{IH\,min} = 0.9V$$

$$\Delta VL = V_{IL\,max} - V_{OL\,max} = 1{,}17V$$

4- Sortance des portes logiques

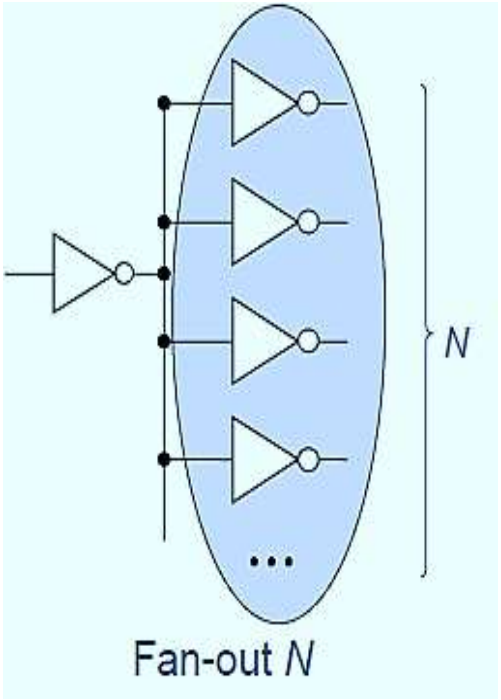
- **La sortance (Fan out)** correspond au nombres maximum d'entrées logiques de la même famille que l'on peut connecter à la sortie d'une porte logique.
- On définit la sortance à l'état logique bas par:

$$NL = \left| \frac{I_{OL\,max}}{I_{IL\,max}} \right|$$

- On définit la sortance à l'état logique haut:

$$NH = \left| \frac{I_{OH\,max}}{I_{IH\,max}} \right|$$

- La sortance est définit par: **N = min(NL,NH)**



Exercice d'application:

Calculer la sortance de la porte AND (7408) de la technologie TTL standard.
On donne : IIHmax = 40 µA, IILmax = -1,6mA, IOHmax = -0,8 mA, IOLmax =16 mA.

Réponse: la sortance est : N =10

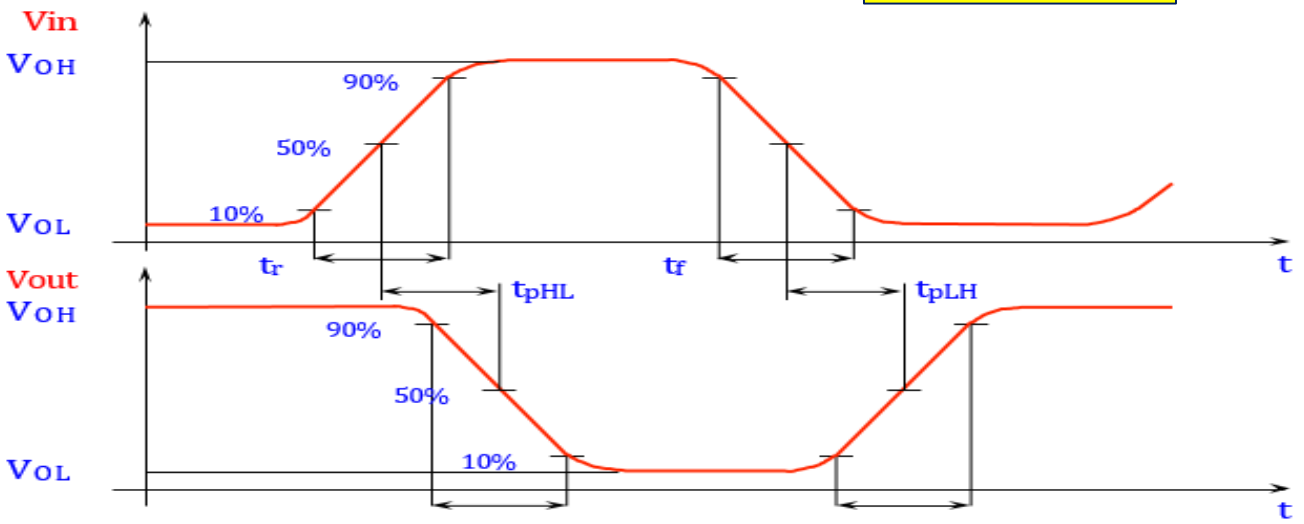
5- Temps de propagation des circuits intégrés

- Le changement d'un état logique n'est pas instantané. Il faut prendre en compte **le temps de montée (tr ou tm) ou de descente (tf ou td)** correspondent à la durée nécessaire pour passer de 10% à 90 % de la valeur finale de la tension.
- Dans les portes logiques, les signaux sont transmises avec un retard, c'est **Le temps de propagation** .
- On définit : $\mathbf{t_{pHL}}$ le temps de propagation lorsque la sortie passe de l'état haut à l'état bas.
- On définit : $\mathbf{t_{pLH}}$ le temps de propagation lorsque la sortie passe de l'état bas à l'état haut.
- On définit le **temps de propagation moyen t_p** par:

$$t_p = \frac{t_{pHL} + t_{pLH}}{2}$$

En entrée:

En sortie:



6- Technologie des circuits intégrés TTL

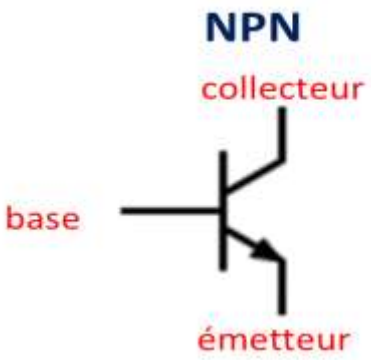
- La famille des circuits intégrés logiques TTL (Transistor Transistor Logic) est une famille de circuits numérique inventée en 1961 et fabriquée à la base des transistors bipolaires.
- Deux séries de famille TTL: série commerciale 74 (0 à 70 °C) et la série militaire 54 (- 55 à +125 °C)
- La technologie TTL de série 74 se décompose de:
  - TTL standard 74xx
  - TTL Low power 74Lxx (faible consommation)
  - TTL Schottky 74Sxx (réalisé avec des transistors schottky)
  - TTL Low power Schottky 74LSxx (schottky faible consommation)
  - TTL Advanced Schottky 74ASxx (technologie schottky avancée)
  - TTL Advanced Lowe power Schottky 74ALSxx
  - TTL Fast 74Fxx (rapide)

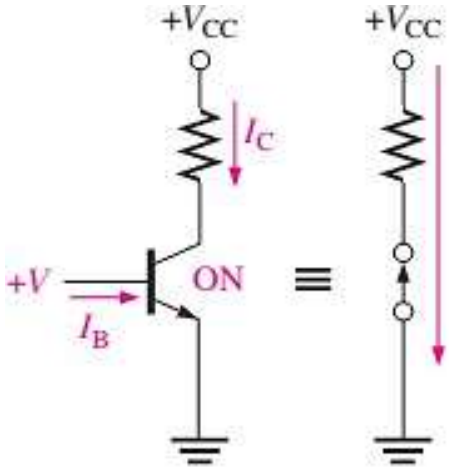
Porte NAND TTL 74LS37



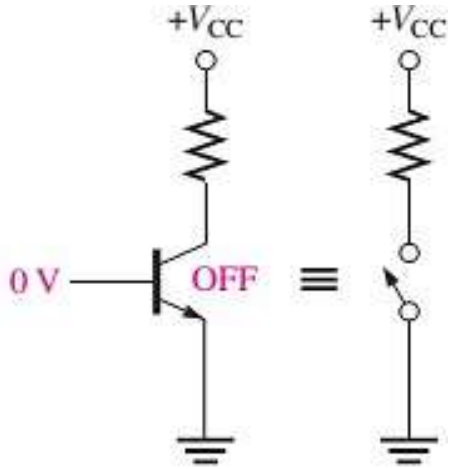
7- Technologie des circuits intégrés TTL

7-1 Transistor bipolaire en comutation





Transistor saturé: état « ON »



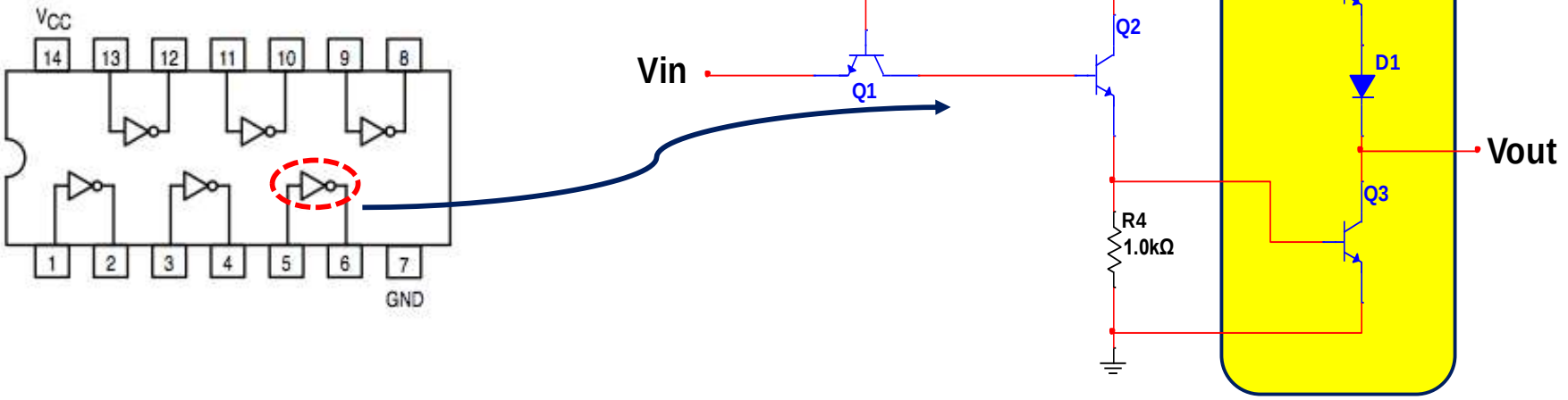
Transistor bloqué: état « OFF »

- Lorsque la tension à la base est supérieur ou égale à V_{BE}; un courant suffisant est fourni à la base, le transistor devient passant et entre en saturation. Le transistor est similaire à un interrupteur fermé.
- Lorsque la tension à la base est nulle; un courant nul est fournie à la base, le transistor devient bloqué. Le transistor est similaire à un interrupteur ouvert.

7- Technologie des circuits intégrés TTL

7-1 Circuits TTL standard à sortie « Totem Pole »

❑ **Exemple d'une porte logique inverseuse 7404**



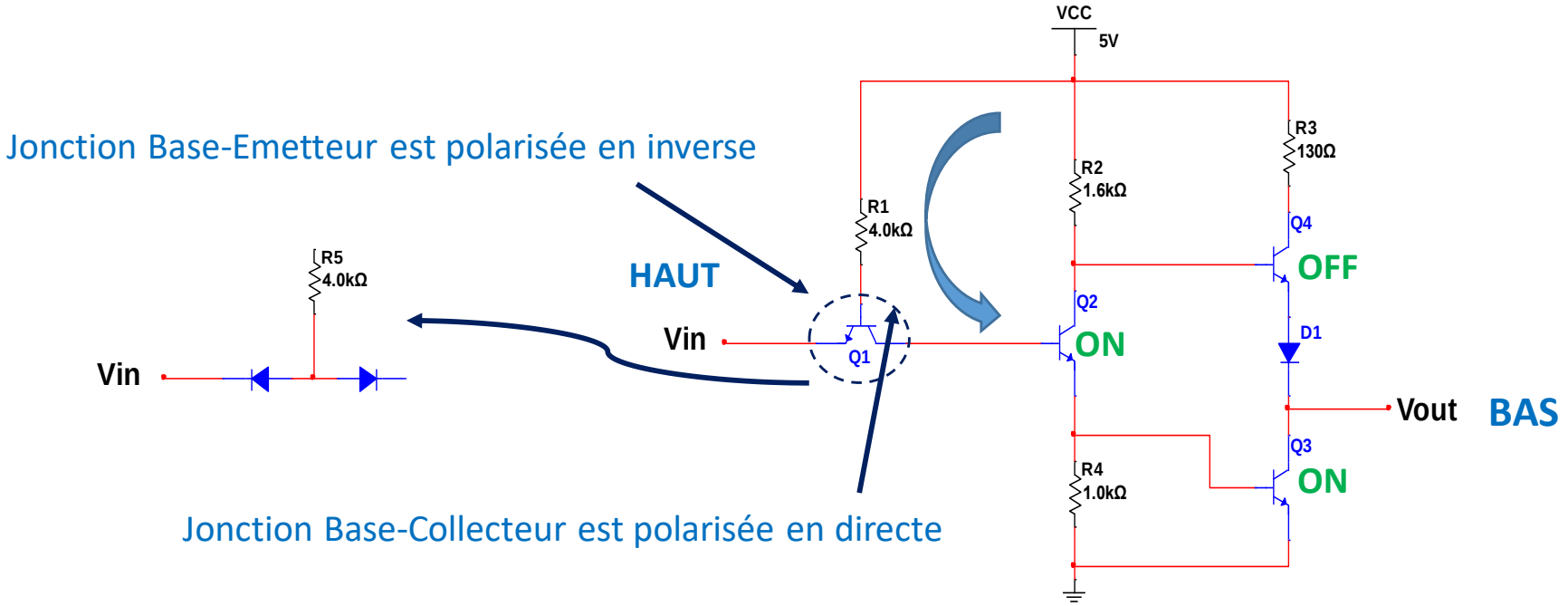
Etage de sortie totem pole

7- Technologie des circuits intégrés TTL

7-1 Circuits TTL standard à sortie « Totem Pole »

❑ **Exemple d'une porte logique inverseuse 7404**

- **Lorsque l'entrée est HAUT**, la jonction base-émetteur de Q1 est polarisée en inverse et la jonction base-collecteur est polarisée en direct. Cette condition permet au courant de traverser R1 et la jonction base-collecteur de Q1 dans la base de Q2, entraînant ainsi Q2 et Q3 en saturation et Q4 devient bloqué. **La sortie Vout est donc à l'état BAS.**

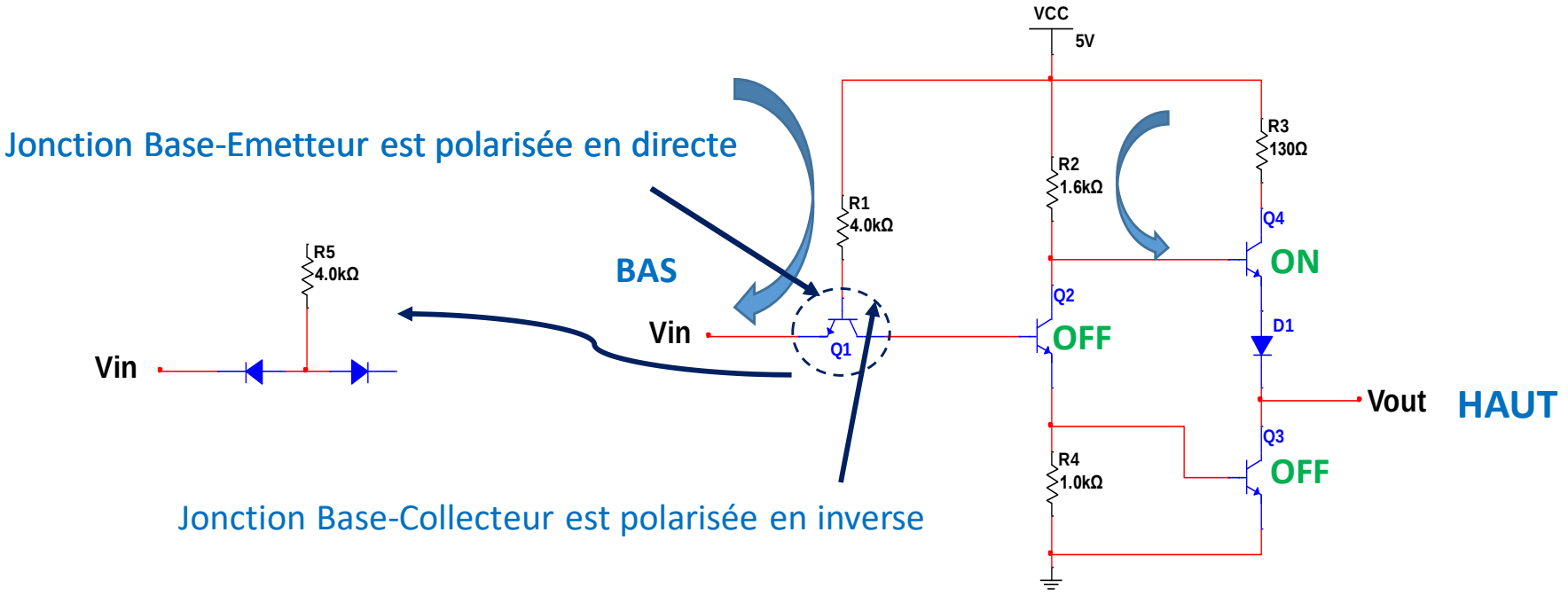


7- Technologie des circuits intégrés TTL

7-1 Circuits TTL standard à sortie « Totem Pole »

❑ Exemple d'une porte logique inverseuse 7404

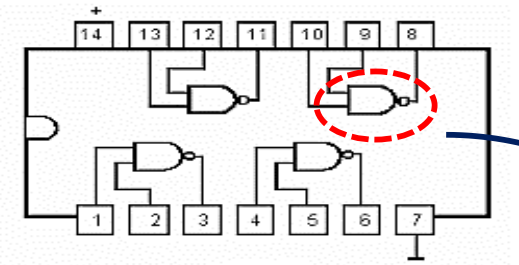
- **Lorsque l'entrée est à l'état BAS**, la jonction base-émetteur de Q1 est polarisée en direct et la jonction base-collecteur est polarisée en inverse. Un courant traverse R1 et la jonction base-émetteur de Q1 vers l'entrée BASSE. Un état BAS fournit un chemin vers la masse pour le courant. Il n'y a pas donc de courant à la base de Q2, qui est donc désactivée permetant le blocage de Q3. Le collecteur de Q2 est à l'état HAUT, ce qui active Q4. **La sortie Vout est donc à l'état HAUT.**



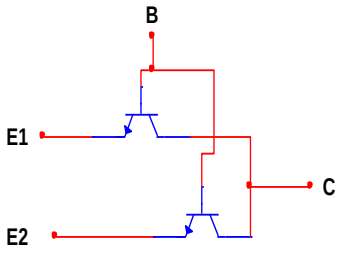
7- Technologie des circuits intégrés TTL

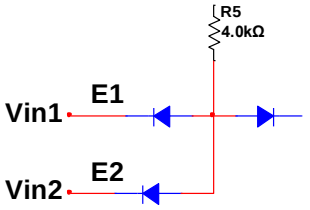
7-1 Circuits TTL standard à sortie « Totem Pole »

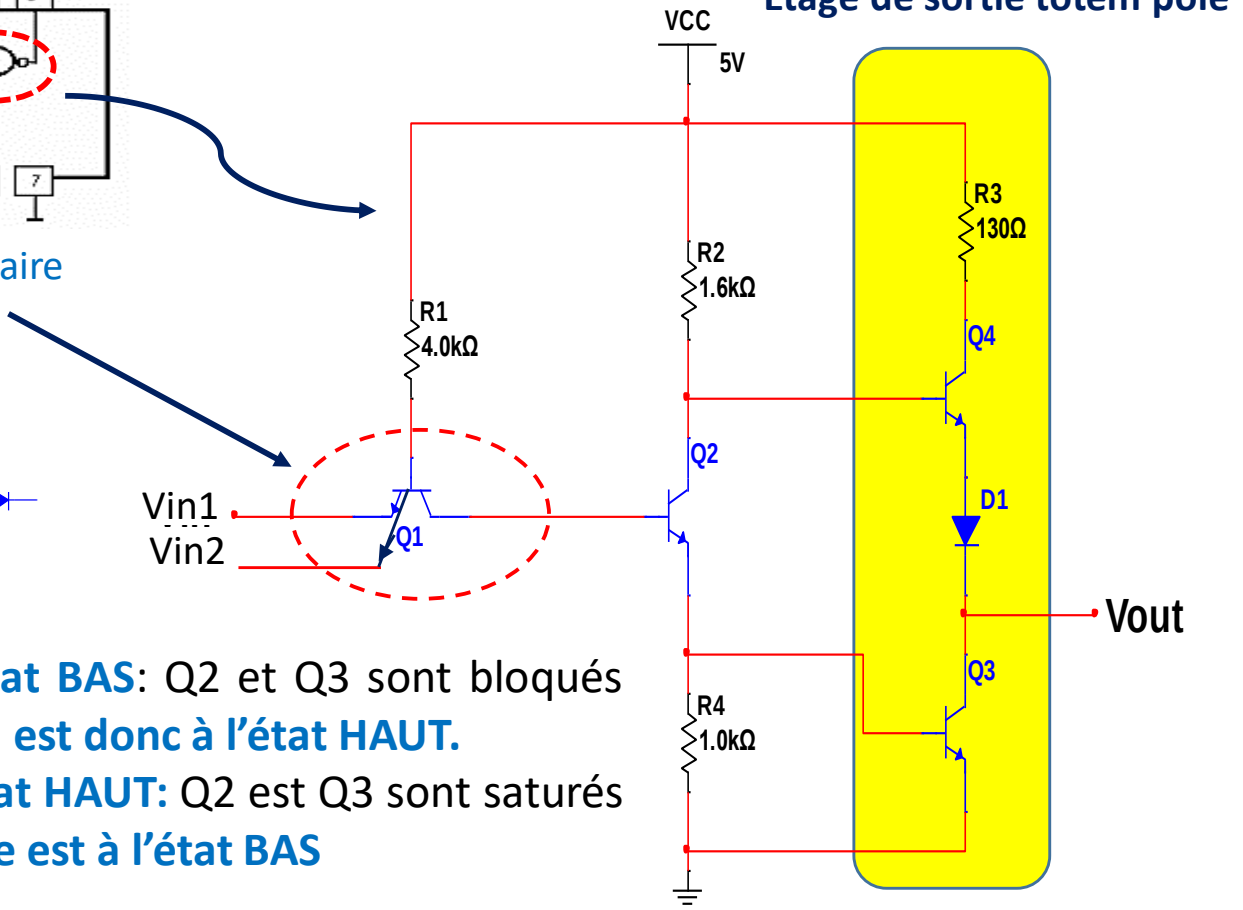
❑ **Exemple d'une porte logique NAND 7400**



Technologie TTL basée sur un Transistor bipolaire multi-émetteurs







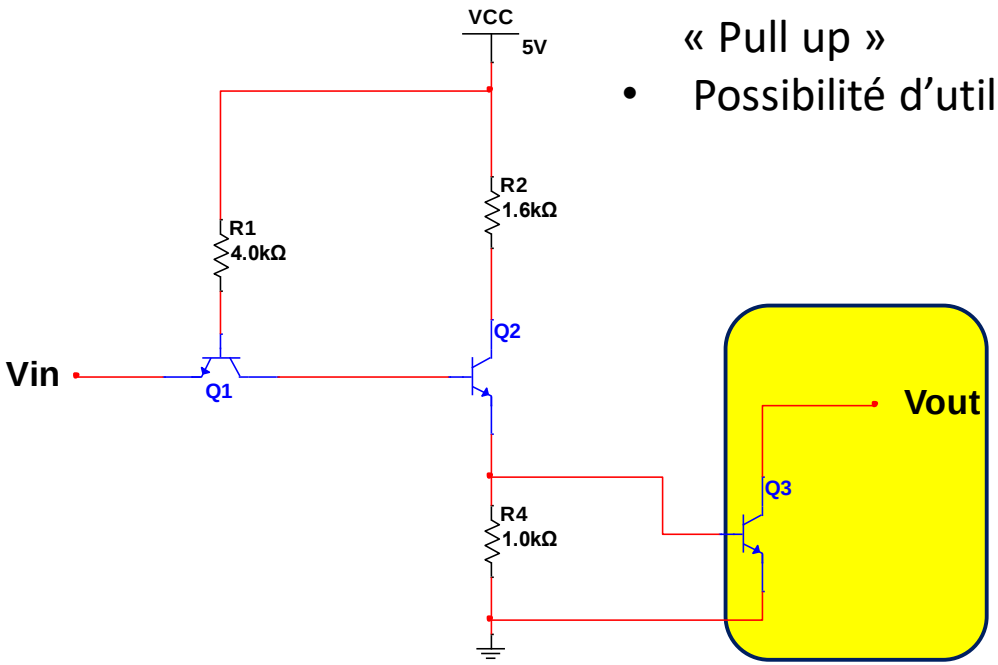
- **Lorsque l'une des entrées est à l'état BAS**: Q2 et Q3 sont bloqués (OFF) et Q4 est saturé (ON). **La sortie est donc à l'état HAUT.**
- **Lorsque les deux entrées sont à l'état HAUT:** Q2 est Q3 sont saturés (ON) et Q4 est bloqué (OFF). **La sortie est à l'état BAS**

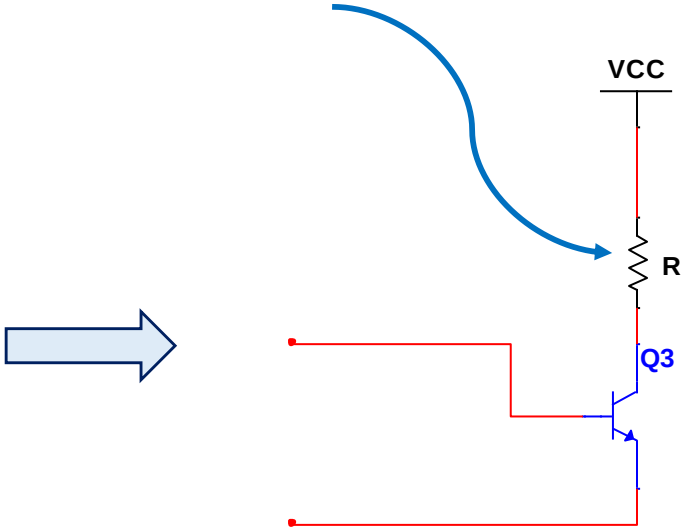
7- Technologie des circuits intégrés TTL

7-2 Circuits TTL standard à sortie « Collecteur ouvert »

❑ **Exemple d'une porte logique inverseuse**

- Pour avoir l'état logique 1, on place une résistance de rappel « Pull up »
- Possibilité d'utiliser une alimentation supérieur à 5V





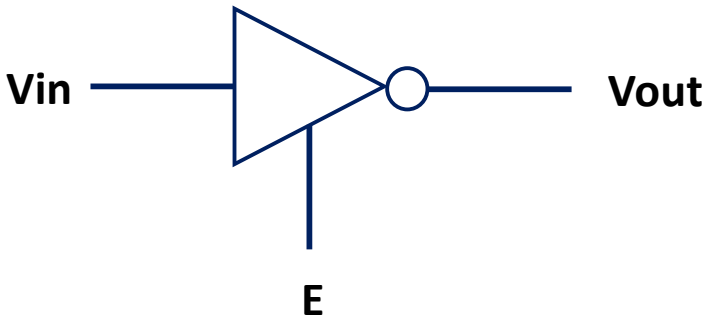
Sortie à collecteur ouvert

7- Technologie des circuits intégrés TTL

7-3 Circuits TTL à trois états logiques

❑ Exemple d'une porte logique inverseuse à trois étas

- Ce type de portes possède une entrée d'activation E (Enable) qui permet de:
  - Si E = 0, la sortie est déconnectée de la porte: La sortie et à l'état « Haute impédance » (HZ) ou Z.
  - Si E = 1, la sortie est soit à l'état 0 ou 1 selon les entrées.
- Cette porte possède donc trois états: 0, 1 ou HZ.

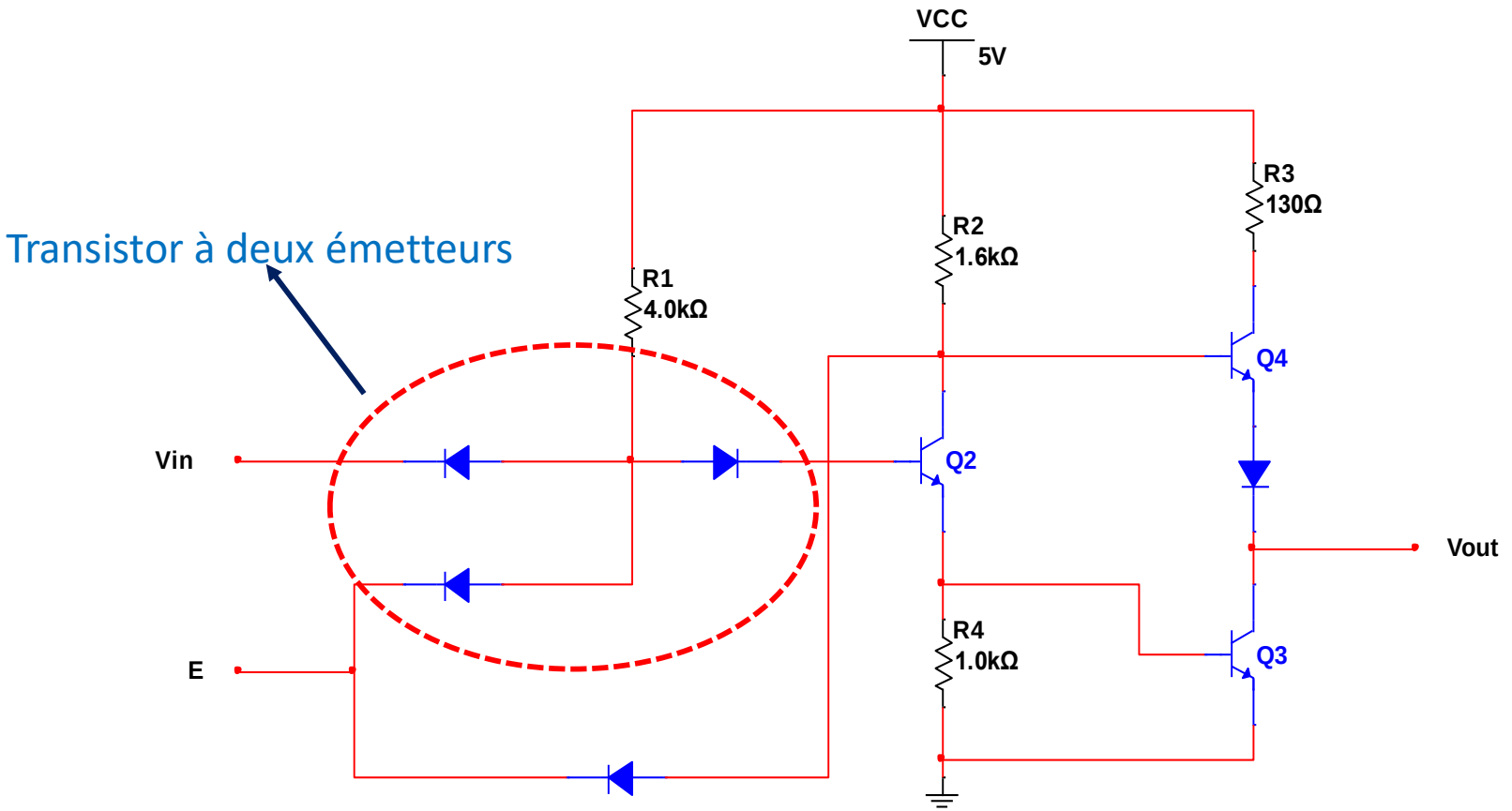


E	Vin	Vout
0	X	HZ
1	0	1
1	1	0

7- Technologie des circuits intégrés TTL

7-3 Circuits TTL à trois états logiques

❑ **Exemple d'une porte logique inverseuse à trois étas**



8- Technologie des circuits intégrés CMOS

- La famille des circuits intégrés logiques CMOS (Complementary Metal-Oxide-Semiconductor) est une technologie électronique très répandue dans la conception de circuits intégrés.
- Cette technologie combine entre **les transistors MOSFMetal-Oxide-Semiconductor Field-Effect Transistor) ET (de type N (NMOS) et de type P (PMOS**)
- C'est une technologie plus efficace que la famille TTL notamment pour les applications nécessitant une faible consommation.
- Une technologie caractérisée par: une grande immunité au bruit, haute densité d'intégration et rapide.
- Les séries de la famille CMOS:
  - Série 4000 (CD4000)
  - Série 74C00: Compatible avec les circuits TTL en termes de brochage et de fonctions.
  - Série 74HC00 (High-speed CMOS): circuits rapides.
  - Série 74HCT00 (High-speed CMOS, TTL compatible):
- les séries **74HC** et **74HCT** sont les plus utilisées aujourd'huit en électronique numérique.

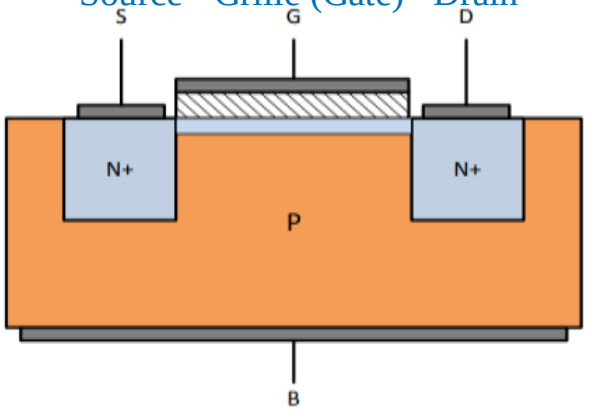
Porte NAND CMOS CD4011



8- Technologie des circuits intégrés CMOS

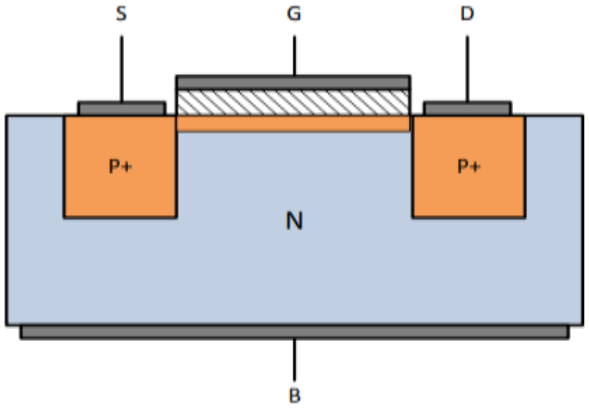
MOSFET à canal N

Source Grille (Gate) Drain



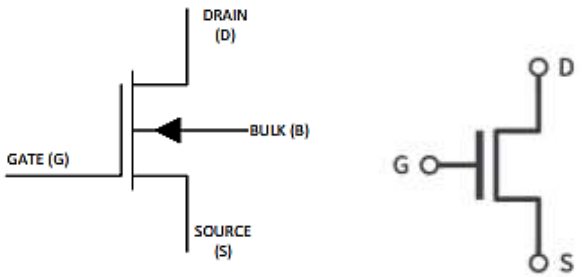
Substrat (Bulk)

MOSFET à canal P

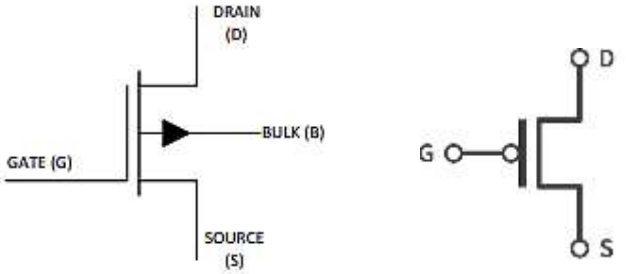


Symboles

MOSFET à canal N



MOSFET à canal P

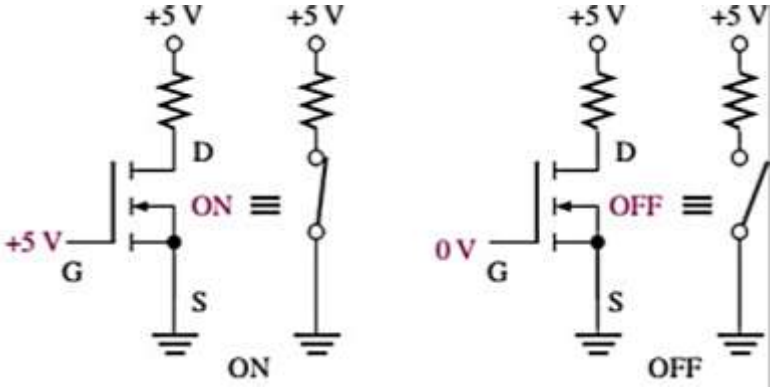


8- Technologie des circuits intégrés CMOS

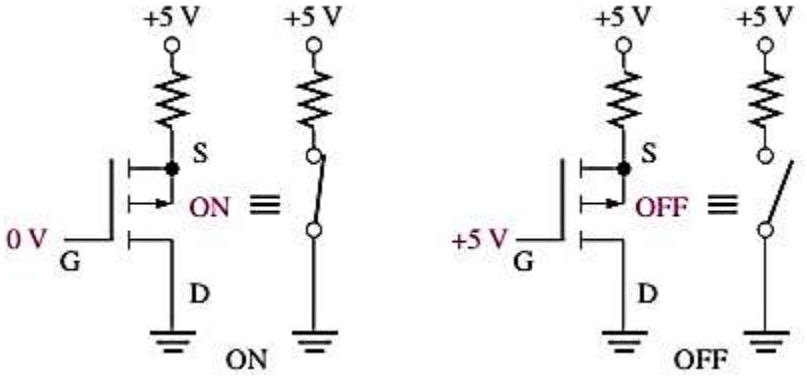
8-1 Fonctionnement des transistors MOS en commutation

- Dans les transistors MOS, le substrat est généralement commun à la source

❑ **Commutation des MOSFET à canal N**



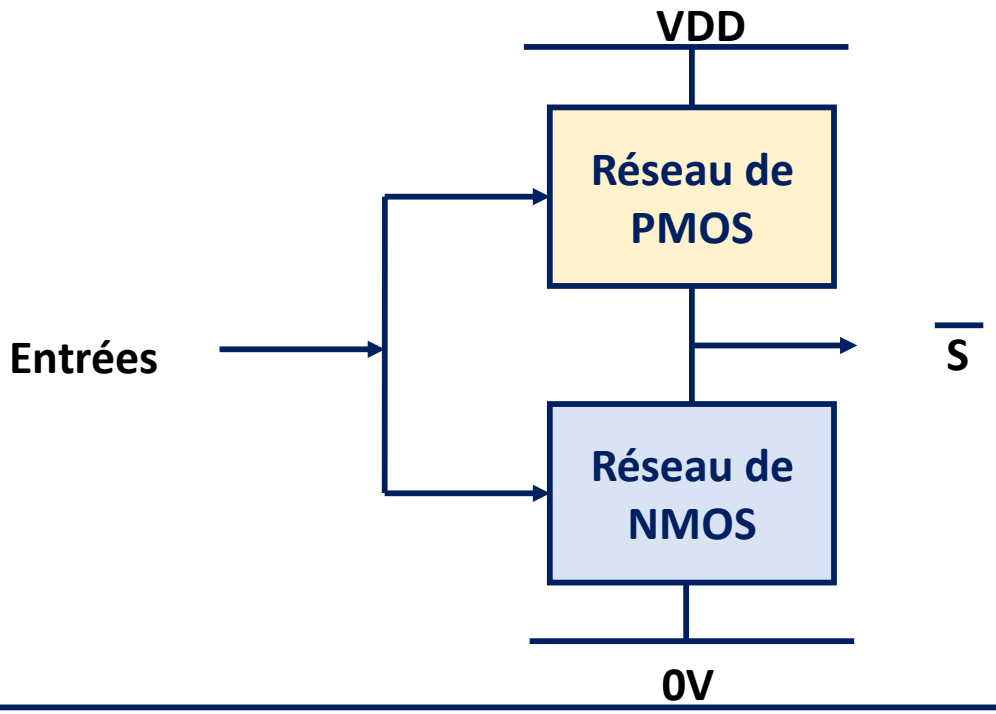
❑ **Commutation des MOSFET à canal P**



8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

- Un circuit logique CMOS est réalisé par utilisation de deux réseaux PMOS et NMOS duaux commandés par les mêmes entrées.
- Un seule réseau parmi les deux doit être passant.
- La fermeture du réseau NMOS permet d'obtenir **S = 0**.
- La fermeture du réseau PMOS permet d'obtenir **S = 1**.

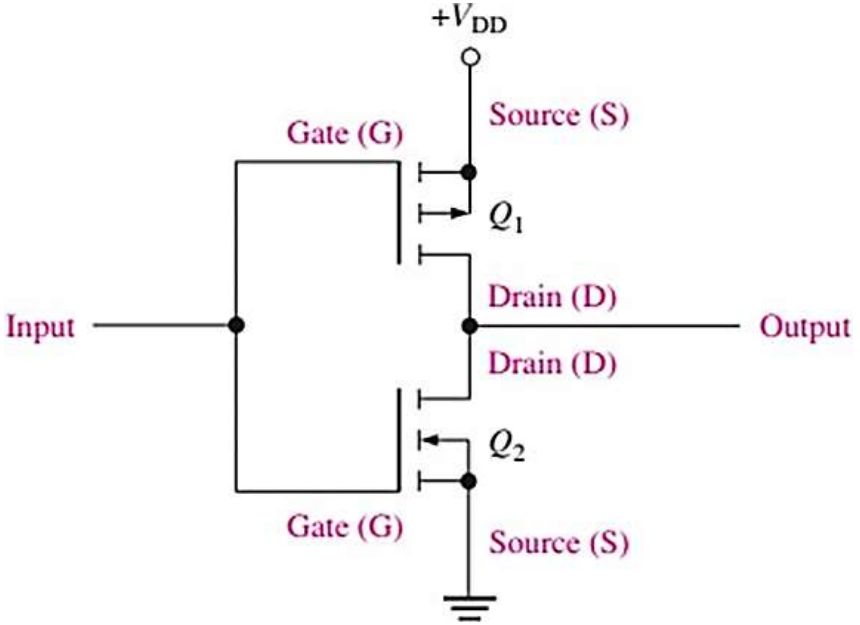


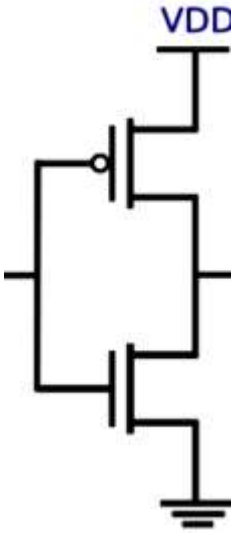
8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

❑ Exemple de la réalisation d'une porte inverseuse CMOS

- Une porte inverseuse est réalisée par deux transistors NMOS et PMOS.
- Lorsque l'entrée est à l'état **HAUT,** Le transistor PMOS (Q1) est bloqué (interrupteur ouvert) et le transistor NMOS (Q2) est passant (interrupteur fermé). **La sortie est à l'état BAS**.
- Lorsque l'entrée est à l'état **BAS**, Le transistor PMOS (Q1) est passant (interrupteur fermé) et le transistor NMOS (Q2) est bloqué (interrupteur ouvert). **La sortie est à l'état HAUT**.





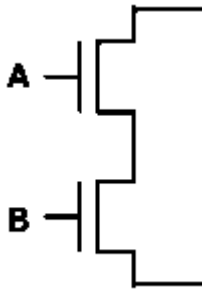
8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

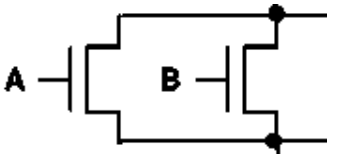
❑ Règles de conception d'un circuit logique CMOS

- **Réseau NMOS:**
  - ➢ Pour réaliser un ET logique, les transistors NMOS doivent être connectés en série.
  - ➢ Pour réaliser un OU logique, les transistors NMOS doivent être connectés en parallèle.

ET logique

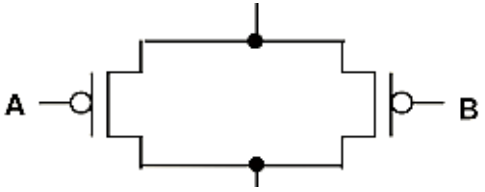


OU logique

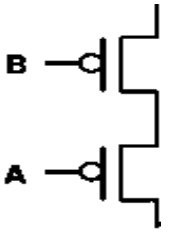


- **Réseau PMOS:**
  - ➢ Croiser les règles de construction du réseau NMOS.
  - ➢ Complémenter la fonction logique puis la procéder comme pour le réseau NMOS.

ET logique



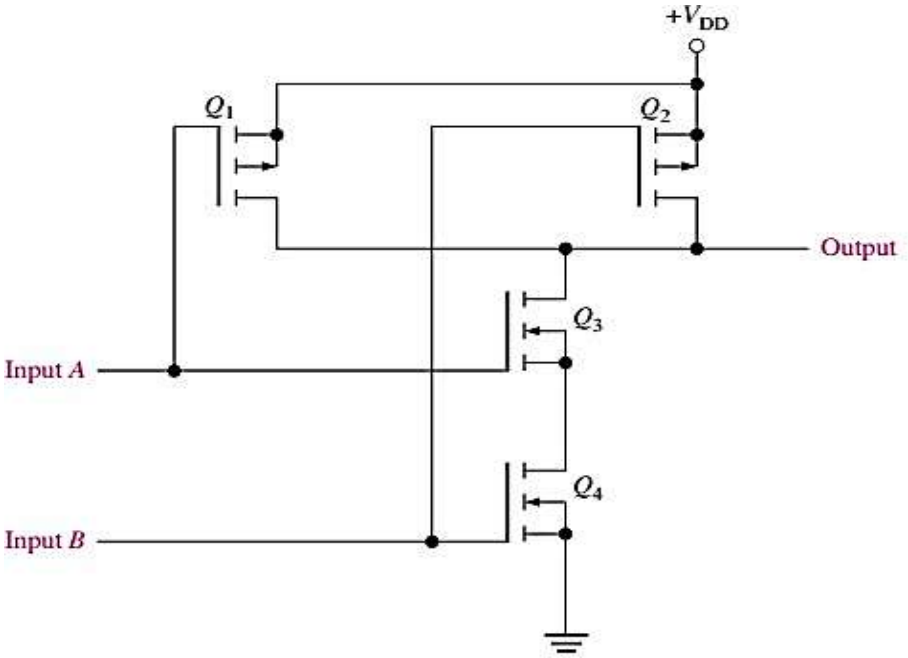
OU logique

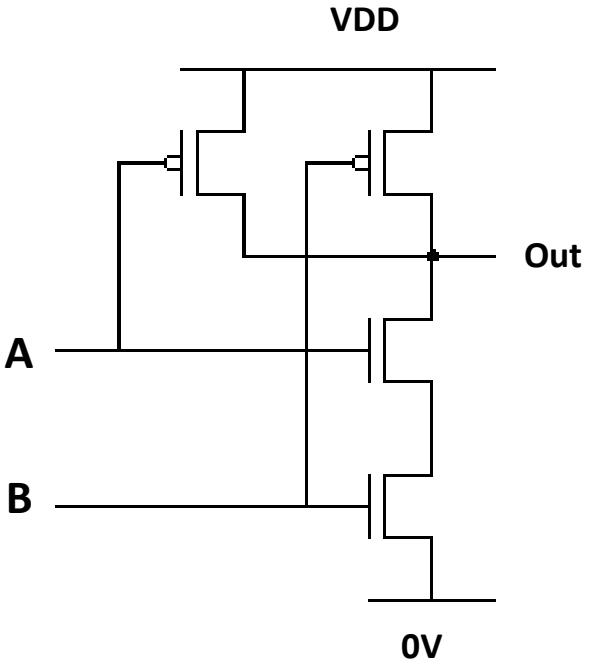


8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

- **Exemple de la réalisation d'une porte NAND CMOS**

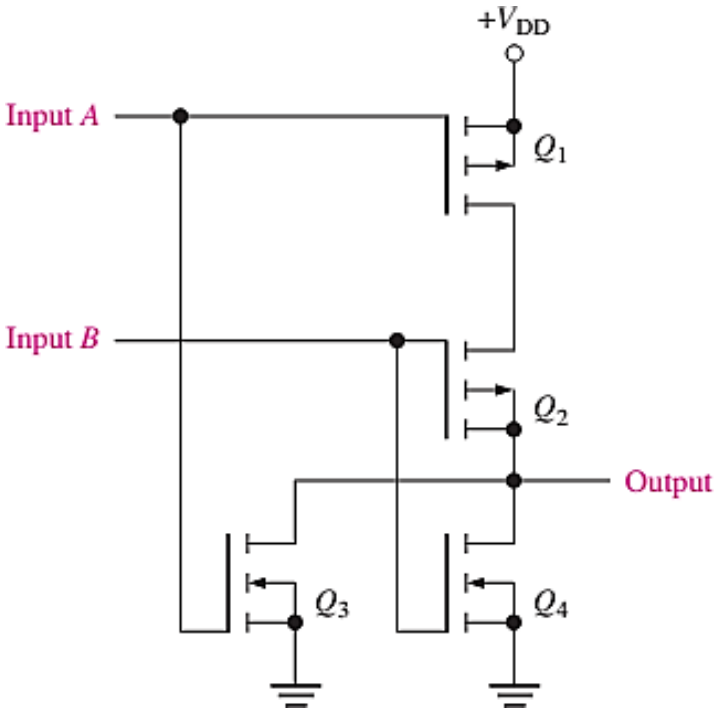


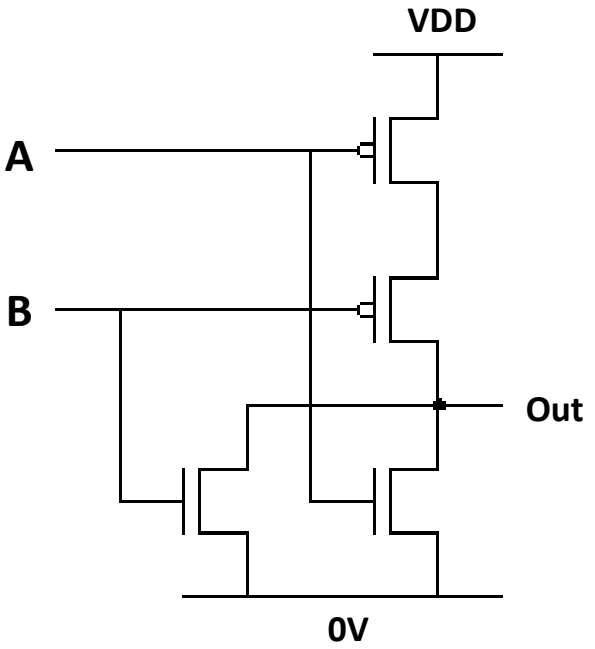


8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

- **Exemple de la réalisation d'une porte NOR CMOS**



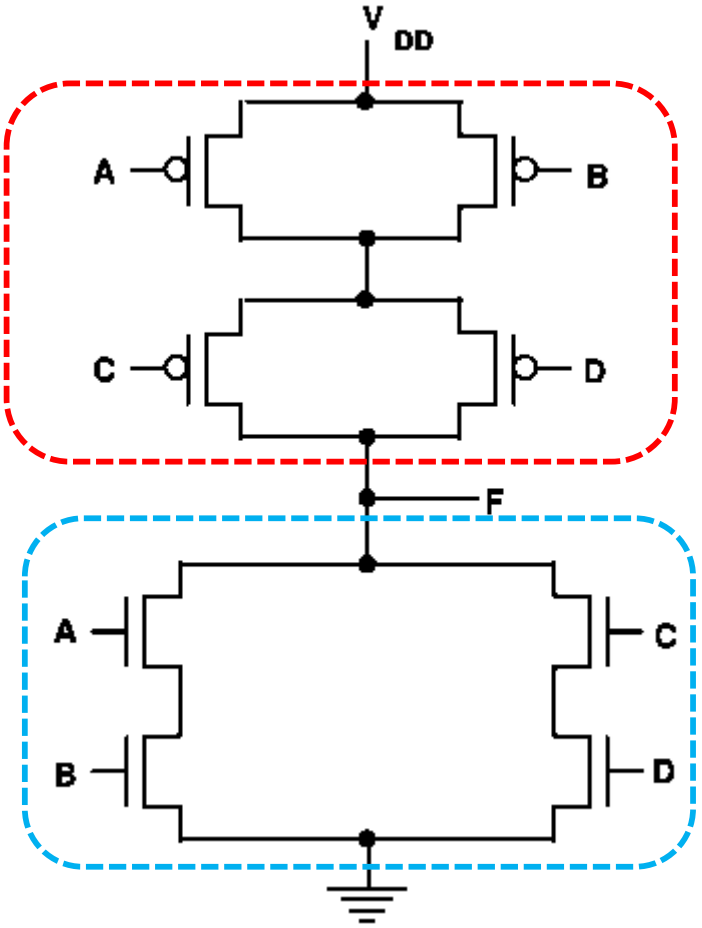


8- Technologie des circuits intégrés CMOS

8-2 Conception des circuits logiques CMOS

❑ **Exemple de la réalisation d'une fonction logique en CMOS**

$$F = \overline{A.B + C.D}$$



Réseau PMOS

Réseau NMOS

Chapitre II :
Mémoires à semiconducteur

1- Définitions

- Une mémoire à semi-conducteur est un circuit numérique intégré capable de stocker et de restituer des informations binaires temporaires ou permanentes.
- Ces mémoires sont utilisées dans toutes les applications qui fonctionnent à base de microprocesseurs.

Exemple de mémoire à semi-conducteur



- La mémoire peut être dans le microprocesseur (des registres) , interne (Mémoire centrale ou principale: mémoire à semiconducteur) ou externe (mémoire de masse).
  - **Exemples de mémoires de masse:**
    - Un disque dure, disquette (mémoires magnétiques)
    - Un CDROM, DVD (mémoires optiques)

2- Caractéristiques d'une mémoire

2-1 Capacité d'une mémoire

- La capacité (ou taille) d'une mémoire correspond à la quantité d'informations qu'elle peut mémoriser.
- La capacité peut s'exprimer en différentes unités :

- **Bit** : élément de base pour la représentation de l'information .
- **Octet** : un octet = 8 bits
- **kilo-octet (KO)** : 1 KO = 2^{10} octets= 1024 octets
- **Méga-octet (MO)** : 1 MO = 2^{10} KO = 1024 KO = 2^{20} octets
- **Géga-octet (GO)** :1 GO = 2^{10} MO = 1024 MO = 2^{30} octets
- **Téra-octet (To)** : 1 To = 2^{10} GO = 1024 Go =2^{40} octets

2- Caractéristiques d'une mémoire

2-2 Volatilité

- Une mémoire **volatile** est un type de mémoire qui perd son contenu dès que l'alimentation électrique est coupée.

  Exemple : la RAM (mémoire vive) est une mémoire volatile.
- Une mémoire **non volatile** est un type de mémoire capable de conserver les informations même en l'absence d'alimentation électrique.

  Exemples : ROM (mémoire morte), disque dur...

2-3 Mode d'accès à l'information

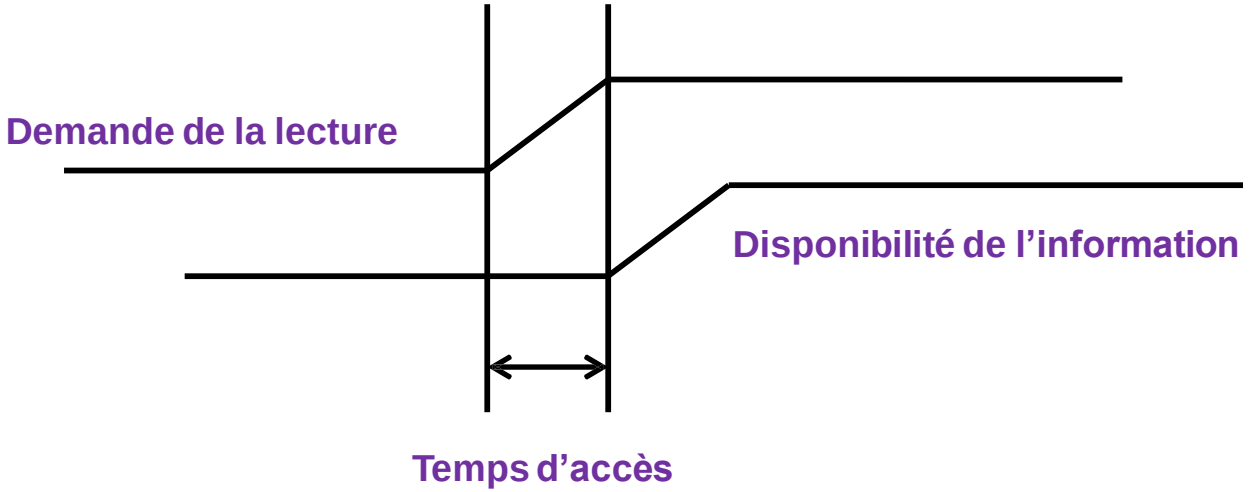
- Une mémoire permet de réalise deux types d'opérations :
  - **Lecture** : récupérer ou restituer une information mémorisée.
  - **Écriture** : enregistrer une information.

- Les mémoires qui utilisent les deux modes lecteur/écriture , s'appelles **mémoires vives** comme la mémoire RAM (Random Acces memory).
- Les mémoires qui utilisent le mode lecture, s'appelles **mémoires mortes** comme la mémoire ROM (Read Only Memory)

2- Caractéristiques d'une mémoire

2-4 Temps d'accès d'une mémoire

- Le temps d'accès d'une mémoire est le délai entre la demande d'une donnée et le moment où celle-ci devient disponible pour la lecture ou l'écriture.



- Le temps d'accès d'une mémoire est un critère important pour déterminer ces performances

3- Types des mémoires à semiconducteur

3-1 Mémoires vives

- La mémoire vive est une mémoire centrale volatile à accès aléatoire : **mémoire RAM**
  - Il existent deux grandes familles des mémoires RAM:

    - **Les mémoires statiques (SRAM)** à base de bascules de type D, sont caractérisé par un faible taux d'intégration mais un temps d'accès rapide

    - **Les mémoires dynamiques (DRAM)** à base de condensateurs, sont caractérisé par un très grand taux d'intégration, elle sont plus simples que les mémoires statiques mais avec un temps d'accès plus long.

3-2 Mémoires mortes

- La mémoire morte est une mémoire non volatile:
  - Il existent plusieurs familles des mémoires mortes:

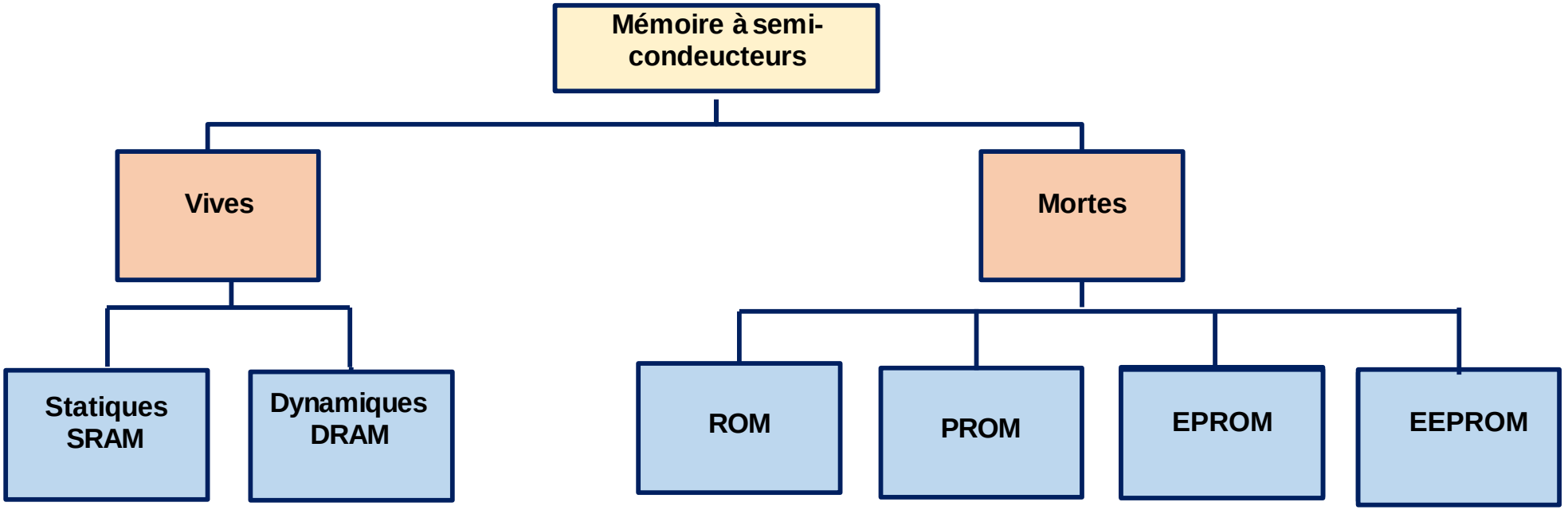
    - Les **mémoires ROM** (Read Only Memory) utilisée pour stocker des données permanentes, comme le BIOS.

    - Les **mémoires PROM** (Programmable Read-Only Memory) est livrée vide par le fabricant et peut être programmée une seule fois par l'utilisateur pour enregistrer des données permanentes.

    - Les **mémoires EPROM** sont des PROM effaçables par rayons UV.

    - Les **mémoires EEPROM** sont des PROM effaçables électriquement.

3- Types des mémoires à semiconducteur



4- Organisation d'une mémoire

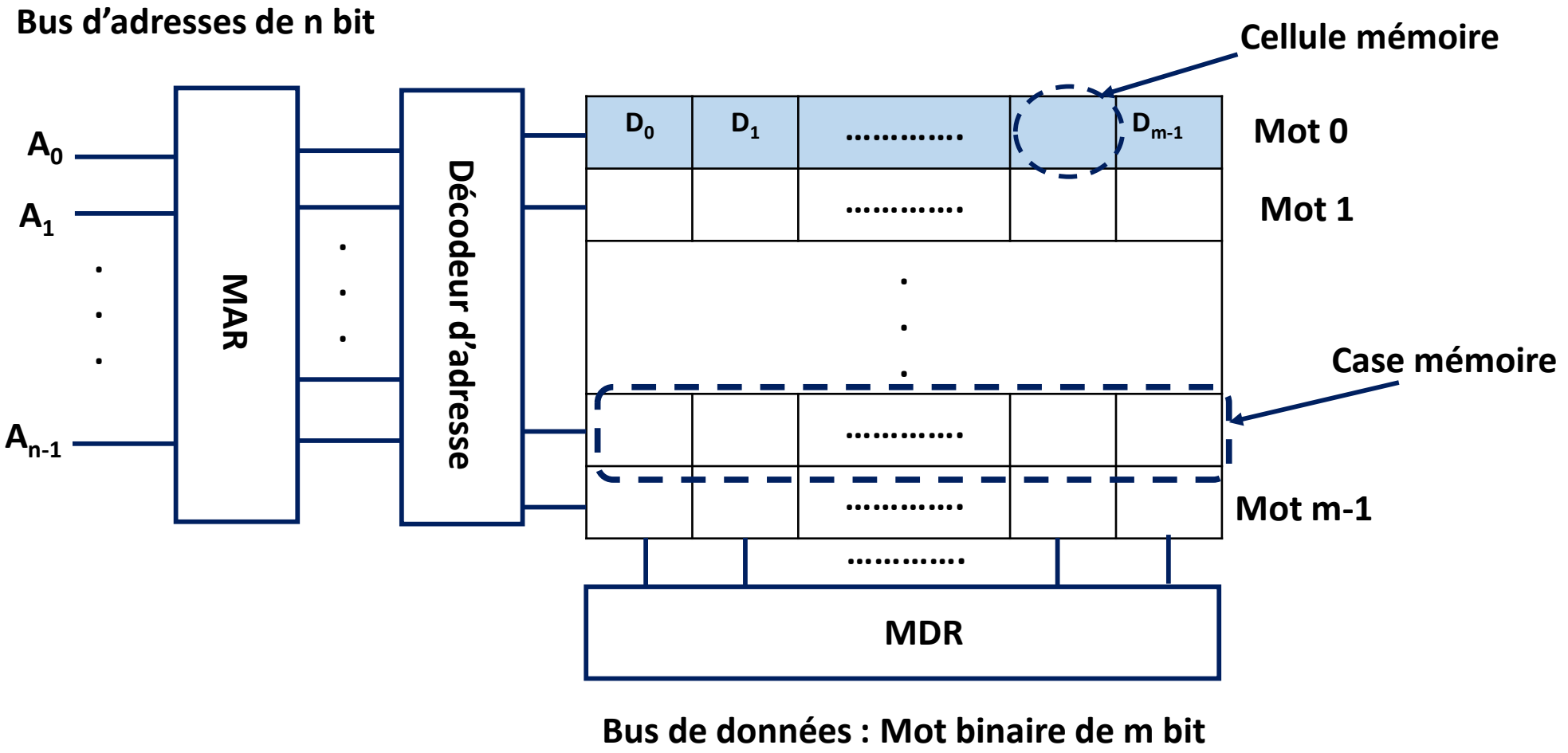
- Une mémoire est représentée par un ensemble de **cases mémoires** contenant chacune un **mot binaire de taille m bits**. Les m bits sont des valeurs multiples de deux: 4,8,16, 32…
- Chaque bit d'une case mémoire est stocker par **une cellule mémoire**.
- Chaque case mémoire est sélectionnée par une **adresse unique**, en un bus d'adresses de de taille n bits (n lignes d'adresse)
- La valeur binaire est chargée dans un **registre d'adresse mémoire MAR** (memory adresse registre).
- Lorsque une adresse est chargée dans le registre MAR , **un décodeur d'adresses** va recevoir la même information que celle du MAR et sélectionne l'information dans la case mémoire correspondante et la stocke dans le **registre de donnée mémoire MDR** (memory data registre).
- Le bus d'adresse de n bits permet d'adresser **2^n mots binaires (espace adressable) de m bits**.
- La capacité de la mémoire est donc : **$C = 2^n \times m$ bits**

Exemple: On considère une mémoire dont **la taille du bus d'adresse est 10 bits** et **la taille des données est 8 bit**. Quel est le nombre de mots adressés par cette mémoire et quelle est la capacité de cette mémoire?

Nombre de mots (Espace adressable) = 2^{10} = 1024 mots de 8 bits.

$C = 2^{10}.8$ = 1 Ko

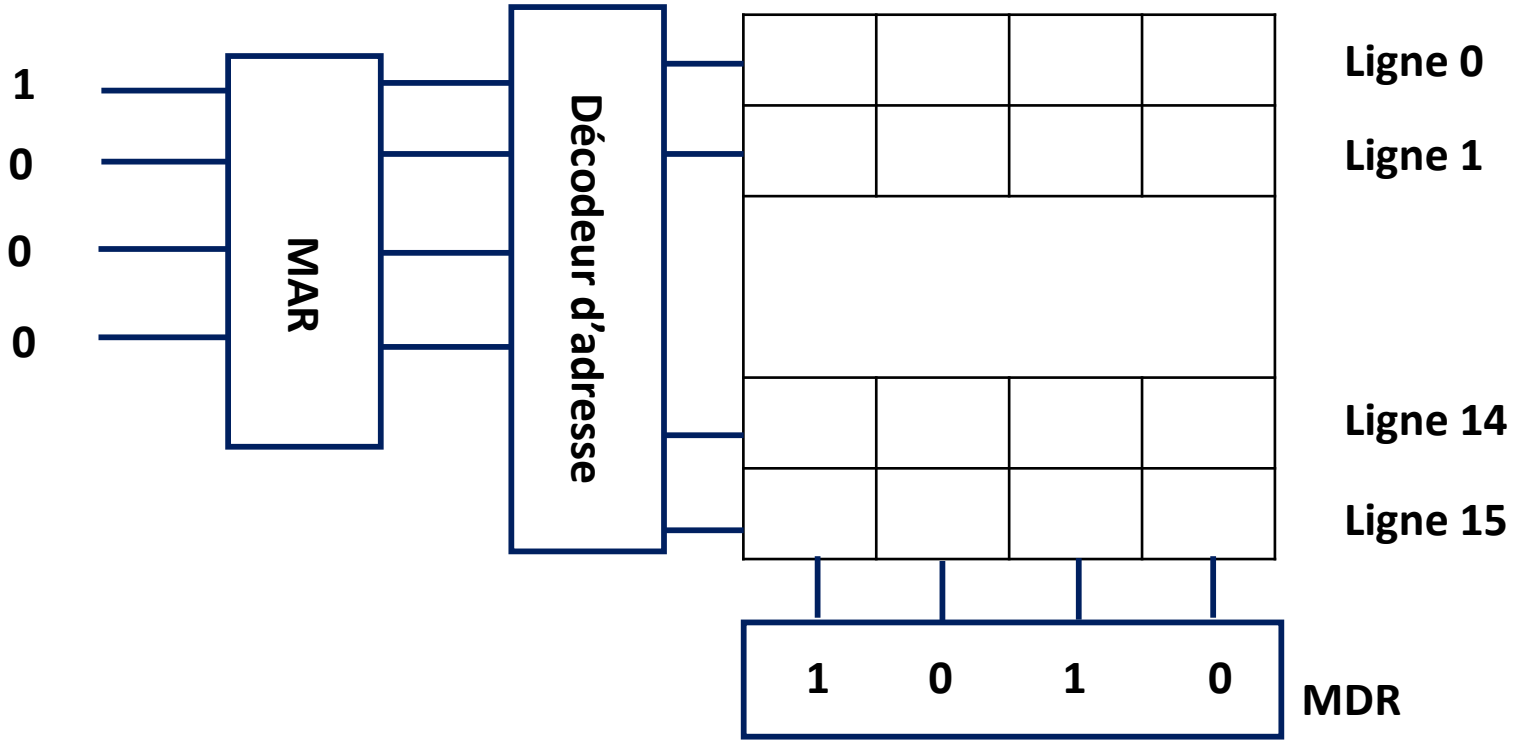
4- Organisation et opération d'une mémoire



4- Organisation et opération d'une mémoire RAM

❑ Opération d'écriture:

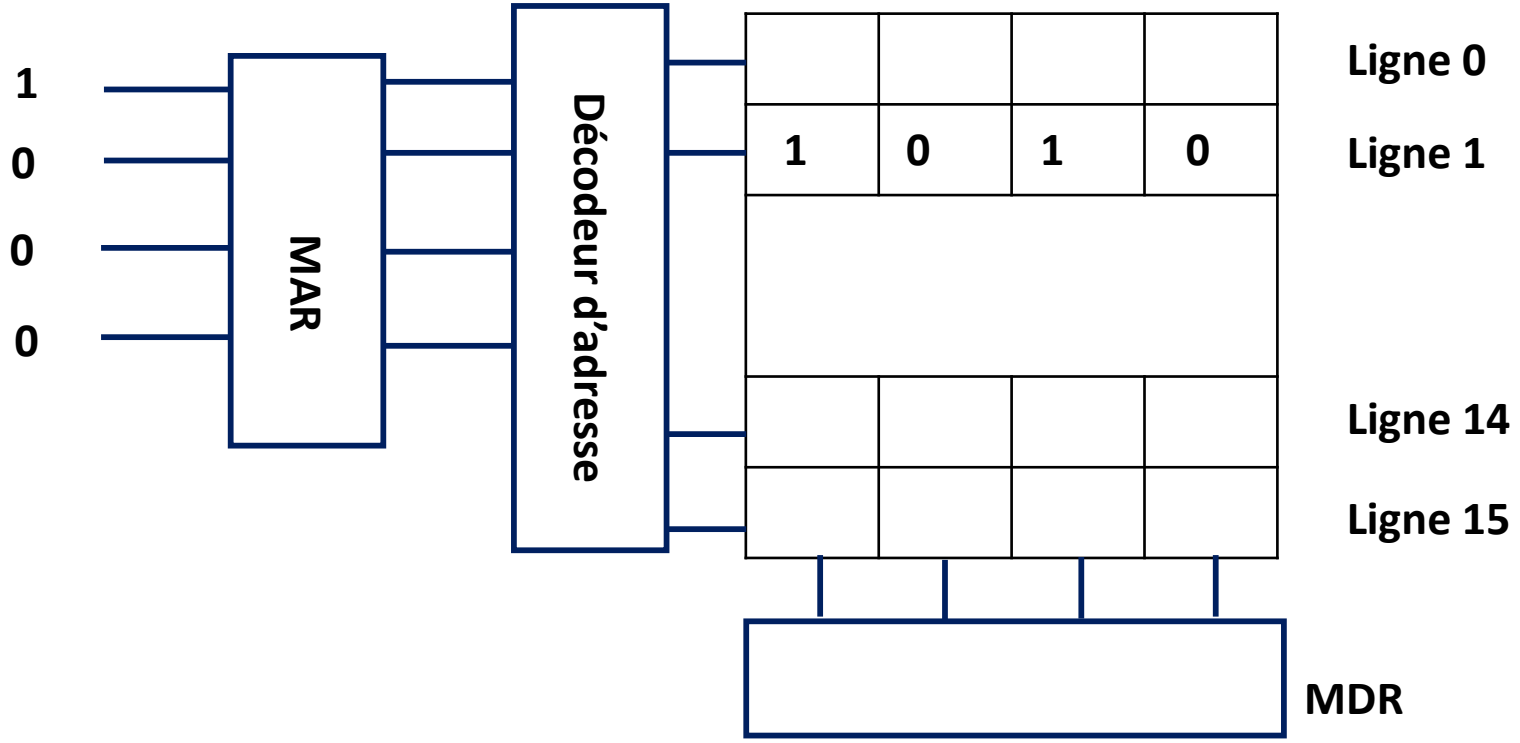
- Pour écrire une information en mémoire, on procède aux opérations suivantes:
  – Charger l'adresse du mot dans le registre MAR.
  – Placer dans le registre MDR l'information binaire à écrire.
  – Lancer la commande d'écriture pour transférer le contenu du registre MDR dans la mémoire .



4- Organisation et opération d'une mémoire RAM

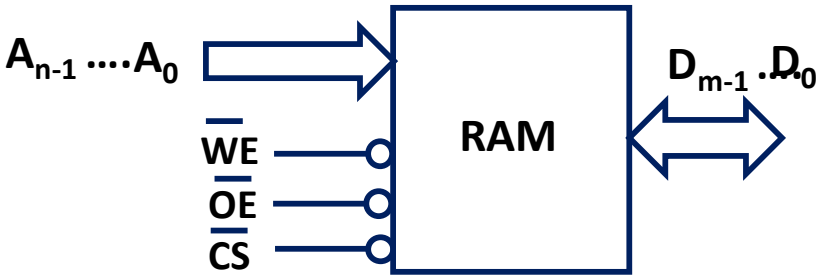
❑ Opération de lecture :

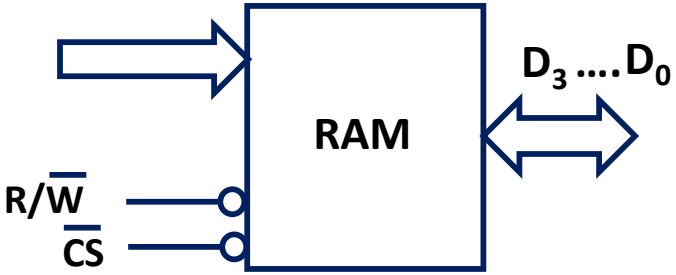
- Pour lire une information en mémoire, on procède aux opérations suivantes:
  – Charger l'adresse du mot à lire dans le registre MAR.
  – Lancer la commande de lecture.
  – L'information est disponible dans le registre MDR au bout d'un certain temps (temps d'accès)

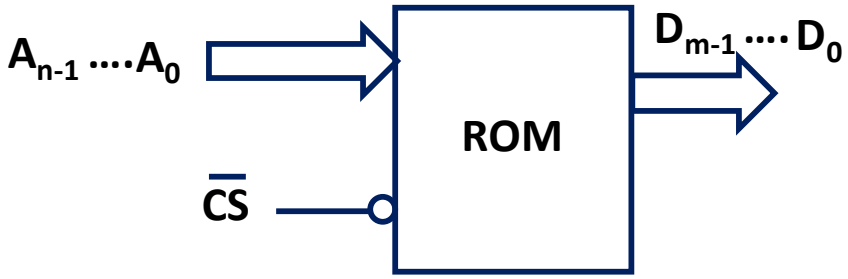


4- Organisation et opération d'une mémoire

- En plus des bus d'adresse et de données, chaque mémoire possède les entrées suivantes:
  - Une entrée de sélection $\overline{CS}$ (chip select) ou $\overline{CE}$ (chip enable) qui permet de l'activer.
    - Sélection de la mémoire si $\overline{CE} = 0$ si non la mémoire est désactivée.
  - Une entrée $\overline{WE}$ (write enable) pour commander le mode lecture (mémorisation des informations) ou écriture (restitution des informations).
  - Une entrée $\overline{OE}$ (output enable) pour la lecture des informations.





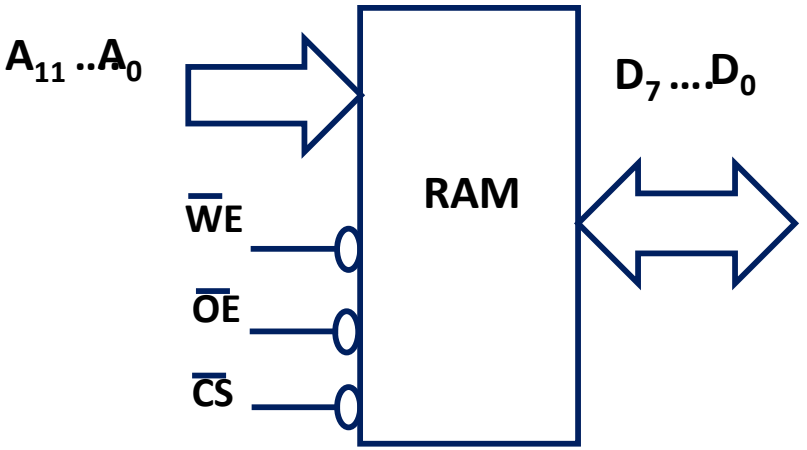


$$R / \overline{W} = 0 \rightarrow \text{Ecriture}$$

$$R / \overline{W} = 1 \rightarrow \text{Lecture}$$

4- Organisation et opération d'une mémoire

❑ Exemple



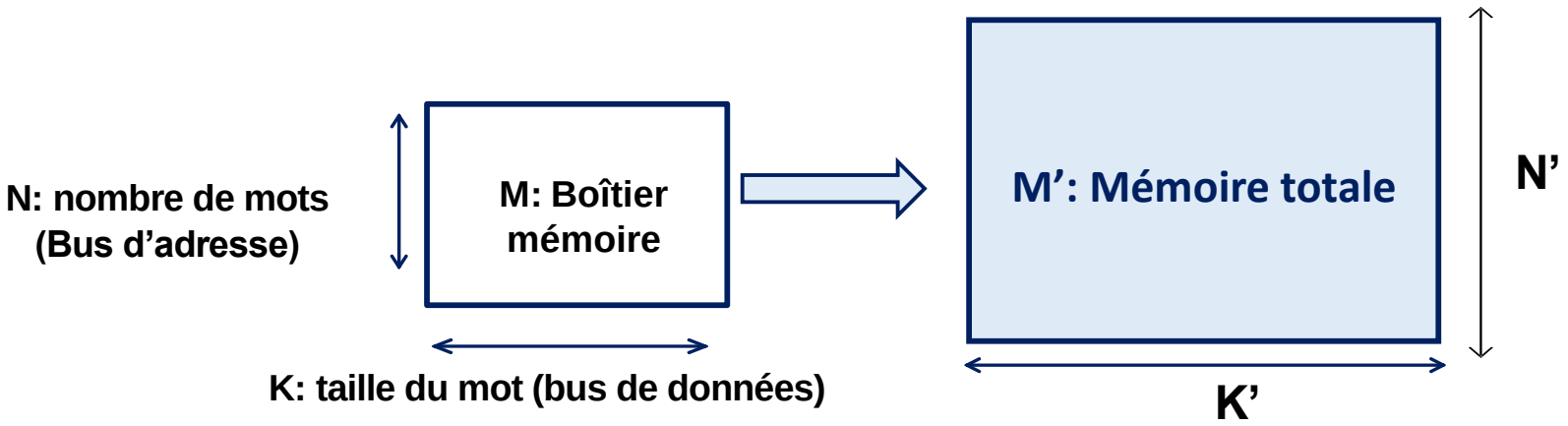
- La capacité de cette mémoire est: C = 2^{12}.8 octets = $2^2.2^{10}$.8 = 4 Ko = 32 Kbits

- Espace adressable : 2^{12} mots binaires = 4 K mots de 8 bits

- Plage d'adresses de cette mémoire:
  - Adresse minimale en hexadécimal: $000
  - Adresse maximale en hexadécimal: $FFFF

5- Extension de la mémoire

- L'extension de la mémoire permet d'augmenter sa capacité et taille de données à partir de l'assemblage de plusieurs boitier mémoire.



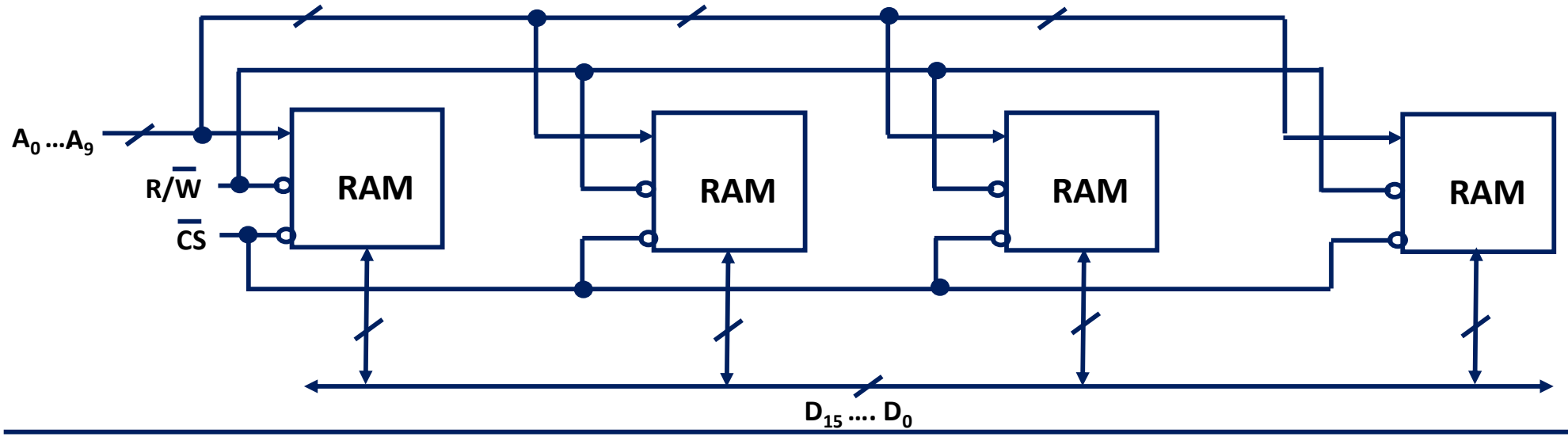
- Afin de déterminer le nombre de boitiers mémoires M à assembler pour réaliser la mémoire M', on calcule:
  - **Ns = N'/N** : Nombre de boîtiers M pour obtenir le nombre de mots de la mémoire M' (extension des lignes d'adresse). Dans ce cas les boitiers M sont sélectionnées simultanément.
  - **Np = K'/K** : Nombre de boitiers M pour obtenir la taille du mot de la mémoire M' (extension du bus de données).). Dans ce cas les boitiers M sont sélectionnées une après une.
- Le nombre totale des boitiers mémoires à assembler pour augmenter la capacité de la mémoire est le produit : **Np.Ns**

5- Extension de la mémoire

❑ Exemple de l'extension de la taille de mots (extension du bus de données)

Réalisation d'une mémoire RAM de capacité 1024×16 à partir d'un boitier mémoire RAM de 1024×4.

- Ns = 1024/1024 = 1 et Np = 16/4 = 4, donc on aura besoin de 4 boitiers mémoires pour augmenter la taille du mot binaire (augmentation du bus de données).
- 1024 = 2^{10}, donc le bus d'adresse est composé de 10 bits (10 lignes d'adresse): A9….A0
- Le boitier mémoire de base a un bus de donnée de 4 bits: D3….D0
- La mémoire totale a un bus de donnée de 16 bits: D15….D0



5- Extension de la mémoire

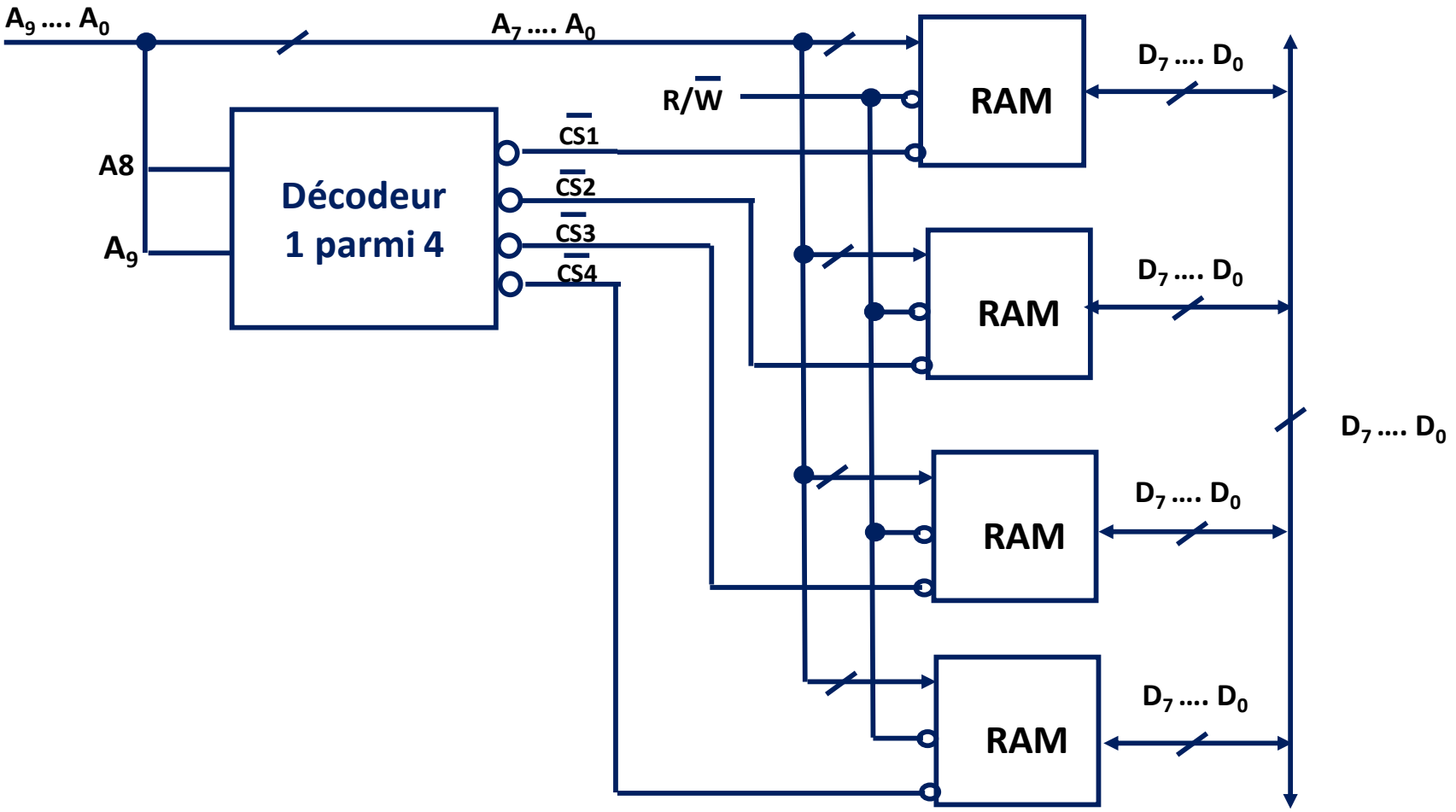
❑ Exemple de l'extension de nombre de mots (extension du bus d'adresses)

Réalisation d'une mémoire RAM de capacité 1024×8 à partir d'un boitier mémoire RAM de 256×8.

- Ns = 1024/256 = 4 et Np = 8/8 = 1, donc on aura besoin de 4 boitiers mémoires pour augmenter la nombre de mots (augmentation du bus d'adresses).
- 1024 = 2^{10}, donc le bus d'adresse de la mémoire totale est composé de 10 bits (10 lignes d'adresse): A9….A0
- 256 = 2^{8}, donc le bus d'adresse du boitier mémoire est composé de 8 bits (8 lignes d'adresse): A7….A0
- Le boitier mémoire et la mémoire totale de base a un bus de donnée de 8 bits: D7….D0
- Dans ce cas, on va décomposer le bus d'adresse de la mémoire totale en deux parties
  - A7….A0 pour le bus d'adresse du boitier mémoire.
  - A9 et A8 pour sélectionner les quatre boitiers mémoires à partir de leur entrée CS, en se basant sur un décodeur d'adresse deux vers quatre (1 parmi 4).

5- Extension de la mémoire

❑ Exemple de l'extension de nombre de mots (extension du bus d'adresses)



5- Extension de la mémoire

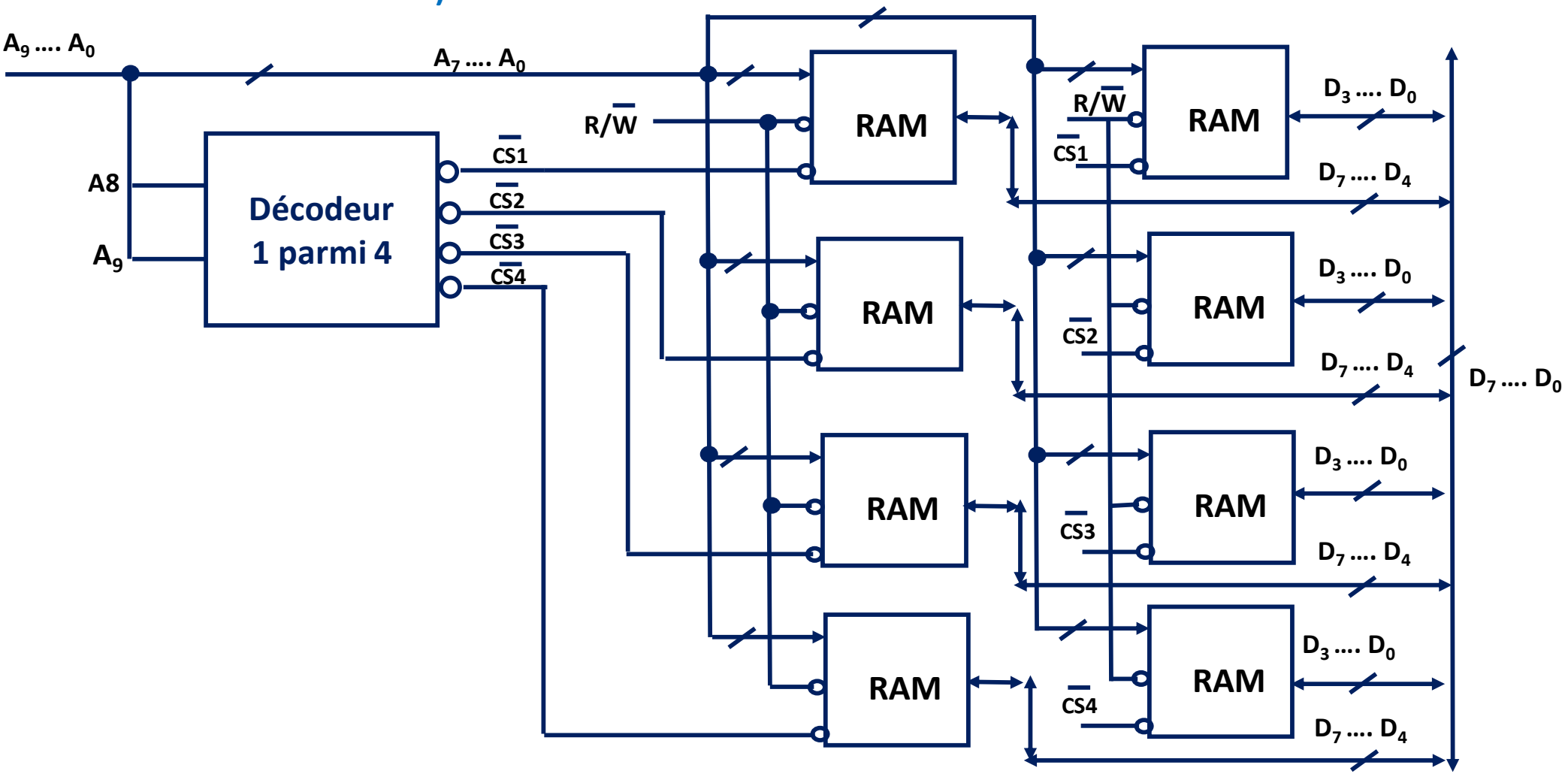
❑ Exemple de l'extension de nombre de mots et la taille du mot (extension du bus d'adresses et du bus de données)

Réalisation d'une mémoire RAM de capacité 1024×8 à partir d'un boitier mémoire RAM de 256×4.

- Ns = 1024/256 = 4 et Np = 8/4 = 2, donc on aura besoin de 4.2 = 8 boitiers mémoires pour augmenter la nombre de mots et la taille du mot.
- 1024 = 2^{10}, donc le bus d'adresse de la mémoire totale est composé de 10 bits (10 lignes d'adresse): A9….A0
- 256 = 2^{8}, donc le bus d'adresse du boitier mémoire est composé de 8 bits (8 lignes d'adresse): A7….A0
- La mémoire totale de base a un bus de donnée de 8 bits: D7….D0
- Le boitier mémoire a un bus de données de 4 bits: D3…D0
- Dans ce cas, on va décomposer le bus d'adresse de la mémoire totale en deux parties
  - A7….A0 pour le bus d'adresse du boitier mémoire.
  - A9 et A8 pour sélectionner les huit boitiers mémoires à partir de leur entrée CS, en se basant sur un décodeur d'adresse deux vers quatre (1 parmi 4).

5- Extension de la mémoire

- **Exemple de l'extension de nombre de mots et la taille du mot (extension du bus d'adresses et du bus de données)**



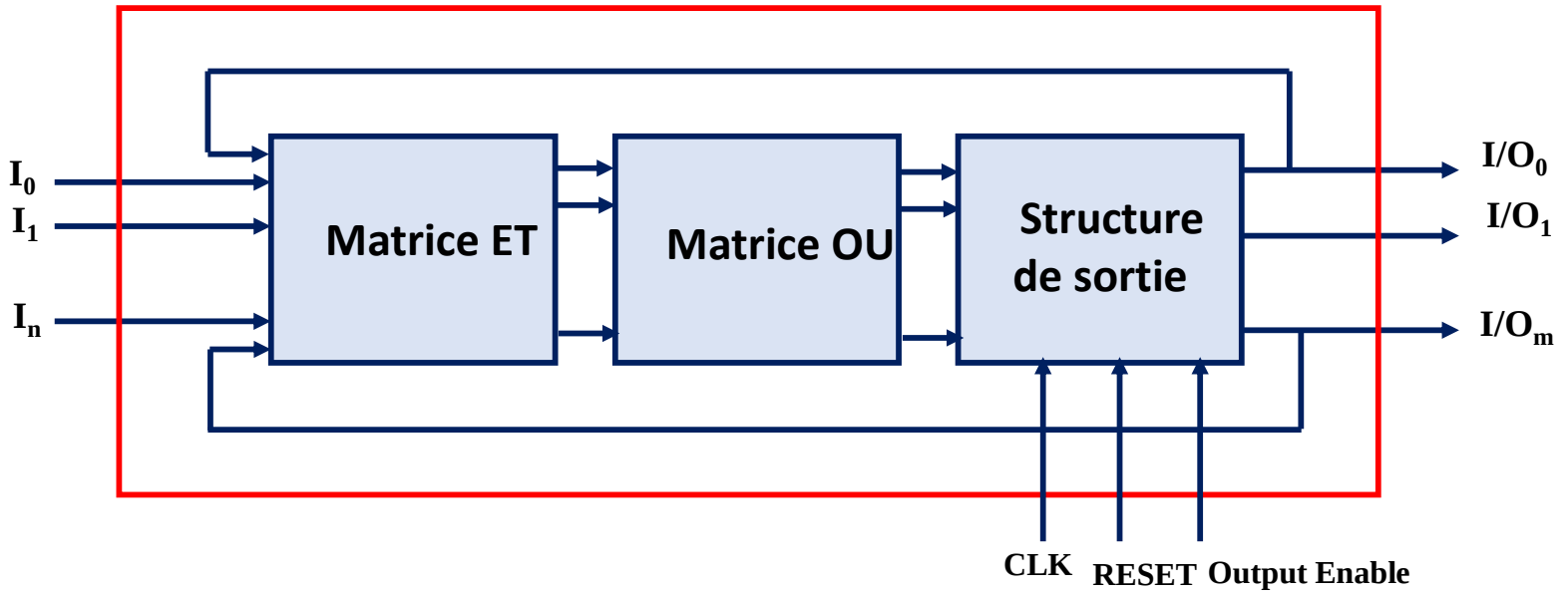
Chapitre III: Circuits logiques programmables

1- Définition

- Un circuit logique programmable ou **PLD (Programmable Logical Device),** est un circuit intégré logique qui peut être programmé après sa fabrication.

2- Structure d'un PLD

- Un circuit PLD contient un ensemble de **portes ET** sur lesquels se connectent les variables d'entrée et leurs compléments. Un ensemble de **portes OU** sur lesquels les sorties des opérateurs ET sont connectées.
- Une éventuelle structure de sortie : Portes inverseuses, Portes 3 états, Multiplixeurs, Portes XOR, Bascules, …etc.

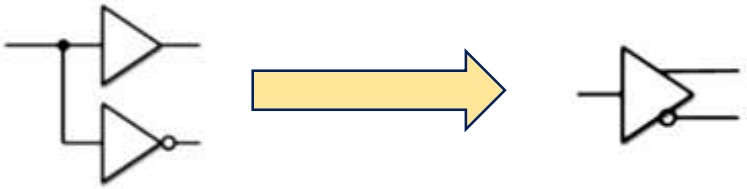


2- Structure d'un PLD

- Un circuit programmable **n × m** possède **n entrées** et **m sorties**.
- L'ensemble des portes logiques forment des matrices (matrice OU et matrice ET).
- Les interconnexions de ces matrices sont assurées par des fusibles qui sont « grillés » lors de la programmation. Lorsqu'un PLD est vierge toutes les connexions sont assurées (fusibles intacts).
- La technique des fusibles est en voie de disparition : le PLD est programmé une seule fois.
- Actuellement la technique la plus répandue utilise le transistor MOSFET à la place du fusible pour programmer et reprogrammer le PLD par utilisation d'un effacement électrique.

Exemple de symbolisation avec fusibles

- Exemple de symbole d'une entrée avec son complément.



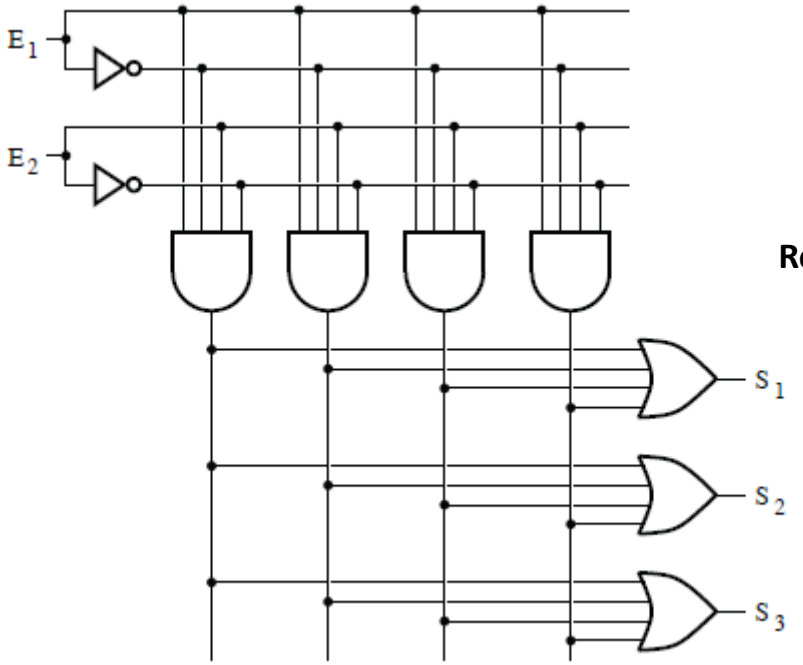
- Exemple de symbole d'une portes ET à trois entrées avec fusibles d'interconnection.



Les croix représentent les fusibles intacts (non détruits).

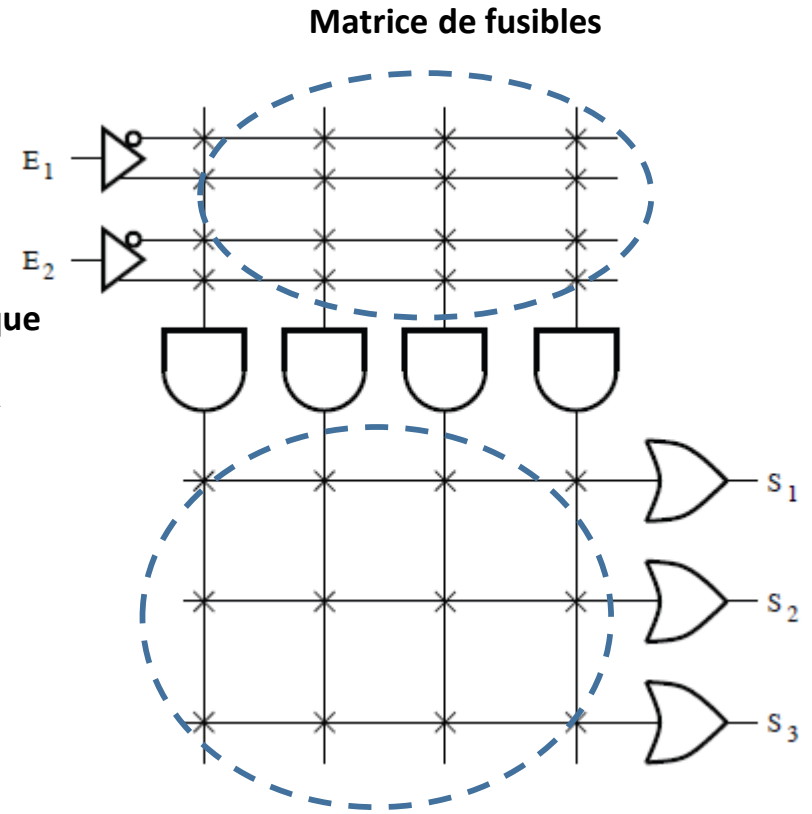
2- Structure d'un PLD

Exemple de circuit programmable 2×3



Représentation symbolique

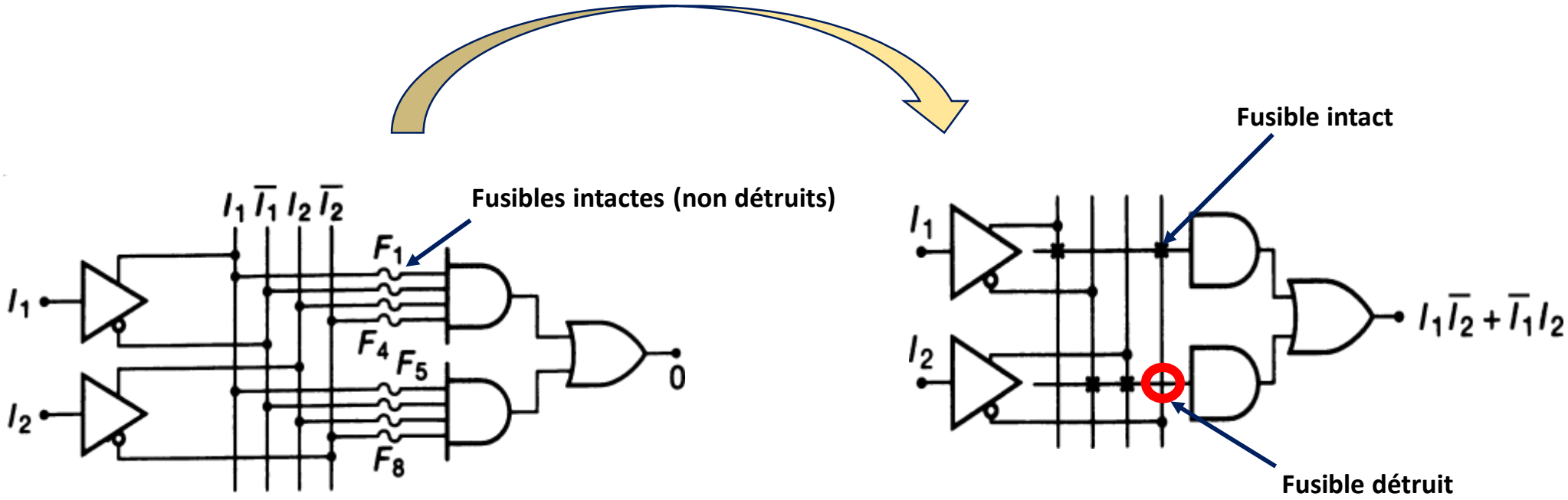
Matrice de fusibles



2- Structure d'un PLD

Exemple de programmation d'un OU exlusif

Représentation symbolique après programmation (destruction des fusibles)



3- Différents types de circuits PLD simple : SPLD

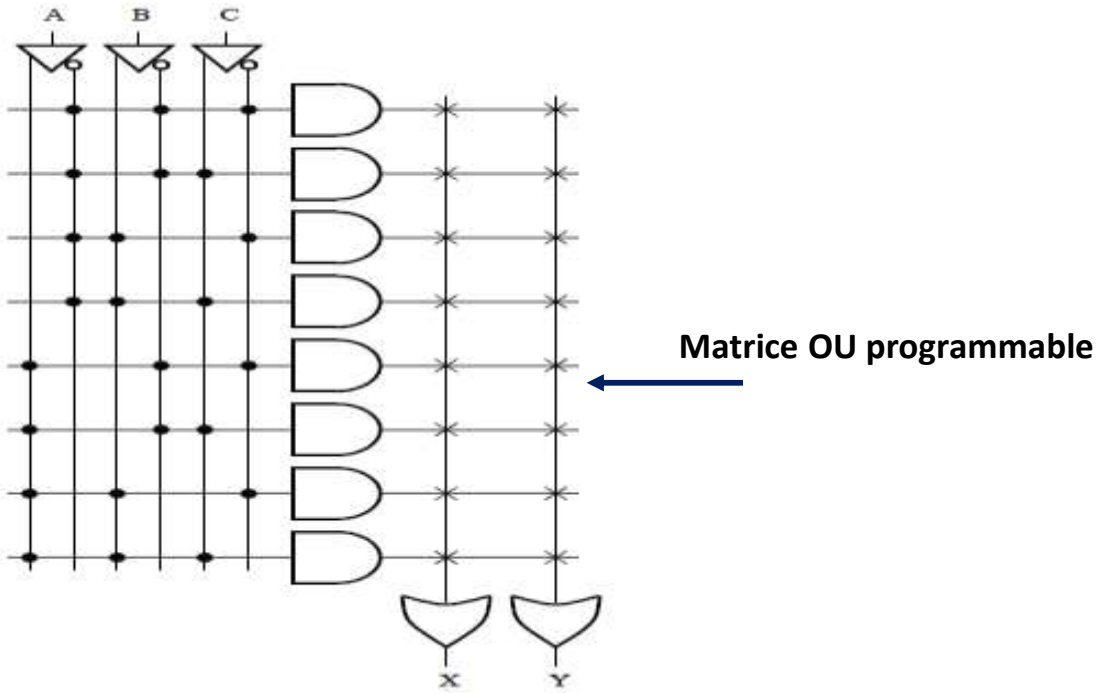
3-1 Circuits PROM

- Un circuit PROM (Programmable Read Only Memory) est composée d'un ensemble de portes ET précâblées (fixes), qui jouent le rôle de décodeur, et d'un ensemble de portes OU programmables.
- La programmation des PROM s'effectue par destruction de fusible, ils sont programmables une seule fois.

Exemple de PROM 3×2

- La matrice ET connecte toutes les combinaisons logiques des trois variables. Il faut utiliser donc la représentation algébrique canonique somme de produit
- 3 entrées donc 8 combinaisons possibles:

$$\overline{A}.\overline{B}.\overline{C} \rightarrow A.B.C$$



Les points représentent des entrées non programmables (fixes: pas de fusible à détruire).

3- Différents types de circuits PLD simple : SPLD

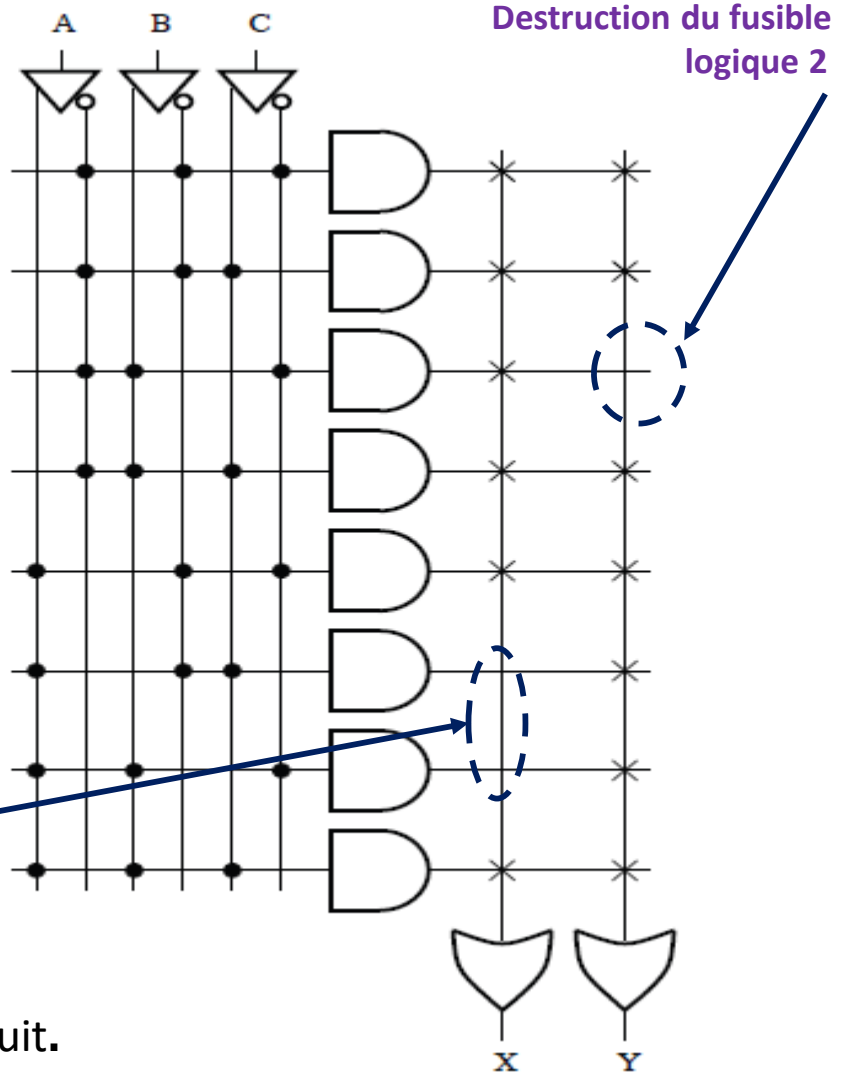
3-1 Circuits PROM

❑ **Exemple de réalisation des fonctions logiques par PROM 3×2**

- Réaliser les deux fonctions logiques X et Y:

$$X(A,B,C) = \sum(0,1,2,3,4,7)$$

$$Y(A,B,C) = \sum(0,1,3,4,5,6,7)$$



Destruction du fusible de l'entrées logique 2

Destruction des fusibles des entrées logique 5 et 6

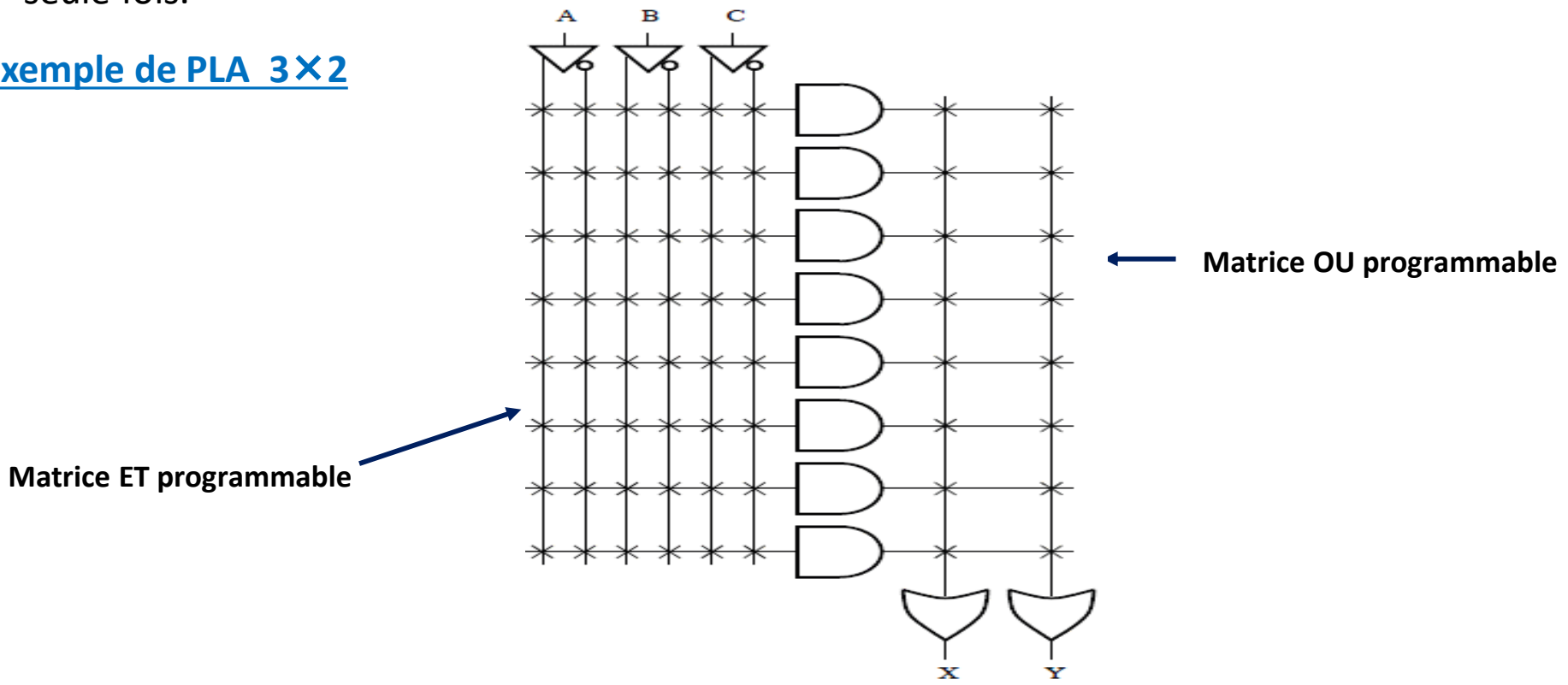
NB: Absence de croix représente un fusible détruit.

3- Différents types de circuits PLD simple : SPLD

3-2 Circuits PLA

- Un circuit PLA (Programmable Logic Array) est composée d'un ensemble de portes ET programmables, et d'un ensemble de portes OU programmables.
- La programmation des PLA s'effectue par destruction de fusible. , ils sont programmables une seule fois.

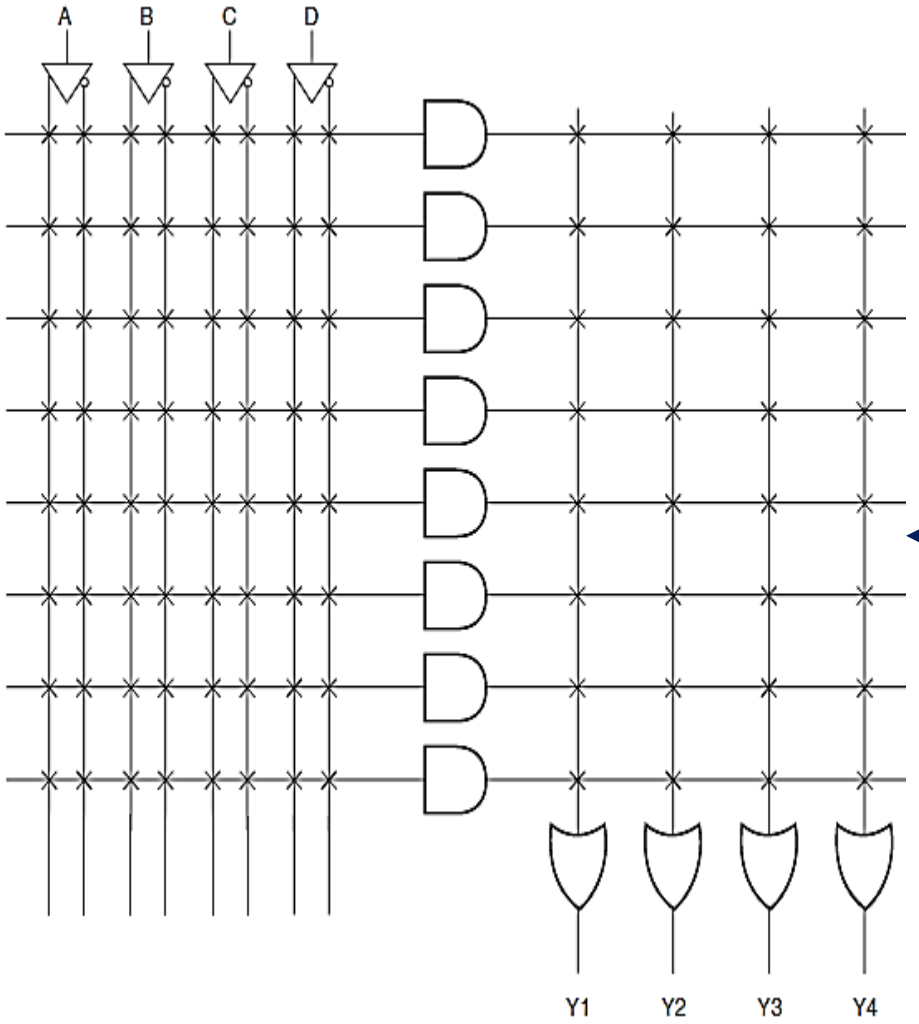
Exemple de PLA 3×2



3- Différents types de circuits PLD simple : SPLD

3-2 Circuits PLA

Exemple de PLA 4×4



Matrice OU programmable

Matrice ET programmable

3- Différents types de circuits PLD simple : SPLD

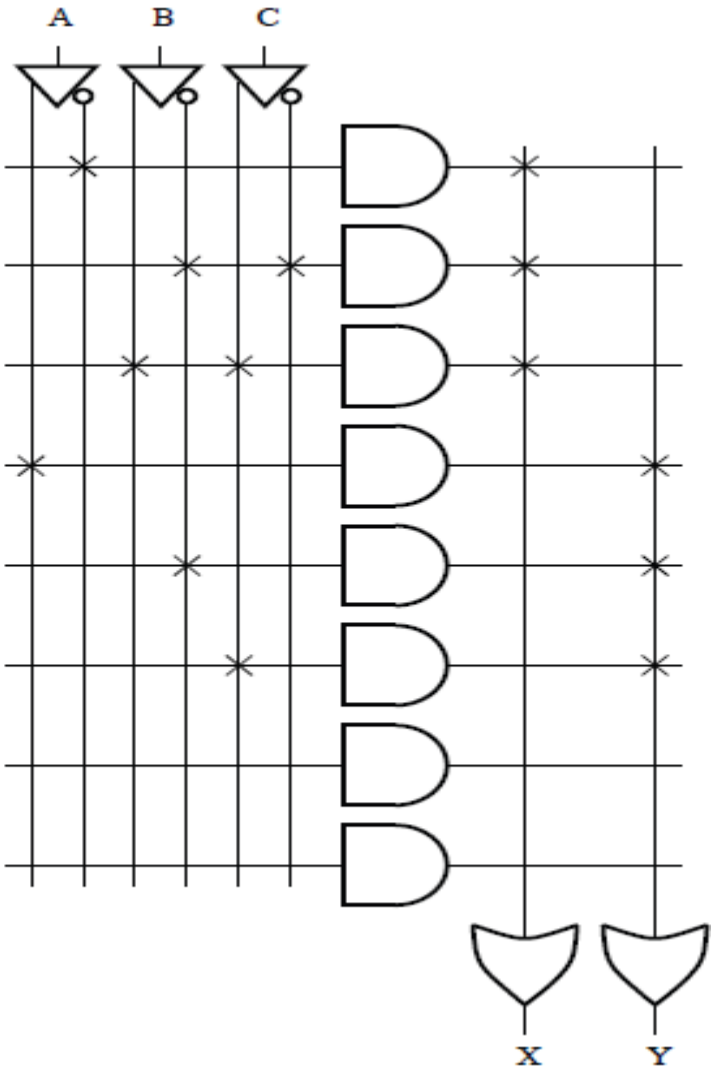
3-2 Circuits PLA

❑ **Exemple de réalisation des fonctions logiques par PLA 3×2**

- Réaliser les deux fonctions logiques X et Y de l'exemple précédent avec une forme algébrique simplifiée:

$$X(A,B,C) = \overline{A} + \overline{B}.\overline{C} + B.C$$

$$Y(A,B,C) = A + \overline{B} + C$$

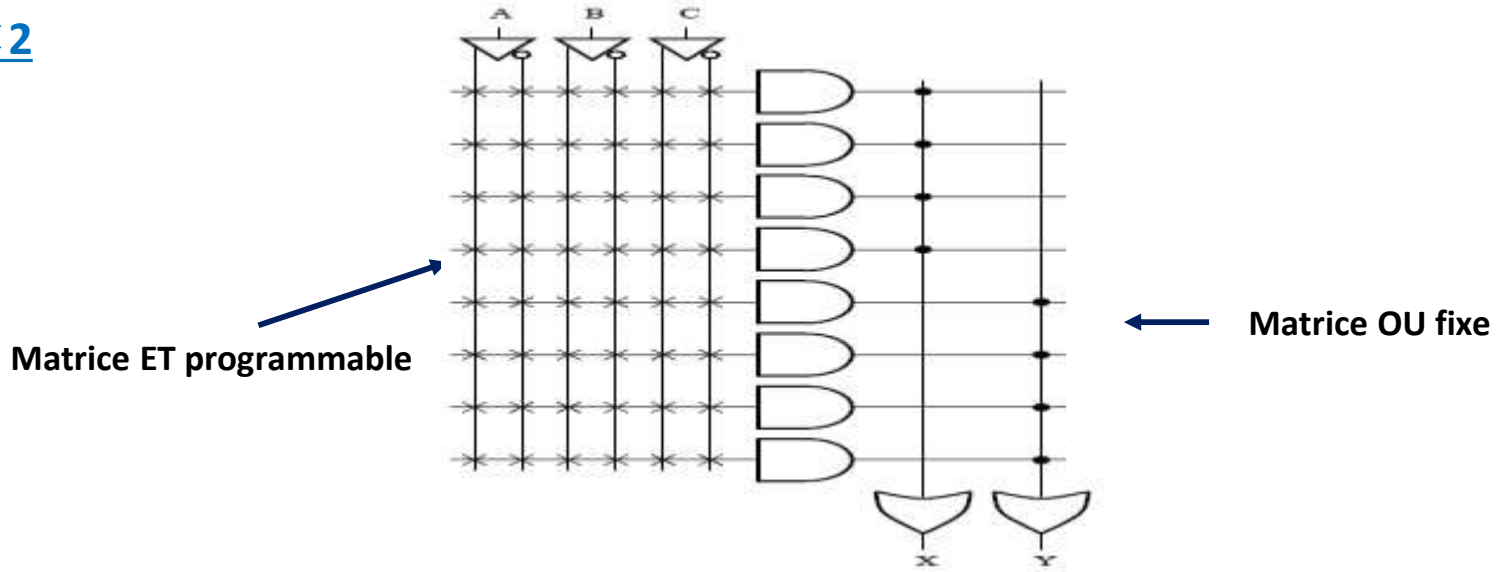


3- Différents types de circuits PLD simple : SPLD

3-3 Circuits PAL

- Un circuit PAL (Programmable Array Logic) est composée d'un ensemble de portes ET programmables, et d'un ensemble de portes OU fixe.
- Les premiers PAL, apparus à la fin des années 1970, présentaient une architecture plus simple que celle des PLA, ce qui permettait de concevoir des circuits plus compacts, plus rapides et moins couteus.
- La programmation des PAL s'effectue par destruction de fusible, ce qui correspond à un circuit ouvert. Ils ne sont programmées qu'une seule fois et ne peuvent pas être modifiées après fabrication.

Exemple de PAL 3×2



3- Différents types de circuits PLD simple : SPLD

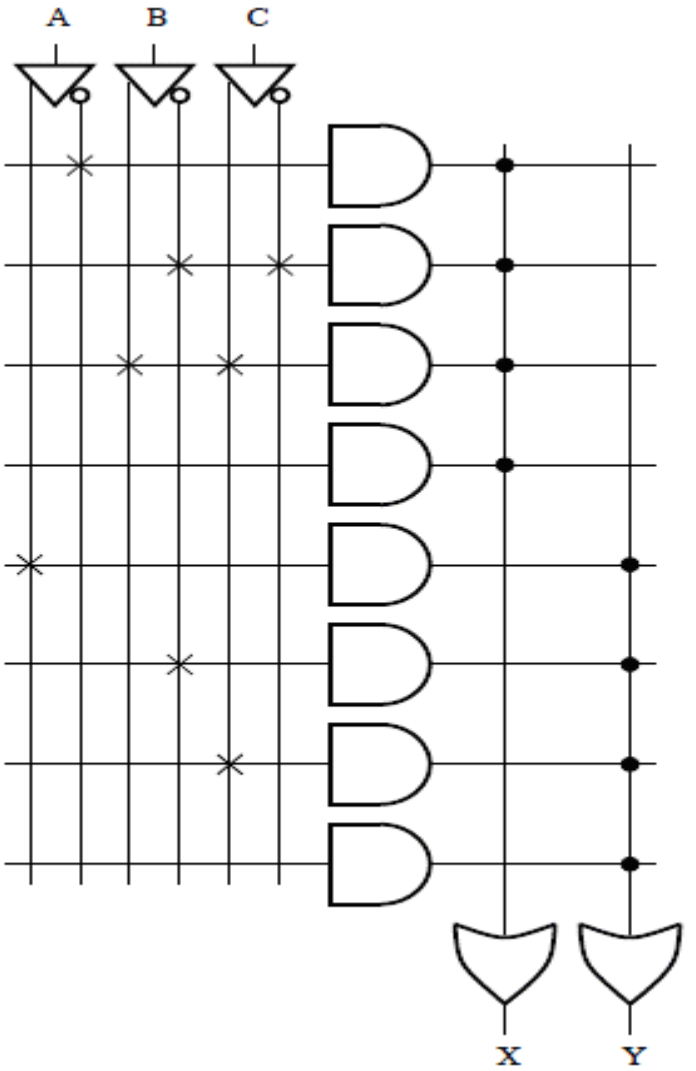
3-3 Circuits PAL

❑ **Exemple de réalisation des fonctions logiques par PLA 3×2**

- Réaliser les deux fonctions logiques X et Y de l'exemple précédent avec une forme algébrique simplifiée:

$$X(A,B,C) = \overline{A} + \overline{B}.\overline{C} + B.C$$

$$Y(A,B,C) = A + \overline{B} + C$$

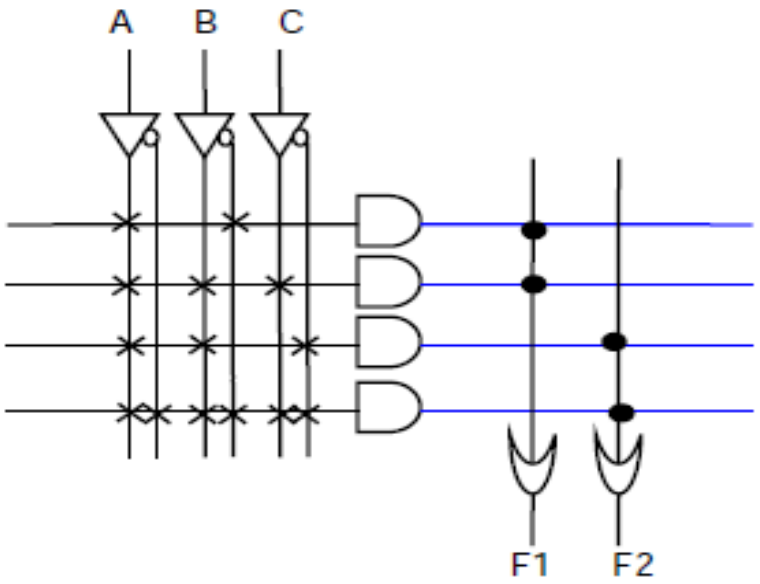


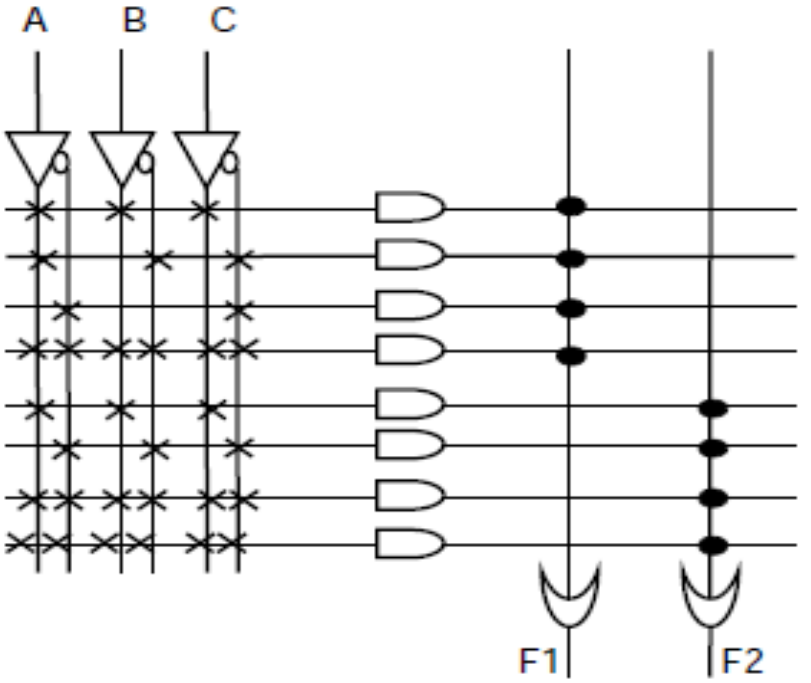
3- Différents types de circuits PLD simple : SPLD

3-3 Circuits PAL

❑ **Exercice d'application 1**

- Préciser le type du circuit et donner les équations logiques de F1 et F2 pour les deux circuits ci-dessous:



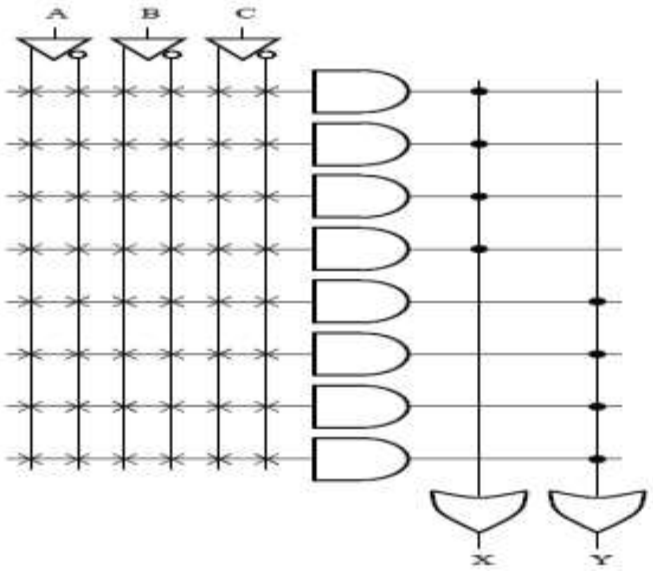


3- Différents types de circuits PLD simple : SPLD

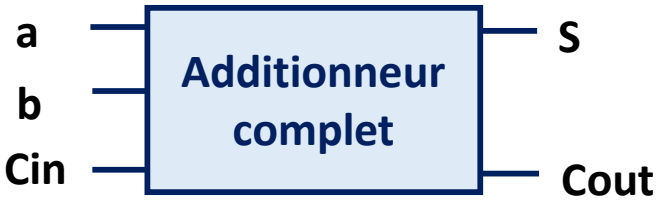
3-3 Circuits PAL

❑ **Exercice d'application 2**

- Réaliser un additionneur complet par un circuit PAL



a	b	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Equations logiques

$$S = \bar{a}.\bar{b}.Cin + \bar{a}.b.\overline{Cin} + a.\bar{b}.\overline{Cin} + a.b.Cin$$

$$Cout = \bar{a}.b.Cin + a.\bar{b}.Cin + a.b.\overline{Cin} + a.b.Cin$$

3- Différents types de circuits PLD simple : SPLD

3-4 Circuits GAL

- Un circuit GAL est une version améliorée du PAL dans lequel la configuration peut être effacée et reprogrammée plusieurs fois.
- Les GAL sont développés dans les années 1980.
- les GAL sont réalisés par utilisation de la technologie CMOS (transistors MOSFET).

4 Structure de sortie d'un circuit PAL/GAL

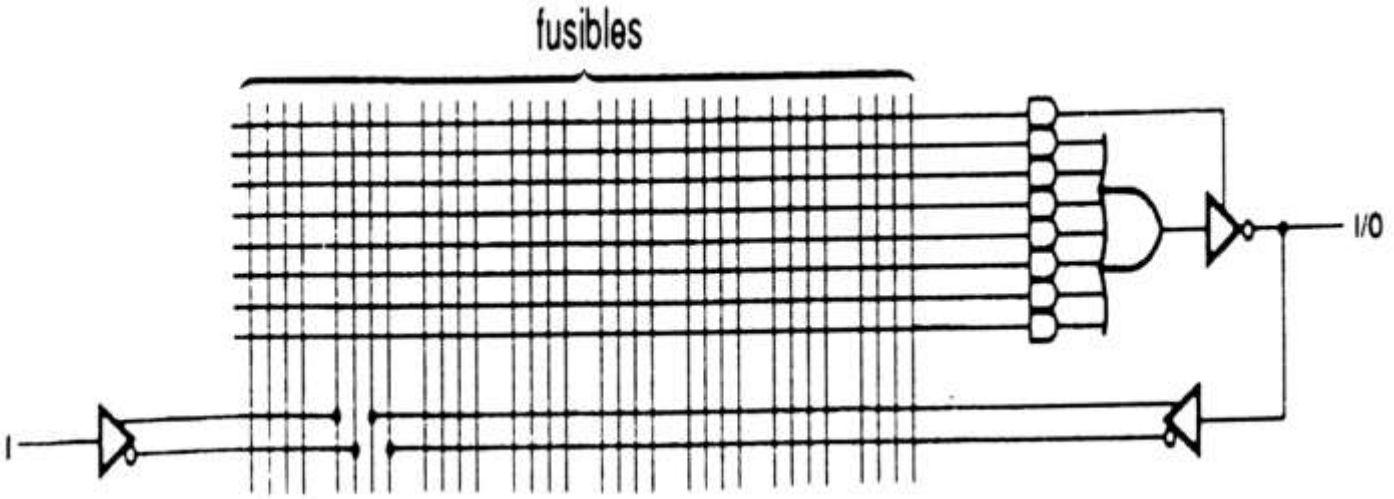
- Il existe 3 structures de sortie qui peuvent-être aussi de entrées/sorties : Combinatoire, Séquentielle et Versatiles.

4-1- Entrées / Sorties combinatoires

- Sortie 3 états rebouchée vers la matrice
  - ❑ une sortie peut servir de variable intermédiaire
  - ❑ une sortie peut servir d'entrée (en mode haute impédance)
- Il existe trois types:
  - **H** : (High) Porte ET suivit d'une Porte OU. Sortie active à l'état haut.
  - **L** : (Low) Porte ET suivit d'une Porte NON OU. Sortie active à l'état bas.
  - **C** : (Combinée) programmable en type H ou L.

4- Structure de sortie d'un circuit PAL/GAL

4-1- Entrées / Sorties combinatoires



Buffer à trois états



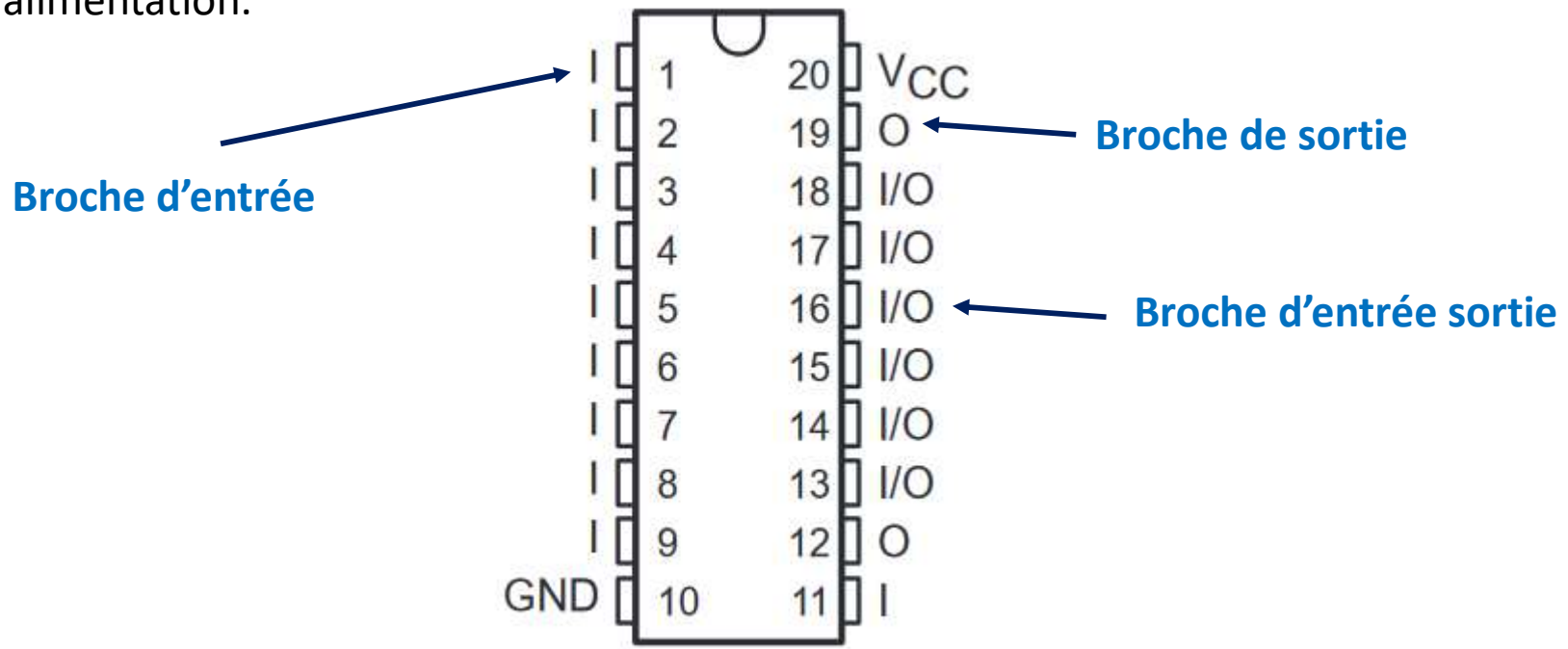
E=0	Haute impédance (la sortie est déconnectée)
E = 1	Inverseur normale

4- Structure de sortie d'un circuit PAL/GAL

4-1- Entrées / Sorties combinatoires

Exemple PAL entrée / sorties combinatoires de type L (activation niveau bas) PAL 16L8 :

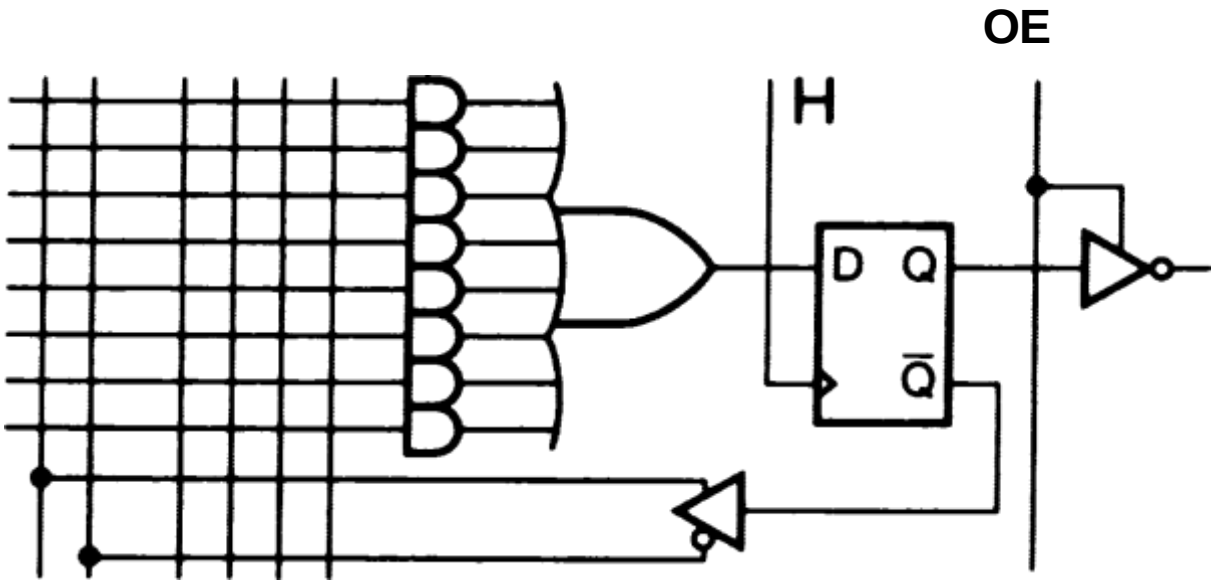
- 10 broches configurables uniquement en entrée.
- 2 broches configurables uniquement en sortie.
- 6 broches configurables en entrée ou en sortie.
- 2 broches d'alimentation.



4- Structure de sortie d'un circuit PAL/GAL

4-2 Entrées / Sorties séquentielles, à registre

- Une bascule D pour la logique séquentielle synchronisée sur une horloge H.
- La sortie ne peut pas servir d'entrée
- Une commande OE (Ouput Enable) permet de désactiver la sortie (mode haute impédance)

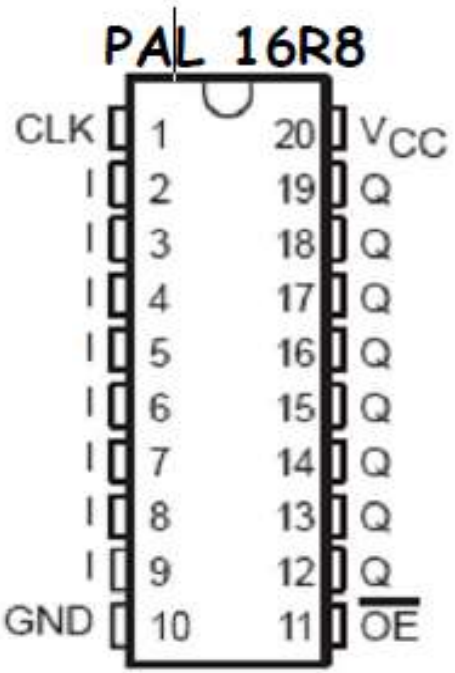


4- Structure de sortie d'un circuit PAL/GAL

4-2 Entrées / Sorties séquentielles, à registre

Exemple PAL entrée / sorties combinatoires de type R (à registre) PAL 16R8 :

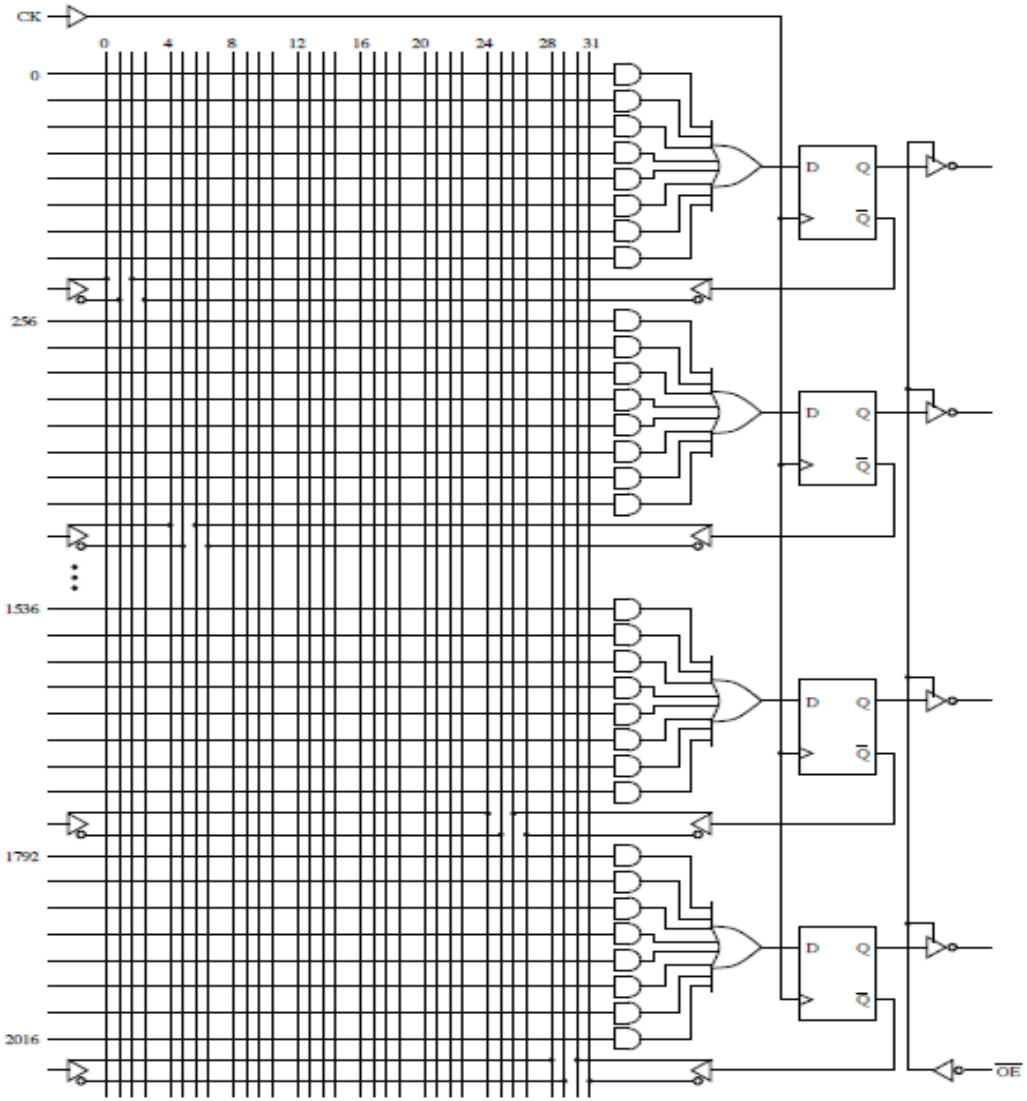
- 8 broches configurables uniquement en entrée.
- 1 broche d'entrée horloge de l'ensemble des 8 bascules D.
- 1 broche de validation des 8 sorties(Output Enable).
- 8 broches configurables en sortie ou en entrée.
- 2 broches d'alimentation.



4- Structure de sortie d'un circuit PAL/GAL

4-2 Entrées / Sorties séquentielles, à regist

Structure interne PAL 16R8 :

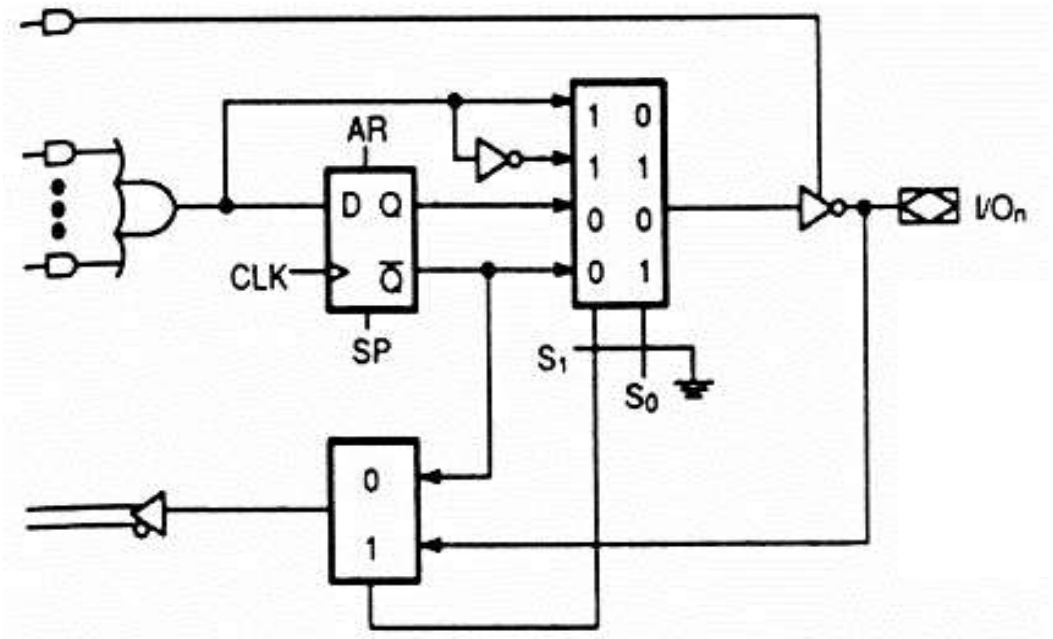


4-Structure de sortie d'un circuit PAL/GAL

4-3- Structure versatile

- La sortie peut être programmée en mode combinatoire ou en registre.
- La sortie est une macro-cellule logique (OLMC: Output Logic MacroCell).

❑ **Exemple de macro-cellule**

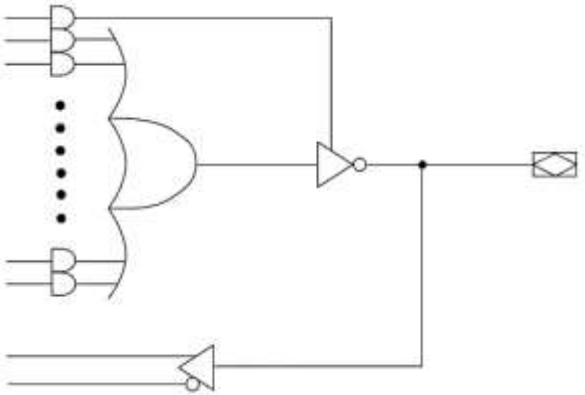


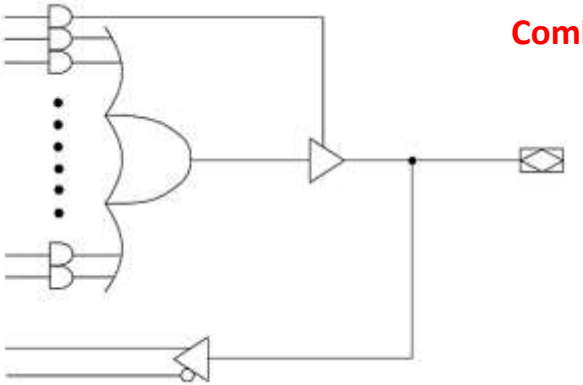
S1	S0	Sortie
0	0	Registre actif bas
0	1	Registre actif haut
1	0	Combinatoire actif bas
1	1	Combinatoire actif haut

4-Structure de sortie d'un circuit PLD

4-3- Structure versatile

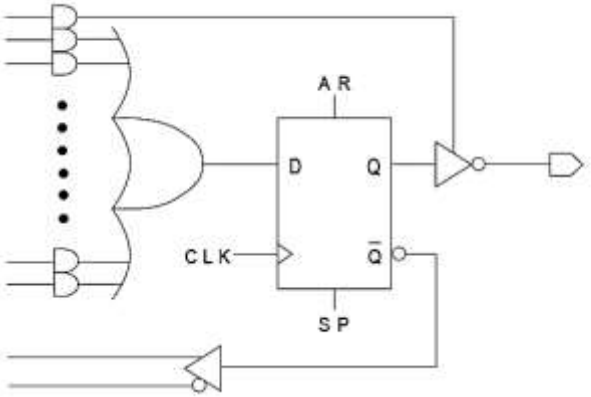
Combinatoire Active bas:
S1S0= 10

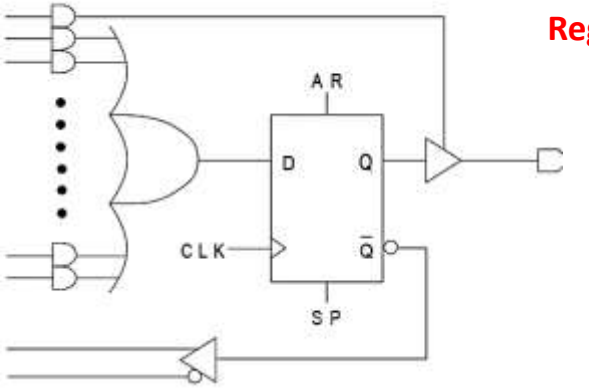




Combinatoire Active haut:
S1S0= 11

Registre Active bas:
S1S0= 00





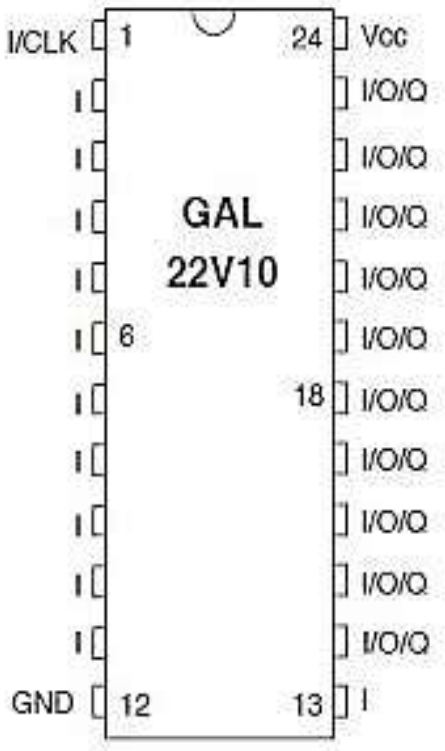
Registre Active haut:
S1S0= 01

4-Structure de sortie d'un circuit PLD

Exemples de circuits PLD versatiles

Exemple PAL/GAL entrée / sorties versatiles (de type V) GAL 22V10 :

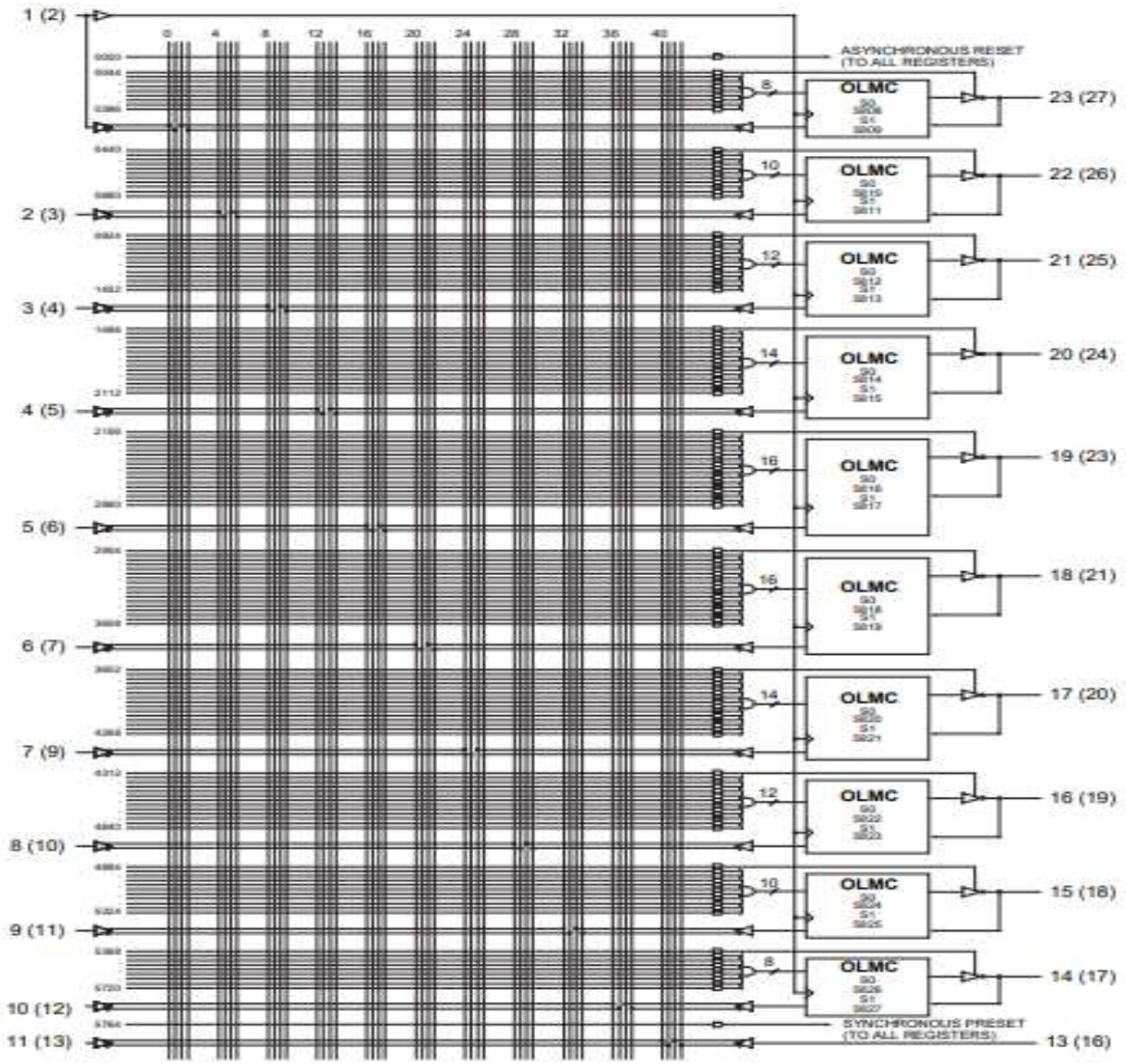
- 22 broches configurables uniquement en entrée.
- 1 broche d'entrée horloge de l'ensemble des bascules D.
- 10 broches configurables en sortie ou en entrée.
- 2 broches d'alimentation.



4-Structure de sortie d'un circuit PLD

Exemples de circuits PLD versatiles

Structure interne du circuit GAL 22V10 :



5- Programmation des circuits PLD

Etapes de programmation d'un PLD simple: PAL ou GAL

- Identifier les entrées et sorties du circuits à réaliser.
- Définir les fonctions logiques à réaliser (souvent données sous forme de tables de vérité ou d'équations logiques).

⇩

- Utiliser un langage comme ABEL, CUPL… pour décrire les équations logiques.

⇩

- Compiler le code avec un logiciel de développement compatible (ex. WinCUPL…).
- Simuler le comportement du PAL pour vérifier le fonctionnement du circuit.

⇩

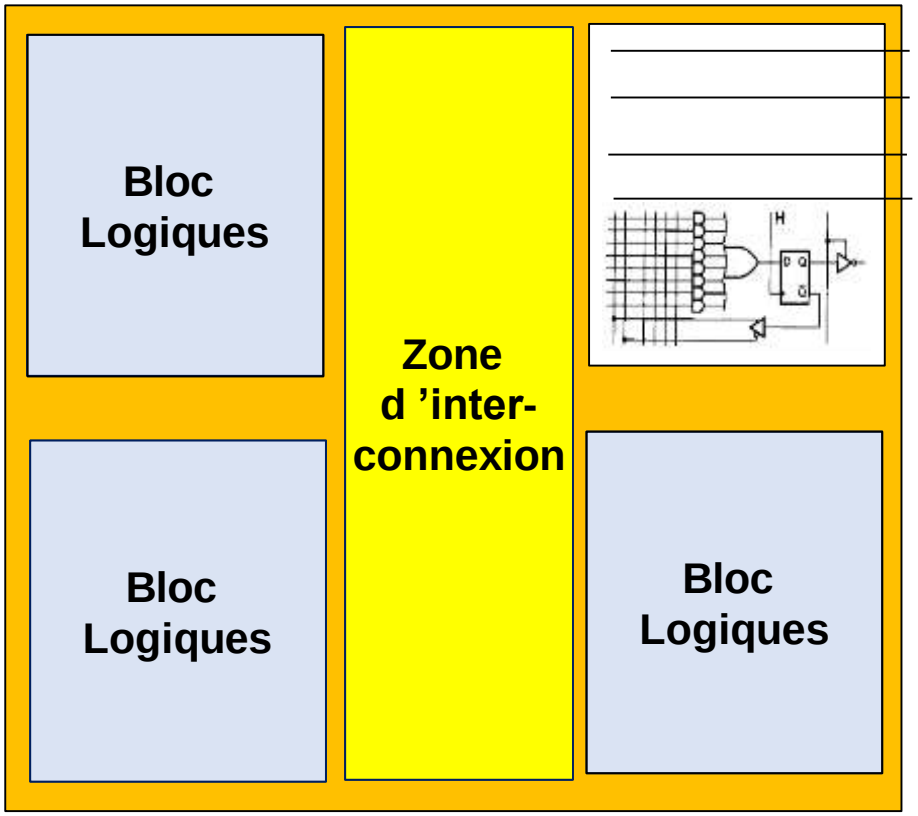
- Le compilateur produit un fichier binaire ou JEDEC (.jed) (Joint Electron Device Engineering Council) contenant les données de programmation des fusibles internes du PAL.

⇩

- Programmer le circuits en chargant le fichier .jed via un programmateur.
- Intégrer le PLD dans le montage électronique réalisé et vérifier que les signaux de sortie respectent bien les fonctions logiques établies.

6- Circuits programmables CPLD

- Un circuit CPLD (Complex Programmable Logic Device) est constitué de blocs logiques reliés entre eux par une matrice d'interconnexions programmables.
- Ce circuit correspond à plusieurs PAL reliées entre elles par une zone d'interconnexion.
- Ces circuits offrent une capacité de portes et de configuration plus élevée que celle des PAL.

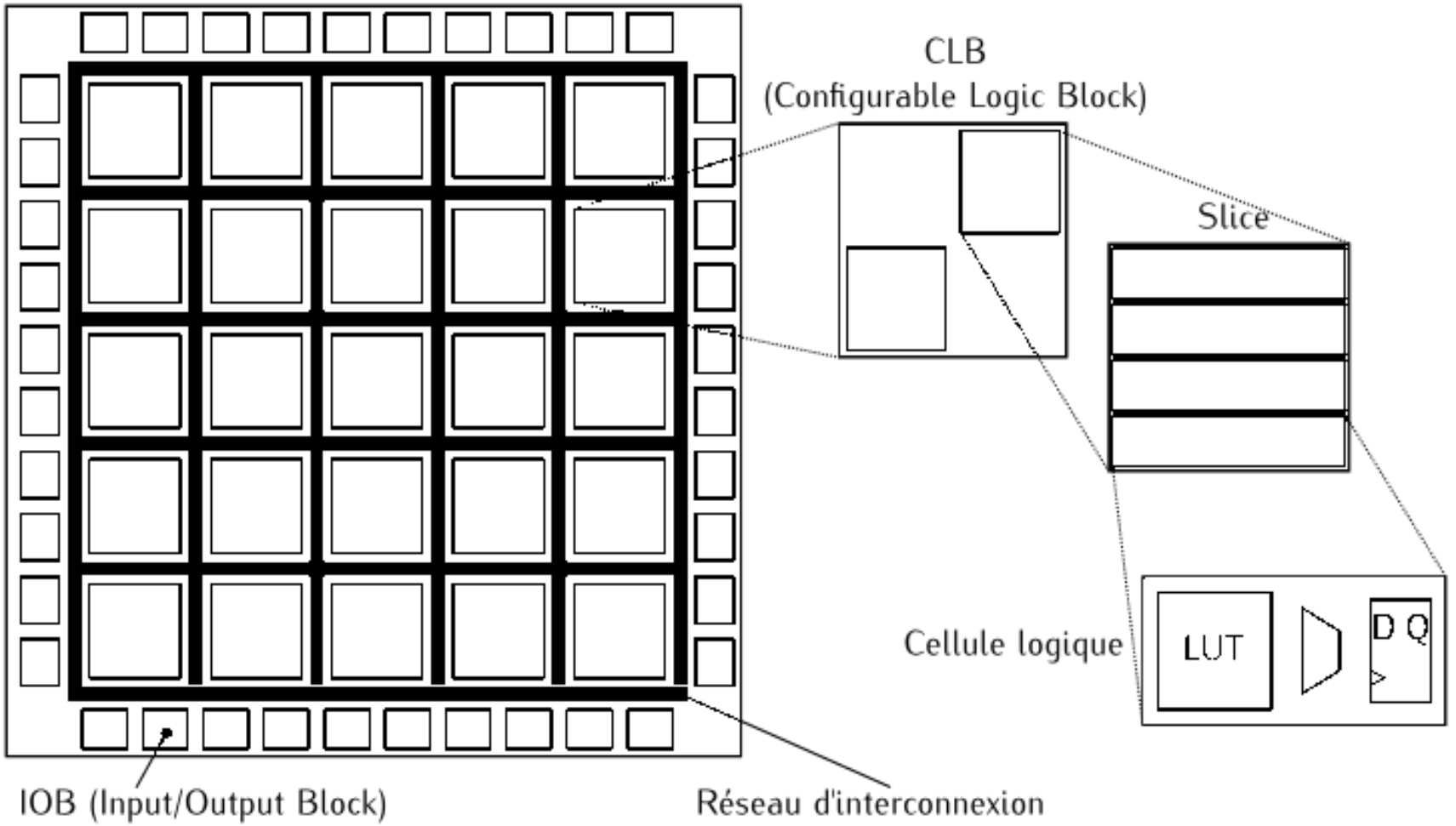


7- Circuits programmables FPGA

7-1 Architecture d'un circuit FPGA

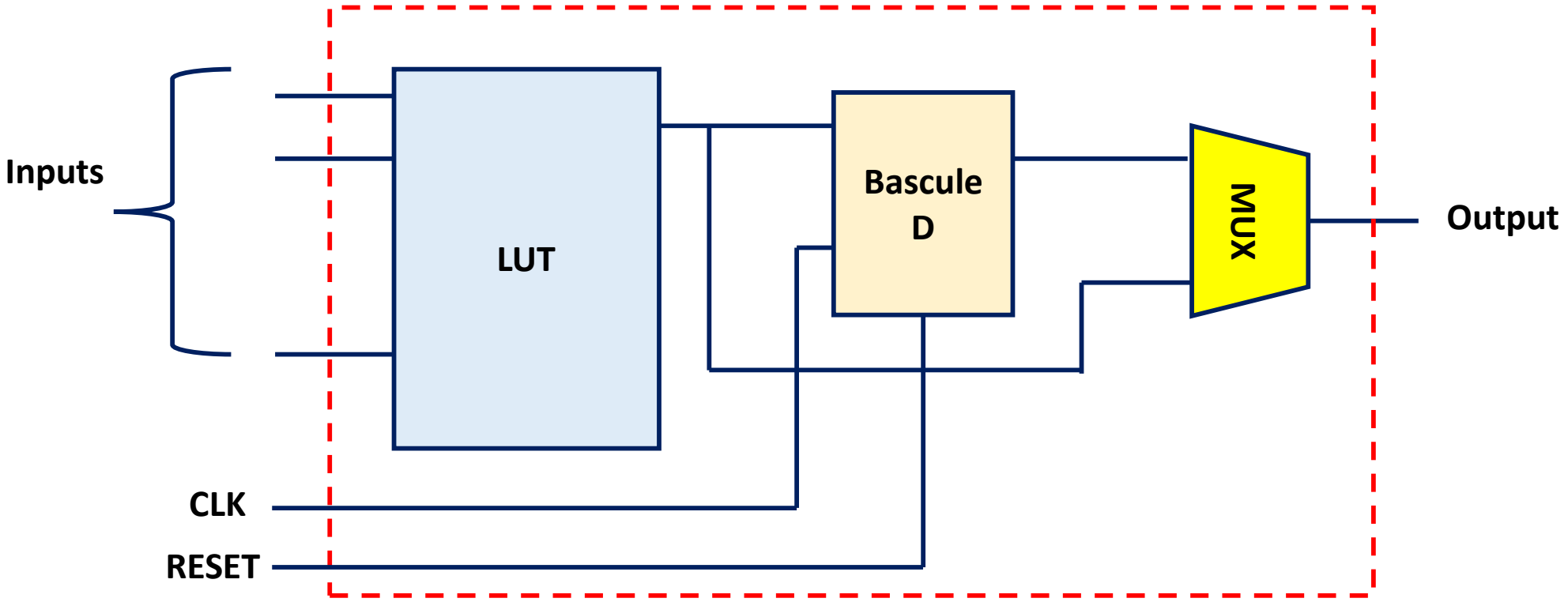
- Un circuit FPGA (Field Programmable Gate Array), est un circuit intégré programmable qui peut être reconfiguré après sa fabrication.
- Ces circuits sont si puissants qu'ils peuvent contenir plusieurs milliers, voire millions, de portes logiques et de bascules. Les FPGA de dernière génération intègrent même de la mémoire vive (RAM). Les principaux fabricants de FPGA sont XILINX et ALTERA.
- Un FPGA est organisé sous forme d'une matrice de blocs logiques interconnectés, dont le nom varie selon les fabricants :
  - ❑ Configurable Logic Block (CLB) chez Xilinx.
  - ❑ Logic Array Block (LAB) chez Intel.
- Chaque bloc contient des cellules logiques configurables séparément pour réaliser des fonctions logiques spécifiques.
- Un bloc logique est découpé en tranches (slices).
- Une tranche contient des cellules logiques.
- Des blocs d'entrée/sortie (IOB – Input/Output Block) assurent la configuration des liaisons entre les blocs logiques et les broches du circuit.
- Les blocs logiques et les blocs d'entrée/sortie sont interconnectés via un réseau de routage programmable.

7- Circuits programmables FPGA



7- Circuits programmables FPGA

7-2 Structure de base d'un bloc logique

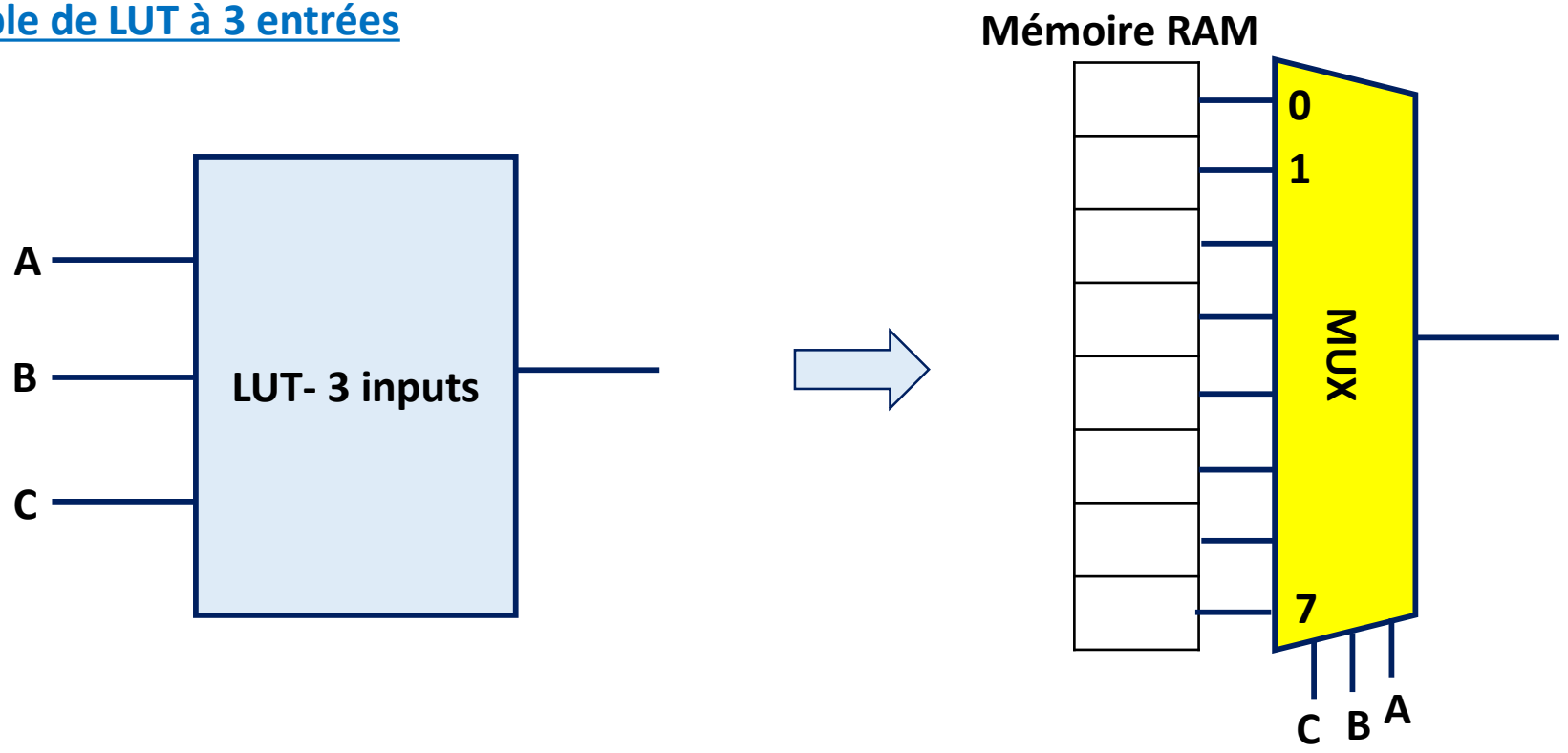


7- Circuits programmables FPGA

❑ Table de correspondance (LUT: Look Up Table)

- Les LUT (Look-Up Tables) dans les FPGA sont des blocs de mémoire qui implémentent la logique combinatoire en stockant la table de vérité de n'importe quelle fonction logique.
- Le nombre et la taille des LUT définissant la puissance de calcul des circuits FPGA.

Exemple de LUT à 3 entrées

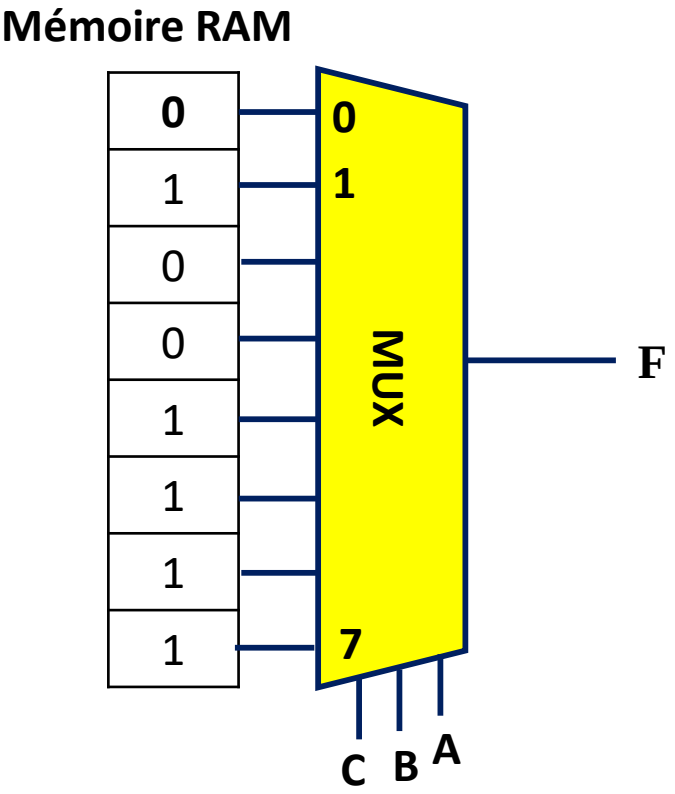


7- Circuits programmables FPGA

❑ Table de correspondance (LUT: Look Up Table)

Exemple: implémentation d'une fonction logique dans un LUT

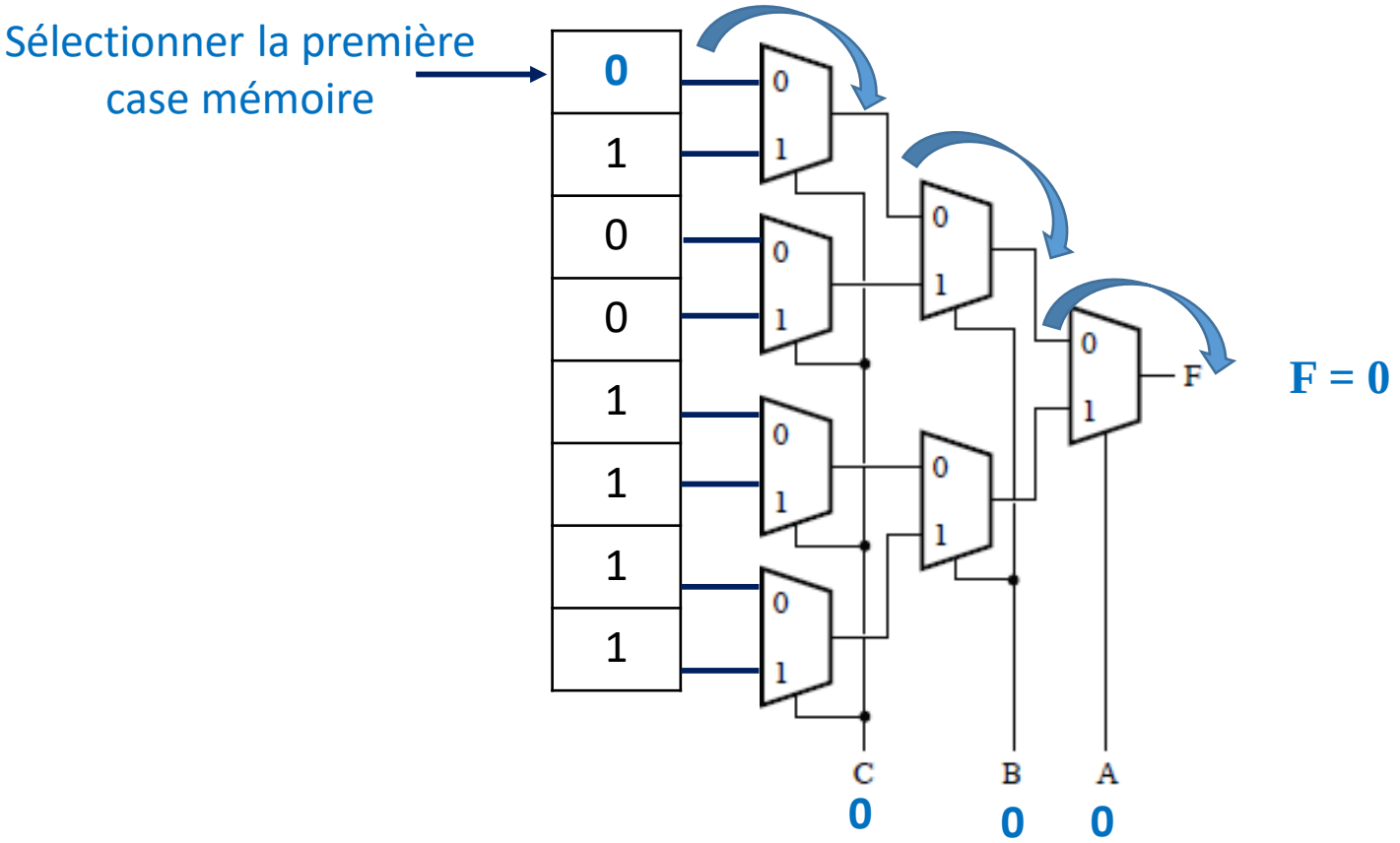
C	B	A	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



7- Circuits programmables FPGA

❑ Table de correspondance (LUT: Look Up Table)

Exemple: implémentation d'une fonction logique dans un LUT



7- Circuits programmables FPGA

❑ Exemple de circuits FPGA

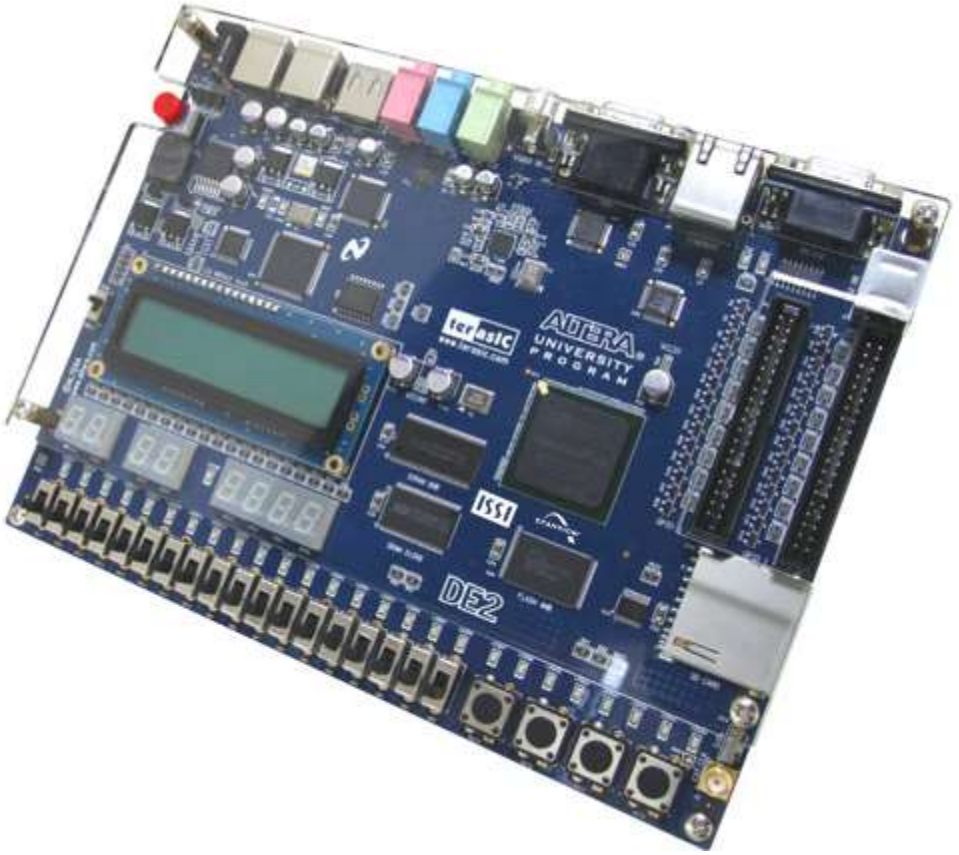
- Les principaux fabricants de FPGA sont:
  - ➢ AMD (via l'acquisition de Xilinx) et Intel (qui a acquis Altera), qui dominent le marché.
  - ➢ D'autres fabricants comme Lattice Semiconductor, Microchip Technology…





7- Circuits programmables FPGA

❑ Exemple de cartes FPGA





Chapitre IV :
Introduction au langage VHDL

1- Définition

- Le langage **VHDL (Very high speed circuit Hardware Description Language)** est un Langage de description matérielle utilisé pour modéliser, simuler et synthétiser des circuits numériques.

2- Structure d'un code VHDL

❖ Déclaration des bibliothèques

- Le programme VHDL commence par la déclaration de la bibliothèque IEEE par utilisation de la commande library IEEE
- La bibliothèque IEEE contient l'ensemble de paquetages utilisés pour décrire les composants logiques.
- Parmi les paquetage indispensable pour les composants logiques, on trouve: STD_LOGIC_1164; STD_LOGIC_UNSIGNED; STD_LOGIC_ARITH; NUMERIC_STD…
- Pour utiliser un paquetage, on utilise le mot clé « use »
- Pour utiliser l'intégralité d'un paquetage on utilise le suffixe «.all »

❑ Exemple de déclaration d'une bibliothèque

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

2- Structure d'un code VHDL

❖ Déclaration de l'entité

- Une entité définit les entrées et les sorties d'un composant

❑ Syntaxe d'une entité

```
entity nom de l'entité is
Port ( description des signaux);
end nom de l'entité;
```

❖ Déclaration de l'architecture

- L'architecture décrit le fonctionnement interne d'un circuit.

❑ Syntaxe d'une architecture

```
architecture nom de l'architecture of nom de l'entité is
   -- Signaux et constantes internes
   -- Fonctions internes
    begin
   -- Instructions
   end nom de l'architecture;
```

2- Structure d'un code VHDL

- Exemple: Description VHDL d'une porte logique AND




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

Déclaration de la bibliothèque

```
entity AND is
    Port ( E1 : in STD_LOGIC;
           E2 : in STD_LOGIC;
            S :  out STD_LOGIC);
end AND;
```

Déclaration de l'entité

```
architecture Behavioral of AND is
begin
    S <= E1 and E2;
    end Behavioral;
```

Déclaration de l'architecture

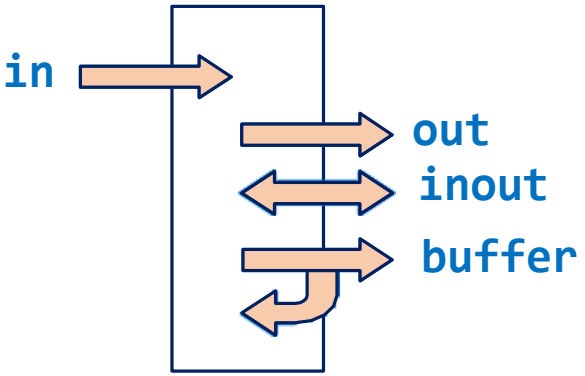
3- Signaux

- Le nom d'un signal est composé de caractères, le premier caractère doit être une lettre.
- Les lettres majuscules et minuscules sont équivalentes.

❑ Sens du signal

- Pour définir les signaux internes dans une entité, il y a quatre sens:

  - **`in:`** pour les entrées,
  - **`out`**: pour les sorties,
  - **`inout:`** pour les signaux bidirectionnels,
  - **`buffer`** : pour les signaux de sortie réentrants.



❑ Types

- Il y a plusieurs types de signaux d'entrées/sorties

  - **Bit**: deux valeurs : '0' ou '1'. Ce type est peu utilisé en synthèse et simulation.
  - **`Boolean`**: deux valeurs possibles : true ou false

3- Signaux

❑ Types

- **`STD_LOGIC:`**
  - Trois valeurs ayant un sens physique
    - **`'0'`**, **`'1'`** et **`'Z'`** pour la haute impédance.
  - Six valeurs utilisées en la simulation :
    - **`'U'`** pour non initialisé,
    - **`'X'`** pour un résultat inconnu,
    - **`'L'`** pour un niveau bas probable,
    - **`'H'`** pour un niveau haut probable,
    - **`'W'`** niveau non qualifiable par 'L' ou 'H',
    - **`'-'`** sans importance.

- **`Types logiques vectoriels`**:
  - Un vecteur de Bit défini par: **Bit_VECTOR**
  - Un vecteur de STD_LOGIC défini par: **STD_LOGIC_VECTOR**
  - Déclaration par: **STD_LOGIC_VECTOR(N-1 downto 0)** ou **STD_LOGIC_VECTOR(0 to N-1)**

3- Signaux

❑ Types

- **Exemple:**
  - A : in std_logic_vector(3 downto 0)
    A(3) désigne le poids fort et A(0) est le poids faible

A(3)	A(2)	A(1)	A(0)

  - A : in std_logic_vecto (0 to 3)
    A(0) désigne le poids fort et A(3) est le poids faible

A(0)	A(1)	A(2)	A(3)

❑ Types mathématiques

- **INTEGER:** Type entier de référence (32 bits par défaut et limité à 32 bits sur certains simulateurs).
  - Mot clé **range**: Permet de spécifier l'intervalle des valeurs d'un INTEGER au moment de sa déclaration.
    - Exemple: a: integer range 0 to 10; -- b est un entier limité à l'intervalle [0;10]
- **NATURAL :** est un sous-type d'INTEGER. C'est un entier positif ou nul
- **POSITIVE:** est un sous-type d'INTEGER. C'est un entier positif
- **REAL:**Type réel en virgule flottante simple précision à la norme IEEE754.

3- Signaux

❑ Types caractères et chaînes de caractères

- CHAR: exemple ‘a’
- STRING: chaine de caractère exemple ‘’Erreur’’

❑ Déclaration d’une constante

- En VHDL, une constante est une valeur fixe, connue à la compilation, qui ne peut jamais changer pendant la simulation ou l’exécution.

constant NOM_CONSTANTE : **TYPE** := valeur;

- TYPE peut être : integer, natural, real, std_logic, std_logic_vector, etc.valeur est fixe et ne peut pas être modifiée après la déclaration.

Exemple

```
constant taille_bus : integer := 8;

constant  C : std_logic_vector(3 downto 0) := "1010";
```

3- Signaux

❑ Conversion de type

- STD_LOGIC_VECTOR est une simple vecteur de STD_LOGIC. Donc, aucune information sur la donnée stockée.
- Pour effectuer des opérations arithmétiques, il faut passer par des types **SIGNED** ou **UNSIGNED** qui sont aussi des vecteurs de STD_LOGIC.
- Le paquetage de référence qui définit ces types est: **NUMERIC_STD (use IEEE.NUMERIC_STD.ALL;)**
- UNSIGNED: Vecteur binaire représentant un entier naturel positif.
- SIGNED : Vecteur binaire représentant un entier signé (positif ou négatif) représenté en complément à deux.
- La déclaration **use IEEE.NUMERIC_STD.ALL;** permet :
  - La définition des types SIGNED et UNSIGNED,
  - Les conversions entre INTEGER et SIGNED ou UNSIGNED.
  - La définition des opérateurs relationnels et arithmétiques pour les SIGNED et UNSIGNED, entre INTEGER et SIGNED ou UNSIGNED.

3- Signaux

❑ Conversion de type

❖ Conversions importantes

- std_logic_vector vers signed ou unsigned

```
signed(vect_y);
unsigned(vect_y);
```

- Unsigned ou signed vers std_logic_vector

```
std_logic_vector(unsigned_signal)
std_logic_vector(signed_signal)
```

- De std_logic_vector vers entire:

```
to_integer(unsigned(vect_y))
to_integer(signed(vect_y))
```

- Entier vers std_logic_vector

```
std_logic_vector(to_unsigned(10, 8)) -- 10 codé sur 8 bits
std_logic_vector(to_signed(-3, 8))   -- -3 codé sur 8 bits
```

3- Signaux

❑ Déclaration des signaux

- Un signal externe (entité) dans **port,** la syntaxe est déclaré par :

  NOM_DU_SIGNAL : sens type;

  - **Exemple: a: in** std_logic**; b : out** std_logic
  - **Exemple de déclaration multiple de signaux: a,b,c : in std_logic;**
- Un signal interne à une architecture est déclaré par:

  signal NOM_DU_SIGNAL : type;

- **Exemples:**
  - **signal A : std_logic;**
  - Déclaration multiple de signaux**: signal a,b,c : std_logic;**
  - Déclaration et initialisation d'un signal logique: signal a : std_logic := '1';
  - Déclaration d'un vecteur de signaux logiques: signal a : std_logic_vector (3 downto 0);
  - Déclaration et initialisation d'un vecteur de signaux logiques:

    signal a : std_logic_vector (3 downto 0) := ''0000'';
  - Déclaration d'un signal integer: signal a : INTEGER range 0 to 1023;

3- Signaux

N.B: - Quand on a un seul bit on le met entre cote (ex: '0', '1')
- Plus d'un bit on le met entre guillemets (''): ex '' 0011''

- Les vecteurs de bits:
  - Représentation binaire "1010"
  - Représentation octale O"12"
  - Représentation hexadécimale X"A"

4- Les opérateurs

❑ Affectation simple

- Cet opérateur permet de modifier l'état d'un signal en fonction d'autres signaux et/ou d'autres opérateurs.
- **Exemple:**

```
S1 <= not(A);
S2 <= A and B;
S3 <= A or B;
S4 <= A xor B;
```

4- Les opérateurs

❑ Opérateurs logiques

- Opérateur logiques de base : and , or , xor, not, nand, nor.
- Opérateurs de décalage à gauche : SLL
- Opérateur de décalage à droite: SRL
- Opérateur de rotation à gauche ROL (rotation de la gauche vers la droite)
- Opérateur de rotation à droite ROR (rotation de la droite vers la gauche)

Exemple :

A <= "1010011"

B <= A SLL 2	B <= 1001100
C <= A SRL 3	C <= 0001010
D <= A ROL 4	D <= 0111010
E <= A ROR 2	E <= 1110100

4- Les opérateurs

❑ Opérateurs arithmétiques

- Ces opérateurs sont utiliés pour les types `INTEGER`, `SIGNED` et `UNSIGNED`)
  - + , - , * , /, `mod` (modulo), `rem` (reste signé).

Exemple

```
signal A, B : unsigned(3 downto 0);
signal S    : unsigned(3 downto 0);
S <= A + B;
```

❑ Opérateurs relationnels

- Ces opérateurs sont utilisés pour les types `INTEGER`, `SIGNED` et `UNSIGNED`)
  - < , >, <= , >= , = , /= (différent de).

❑ Opérateur de concaténation: &

Exemple : A <= “101";
B <= “001";
C <= A & B --------→ C <= 101001

4- Les opérateurs

- Exemple : description VHDL des portes logiques de base

```
library ieee;
use ieee.std_logic_1164.all;

-- déclaration de l'entité
entity portes_logiques is
port( A,B: in std_logic;
      S1, S2, S3, S4, S5, S6:out std_logic);
end portes_logiques;

-- description de l'architecture
architecture description of portes_logiques is
begin
S1 <= not(A);
S2 <= A and B;
S3 <= A or B;
S4 <= A xor B;
S4 <= A nand B;
S5 <= A nor B;
end description;
```

Commentaires

4- Les opérateurs

❑ Exemple : description VHDL d'un additionneur complet

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity add_complet is
   Port (
      A ,B,Cin   : in  STD_LOGIC;
      S,Cout     : out STD_LOGIC
);
end add_complet;
```

```
architecture description of add_complet is
begin
   S    <= A xor B xor Cin;
   Cout <= (A and B) or (Cin and (A xor B));
end description;
```



4- Les opérateurs

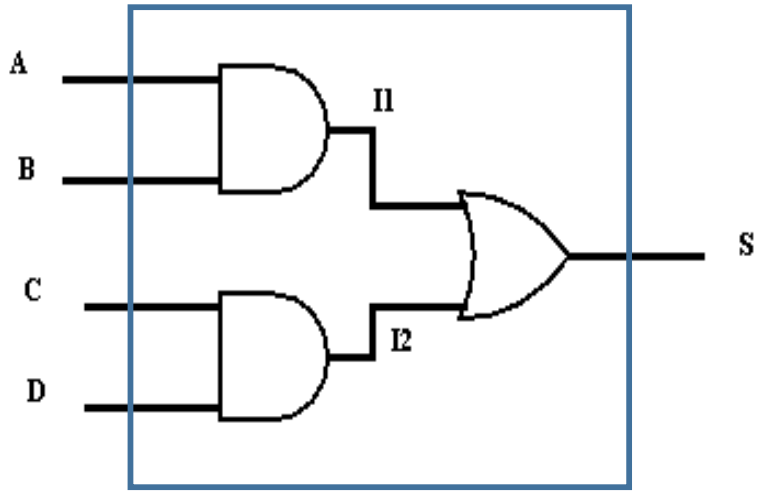
- Exemple : description VHDL d'un circuit logique avec signaux internes

```
library ieee;
use ieee.std_logic_1164.all;

-- déclaration de l'entité
entity circuit is
port( A,B,C,D: in std_logic;
      S:out std_logic);
end circuit;

-- description de l'architecture
architecture description of circuit is
-- Déclaration des signaux internes
signal I1,I2: std_logic;
begin
      I1 <= A and B;
      I2 <= C and D;
       S <= I1 or I2;

end description;
```



I1 et I2 sont des signaux internes: on doit les définir à l'intérieur de l'architecture

5- Domaine concurrent

- Les instructions sont exécutées en parallèle, l'ordre n'a pas d'importance.
- Ce sont des équations logiques, des affectations de signaux, des instanciations de composants, etc.
- Situé à l'intérieur d'une architecture, mais en dehors des process (voir domaine séquentiel).

❑ Instructions du domaine concurrent

- ❖ Affectation simple comme vu précédemment.
- ❖ Affectation conditionnelle:

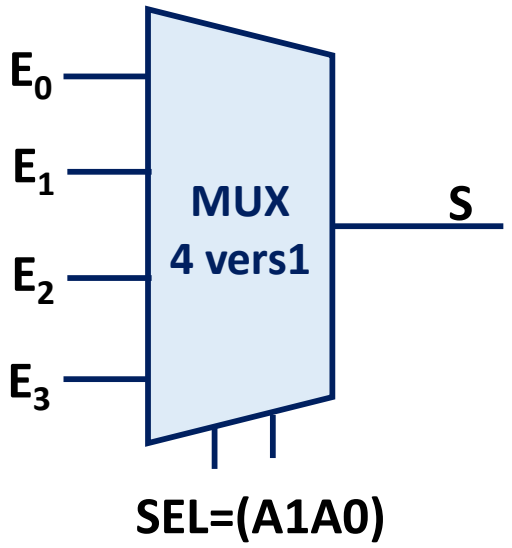
```
Signal <= expression when condition
else expression when condition
else expression when condition
else expression ;
```

Remarque : la dernière ligne "else expression" n'est pas obligatoire mais elle est fortement conseillée.

5- Domaine concurrent

- Exemple d'affectation conditionnelle: multiplexeur quatre vers un.

```
Library ieee;
USE iee.std_logic_1164.all;
Entity mux is
Port ( E0,E1,E2,E3 : in std_logic;
SEL : in std_logic_vector (1 downto 0);
S : out std_logic );
End mux;
Architecture description of mux is
Begin
S <= E1 when SEL = "00"
Else E2 when SEL = "01"
Else E3 when SEL = "10"
Else E4;
End description;
```



5- Domaine concurrent

❖ Affectation sélective: permet d'affecter différentes valeurs à un signal selon les valeurs prises par un signal de sélection.

```
With signal_de_selection select
signal <= expression when valeur_de_selection
expression when valeur_de_selection
expression when others;
```

❑ Exemple d'affectation sélective: multiplexeur quatre vers un.

```
Library ieee;
USE iee.std_logic_1164.all;
Entity mux is
Port ( E0,E1,E2,E3 : in std_logic;
SEL : in std_logic_vector (1 downto 0);
S : out std_logic );
End mux;
```

```
Architecture description of mux is
Begin
with SEL select
   S <= E0 when "00"
        E1 when "01"
        E2 when "10"
        E3 when others;
End description;
```

6- Domaine séquentiel

- Le domaine séquentiel regroupe toutes les instructions qui s'exécutent :à l'intérieur d'un process, d'une function, ou d'une procedure dans un ordre précis, comme un programme classique.
- Contrairement au domaine concurrent (exécuté en parallèle), ici l'ordre des instructions compte.

❑ Le process

- Un process est un bloc qui contient des instructions séquentielles.
- Il permet de décrire du comportement, comme dans un langage de programmation.
- L'exécution d'un process est déclenché par un ou des changements d'états de signaux logiques. Le nom de ces signaux est défini dans la **liste de sensibilité** lors de la déclaration du process.
- Les process peuvent décrire aussi des circuits combinatoires.

```
[nom_du_process :] Process ( liste_de_sensibilite )
Begin
… instruction_du_process … ;
End process;
```

- Le nom du process est facultatif.

6- Domaine séquentiel

❑ Les variables

- Une variable ne peut être déclarée et utilisée que dans un process.
  - Déclaration avant le begin de du process.

```
process(liste_de_sensibilite)
   variable nom_variable : type_variable := valeur_initiale;
begin
```

  - Une variable n'existe qu'à l'intérieur d'un process et disparait au moment de la sortie du process.
  - La variable n'a pas un sens physique.
  - Le comportement d'une variable est différent des signaux, dont l'affectation est programmée pour la fin du process.
  - L'affectation d'une variable est instantanée.

Exemple

```
X := 5;
Y := X + 3;   -- ici Y vaut 8 immédiatement
```

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

- Ces instructions sont utilisées à l'intérieur d'un process

❖ Affectation simple

❖ Instruction de choix avec « if »

```
if condition then
   -- actions du then
end if;
```

```
if condition then
   -- actions du then
else
   -- actions du else
end if;
```

```
if condition then
   -- actions du then
elsif condition then
   -- actions du elsif then
else
   -- actions du elsif else
end if;
```

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

❖ Instruction « case »

- Cette instruction permet le choix entre plusieurs actions.
- Elle utilise aussi l'instruction WHEN.
- En fin de liste, on peut ajouter WHEN OTHERS qui permet de donner une action à tous les choix qui n'ont pu être trouvés dans la liste.

```
case signal de sélection is
       when valeur de sélection 1 =>  Instruction;
       when valeur de sélection 2 => Instruction;
        .
        .
        .
        when others => Instruction;
        end case;
```

- On peut combiner dans un même process les instructions « if » et « case ».

6- Domaine séquentiel

❑ Les instructions du domaine séquentiel

❖ Instruction « for…loop »

- Cette une instruction répétitive de plusieurs actions.

```
FOR parametre IN intervalle (Ex : 0 to 7)
LOOP
instruction;
End LOOP;
```

❖ Instruction « After»

- AFTER spécifie un temps de réponse d'un signal par rapport à son événement.

```
signal_name <=  expression AFTER time_expression
```

Exemple

S <= A and B AFTER 10 ns; -- temps de propagation est de 10ns

6- Domaine séquentiel

- Exemples de description VHDL des circuits combinatoires par utilisation de process
- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « if »

  - Avec if….. Then…elsif….

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      if SEL = "00" then
         S <= E0;
      elsif SEL = "01" then
         S <= E1;
      elsif SEL = "02" then
         S <= E2;
      elsif SEL = "11" then
         S <= E3;
      else
         S <= 'X'; -- si sel n'est ni à 1 ni à 0, on affecte X à S
      end if;
   end process;
End comportement;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits combinatoires par utilisation de process
- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « if »

- Avec if….. then

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      -- valeur par défaut obligatoire
      S <= 'X';
      if SEL = "00" then
         S <= E0;
      end if;
      if SEL = "01" then
         S <= E1;
      end if;
      if SEL = "10" then
         S <= E2;
      end if;
      if SEL = "11" then
         S <= E3;
      end if;
   end process;
End comportement;
```

6- Domaine séquentiel

- Exemple : description VHDL d'un multiplexeur 4 vers 1 par utilisation d'un process et « Case »

```
Library ieee;
USE ieee.std_logic_1164.all;
Entity mux is
   Port (
      E0, E1, E2, E3 : in  std_logic;
      SEL : in  std_logic_vector (1 downto 0);
      S: out std_logic
   );
End mux;
```

```
Architecture comportement of mux is
Begin
   process(E0, E1, E2, E3, SEL)
   begin
      case SEL is
         when "00" => S <= E0;
         when "01" => S <= E1;
         when "10" => S <= E2;
          when "11" =>  S <= E3;
          when others =>   S <= 'X';
end case;
   end process;
```

6- Domaine séquentiel

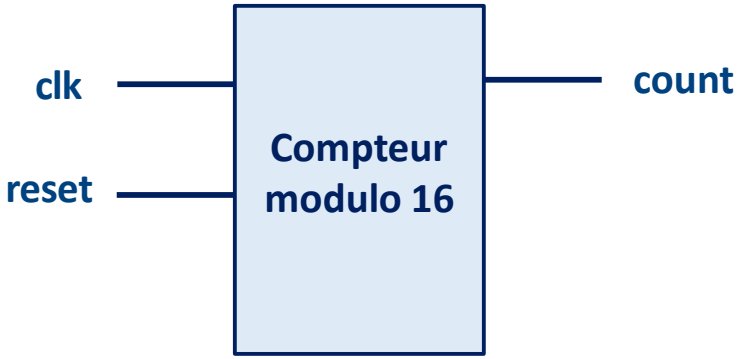
- Exemples de description VHDL des circuits séquentiels par utilisation de process
- Exemple : description VHDL d'une bascule D

- Le signal d'horloge CLK fait partie de la liste de sensibilité.
- Un changement d'état du signal CLK va déclencher le process et par conséquent évaluer les instructions de celui-ci.
- L'instruction « **rising_edge(CLK)** » permet de détecter un front montant du signal CLK. On peut utiliser aussi l'instruction « **CLK'event and CLK='1'** »
- Pour détecter un front descendant, on utilise l'instruction « **falling_edge(CLK)** » ou l'instruction « **CLK'event and CLK='0'** »

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Bascule_d is
   Port (
      D , CLK : in  STD_LOGIC;
       Q   : out STD_LOGIC
   );
end Bascule_d;
architecture description of Bascule_d is
begin
   process(CLK)
   begin
      if rising_edge(CLK) then
         Q <= D;
      end if;
   end process;
end description;
```

6- Domaine séquentiel

- Exemples de description VHDL des circuits séquentiels par utilisation de process
- Exemple : description VHDL d'un compteur modulo 16




```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity compteur_modulo16 is
   Port (
      clk, reset  : in  STD_LOGIC;
     count : out STD_LOGIC_VECTOR(3 downto 0) );
end compteur_modulo16;
```

```
architecture description of compteur_modulo16 is
   signal compteur : unsigned(3 downto 0) := (others => '0');
begin
   process(clk, reset)
   begin
      if reset = '1' then
         compteur <= (others => '0');
elsif rising_edge(clk) then
         if compteur = "1111" then
            compteur <= (others => '0');
         else
            compteur <= compteur + 1;
         end if;
      end if;
   end process;
   count <= std_logic_vector(compteur);  -- Conversion en std_logic_vector
end description;
```

7- Différentes descriptions VHDL

- En modélisation VHDL, on trouve trois principales types de descriptions: flot de données, comportementale et structurelle.

❑ Description flot de données

- Description de la manière dont les données circulent de signal en signal, ou d'une entrée vers une sortie.
- Utiliser des instructions concurrentes d'assignation de signal.
- Trois types d'instructions :
  - ➢ Affectation simple, description par équations logiques,
  - ➢ when…else, with…select

❑ Description comportementale

- La description comportementale décrit le fonctionnement d'un circuit en se basant sur sa fonctionnalité plutôt que sur sa structure interne.
- L'architecture est décrite comme une suite d'instructions, comme en langages de programmation traditionnels.
- Elle fait appel à des process.

7- Différentes descriptions VHDL

❑ Description structurelle

- Pour instancier (interconnecter) les composants dans une description structurelle, on utilise le mot clé " port map" comme suit:
- Pour declarer un composant, on utilise le mot clé "component" comme suit:

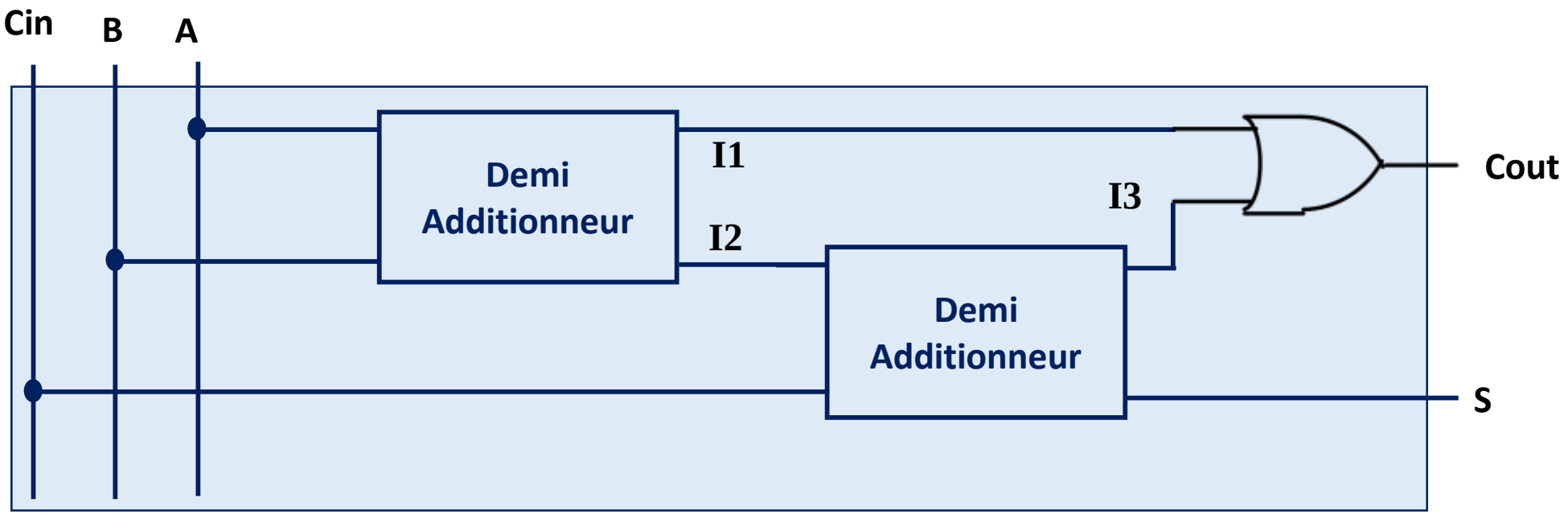
```
component Nom_ component
port (
description des signaux);
end component;
```

- La description structurelle permet d'assembler plusieurs blocs fonctionnel (composants) écrits en VHDL.

```
<NOM_INSTANCE>:<NOM_COMPOSANT> port map (LISTE DES CONNEXIONS);
```

7- Différentes descriptions VHDL

- Exemple de Description structurelle d'un additionneur complet avec des demi-additionneurs



7- Différentes descriptions VHDL

- Exemple de Description structurelle d'un additionneur complet avec des demi-additionneurs

VHDL porte OR

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity demi_add is
   port(
      A, B : in  std_logic;
      Cin    : out std_logic;
      S    : out std_logic
   );
end demi_add;

architecture description of demi_add is
begin
   S <= A xor B;
   Cin <= A and B;
end description;
```

VHDL demi-additionneur

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity demi_add is
   port(
      A, B : in  std_logic;
      Cin    : out std_logic;
      S    : out std_logic
   );
end demi_add;

architecture description of demi_add is
begin
   S <= A xor B;
   Cin <= A and B;
end description;
```

7- Différentes descriptions VHDL

Description structurelle VHDL additionneur complet

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity add_complet is
   port(
      A, B, Cin : in  std_logic;
      S, Cout   : out std_logic
   );
end add_complet ;
architecture structural of add_complet  is
   component demi_add
      port(
         A, B : in  std_logic;
         Cin    : out std_logic;
         S    : out std_logic
      );
   end component;
   component porte_or
      port(
         X, Y : in  std_logic;
         Z    : out std_logic
      );
   end component;
```

Déclaration des composants

Déclaration des signaux internes

```
signal I1, I2, I3 : std_logic;

Begin

Ad1 : demi_add
      port map(A, B, I1, I2);
Ad2 : demi_add
      port map(I2, Cin, I3, S);
OR1 : porte_or
      port map(I1, I3, Cout);
end structural;
```

Instanciation des composants